

Tetris! Traceable Extendable Threshold Ring Signatures and More

Gennaro Avitabile¹, Vincenzo Botta², and Dario Fiore¹

¹ IMDEA Software Institute, Madrid, Spain. {gennaro.avitabile,dario.fiore}@imdea.org

² Sapienza University of Rome, Rome, Italy. vincenzo.botta@uniroma1.it

Abstract. Traceable ring signatures enhance ring signatures by adding an accountability layer. Specifically, if a party signs two different messages within the protocol, their identity is revealed. Another desirable feature is *extendability*. In particular, *extendable threshold* ring signatures (ETRS) allow to *non-interactively* update already finalized signatures by enlarging the ring or the set of signers. Combining traceability and extendability in a single scheme is unexplored and would offer a new tool for privacy-preserving voting schemes in scenarios where the voters are not known in advance. In this paper, we show how to reconcile both properties by introducing and constructing a new cryptographic primitive called Tetris. Notably, our Tetris construction simultaneously achieves strong anonymity and linear-size signatures, which is the main technical challenge in existing techniques. To solve this challenge, we develop a new approach to traceability that leads to several conceptual and technical contributions. Among those, we introduce and construct, based on Groth-Sahai proofs, *extendable* shuffle arguments that can be *non-interactively* updated by several provers.

Keywords: Ring Signatures · Traceability · Malleable Proof Systems.

1 Introduction

A ring signature scheme allows a signer to generate a signature on behalf of a publicly known group of potential signers called ring $\mathcal{R}$ [46]. In ring signatures, the identity of the signer is hidden among all the possible signers in the ring. Ring signatures are crucial in many applications including anonymous authentication [43], privacy-protecting cryptocurrencies [49], whistleblowing [46], and e-voting [50,47]. An interesting feature of ring signatures is that, unlike group signatures [25,12], they operate without the need of an online central authority. This feature is extremely useful in decentralized scenarios where such authority simply does not exist. However, ring signatures offer an unlimited level of anonymity which is not always desirable. For example, consider a simple voting protocol where voters have to pick between two choices. To cast a vote, a user simply produces a signature of the desired choice using their own secret key. A dishonest voter might exploit the high level of anonymity of ring signatures to cast a double vote without being detected. To tackle these issues, variants of ring signatures known as *linkable* ring signatures [40] and *traceable* ring signatures have been proposed [30,31]. The former allows one to establish whether two signatures have been produced by the same user or not. The latter has additional features. In traceable ring signatures, a signature on a message m is also issued w.r.t. a topic τ (e.g., a unique identifier of an election). Given two valid signatures w.r.t. the same topic τ and different messages, one can trace the public key of the signer who generated both signatures. If τ and m are equal in both signatures, the tracing procedure just reveals that they were both generated by the same signer, without revealing their identity. On the other hand, signatures issued w.r.t. different topics are unlinkable. A traceable ring signature must also be *exculpable*, meaning that it is unfeasible to produce two signatures to *falsely* accuse a user of having signed two different messages on the same topic. The level of accountability offered by traceability is strictly stronger than linkability. Indeed, aside from preventing malicious behavior, they enable identification of any dishonest party.

Threshold ring signatures generalize ring signatures allowing $t \geq 1$ signers to hide their identity within a ring of size $n \geq t$. The signature guarantees that at least t *different* signers in the ring signed the message without revealing which ones. Generally, threshold ring signatures require fixing the ring and the threshold at signature generation time with no possibility of further modifying them without a new intervention of the signers themselves. This is a significant limitation as all the potential signers must be known in advance.

More recently, the concept of Extendable Threshold Ring Signatures (ETRS) was introduced in [5]. ETRS allow to non-interactively update a threshold ring signature on a certain message so that the updated signature has a greater threshold and/or an augmented ring. This is done via two operations named *join* and *extend* respectively. A key property of ETRS is the *strong anonymity* notion defined in [8]. The adversary can see all the signature’s updates resulting from threshold increments and ring extensions that a signature undergoes from when it is generated up until the protocol ends. This property is crucial for voting since all the updates are publicly available. Despite the main application of ETRS being voting (when participants are not known in advance), all the known ETRS providing even the weaker notion of linkability (i.e., [4, Sec. 5.3]) have “quadratic” $O(tn)$ size.

1.1 Our Contributions

We formalize and construct the first Traceable Extendable Threshold Ring Signature ($\mathfrak{Tetr}$). Our first contribution is a general definition of traceable ring signatures that takes into account the threshold setting as well as extendability. Indeed, to the best of our knowledge, previous definitions of traceability are only for (plain non-threshold i.e., with $t = 1$) ring signatures and allow tracing only between signatures having the same ring [30]. Simple adaptations of previous approaches fail at capturing the idea that *all* the misbehaving signers should be discovered (and not just a subset of them). Therefore, we adopt a fundamentally different approach in defining traceability (see Sec. 3 for more details).

The main technical challenge we solve is to construct a linear-size $\mathfrak{Tetr}$. Indeed, there are straightforward solutions to build a $\mathfrak{Tetr}$ with signatures of “quadratic”, $O(tn)$, size. On the other hand, linear-size traceability techniques for non-extendable threshold ring signatures are not applicable as they inherently break strong anonymity. Towards our $\mathfrak{Tetr}$, we introduce and construct the following building blocks:

- We propose the notion of extendable non-interactive proof systems (EP). EP model proof systems for a generalized threshold relation (e.g., they cover shuffle arguments), where a proof for a certain statement x can be updated into a new proof for a different but related statement x' . Our notion can be seen as a generalization, also adding new properties, of the ENIW of [8]. Among those, we introduce a notion of zero knowledge for extendable proof systems and a mild, but useful, notion of extractability that allows to perfectly extract *a part of* the witness.
- We introduce and construct extendable shuffle arguments, namely arguments proving that a t -size list of clear-text elements is a permuted subset of the values encrypted in a list of $n \geq t$ ciphertexts. The proof can be updated by several (independent) provers that can re-randomize the ciphertexts, add a ciphertext to the list, or reveal a new clear-text element.
- We introduce and construct doubly-authentication-preventing tags (DAPT) which are *deterministic* tags that are tied to a public key, a topic, and a message. These tags are anonymous when issued on different topics, but two tags generated with the same key, on the same topic, and different messages reveal the corresponding public key.

1.2 Technical Overview

We first discuss some approaches to traceability and their shortcomings. Then, we give a high-level description of our techniques and how we use them to overcome those issues.

A simple quadratic solution. A candidate technique to get a $\mathfrak{Tetr}$ is to adapt a widely known compiler to transform *linkable* ring signatures into *linkable threshold* ring signatures supporting the join operation. In a nutshell, whenever two linkable ring signatures over the same message and ring do not link to the same signer, they can be concatenated together to produce a 2-out-of- n signature. As a result, a signature with threshold t is composed of t ring signatures. Similarly, two traceable ring signatures can be concatenated if they do not trace to any public key in the ring. Therefore, applying this compiler starting from a *traceable ring* signature naturally gives a *traceable threshold* ring signature equipped with the join operation. If the base traceable ring signature also supports the extend operation, then we immediately get a $\mathfrak{Tetr}$. Thus, a viable approach

to get a $\mathfrak{Tetris}$ could involve modifying an existing traceable ring signature into one supporting extensions, and then applying the above compiler to get a $\mathfrak{Tetris}$. Unfortunately, currently known approaches [5,8] to get extendability are inherently linear in n , leading to a base extendable traceable ring signature with size $O(n)$ and thus to a $\mathfrak{Tetris}$ of size $O(tn)$.

Towards a linear-size $\mathfrak{Tetris}$. A natural starting point is to combine a strongly anonymous $O(n)$ -size ETRS [8] with traceability techniques of previous works [30,29,18]. The key idea of these works is to publish a pseudo-random tag for each of the public keys in the ring. Whenever $\mathbf{pk}_i$ is one of the signers, its tag is uniquely determined by $\mathbf{pk}_i$ and the topic τ , while the tags of non-signers are just placeholders indistinguishable from legitimate tags. To trace a pair of signatures, it suffices to look for identical tags in both signatures. However, this approach is at odds with strong anonymity. Indeed, an adversary who has access to the full evolution of a signature can easily guess which signers have performed a join operation along the way. This is because once a *new* signer joins a signature, it is necessary to replace its placeholder tag with a new one as there exists only one valid tag for a key that is in the set of signers. Moreover, for the same reason, it is not possible to change the tags related to the previous signers. Given a pair of signatures (σ_{j-1}, σ_j) before and after the j -th join operation, the signers of σ_{j-1} are the public keys corresponding to the tags that have remained unchanged in σ_j . If the tags were re-randomizable, one could avoid such an attack by re-randomizing the list of tags after every join operation so that every signature in the sequence contains seemingly unrelated tags. However, since the tracing algorithm must work exclusively using publicly available information, it seems unlikely that one could come up with re-randomizable traceability tags that when compared reveal the dishonest signers.

An alternative approach is to break the link between tags and public keys and publish only t tags (i.e., one tag per signer). The signature also contains a NIZK proof³ that the t tags come from t public keys in the ring, while hiding the actual correspondence between individual tags and public keys. Notice that the above attack is not applicable anymore, as no information is published about non-signers. We remark that the NIZK proof should be *extendable*, allowing future signers to extend the ring or to increase the threshold by providing a new tag. One could use extendable proofs for disjunctions and prove the above statement via t disjunctive proofs stating that each of the t tags comes from one of the n public keys in the ring. Known extendable proofs ([8]) support disjunctive threshold relations, that are proofs stating that t out of n base statements $(x_1, \dots, x_n)$ are in a base language $\mathcal{L}$. These proofs have size $O(nS)^4$, where S is the size of a proof for $x_i \in \mathcal{L}$. For example, such relations can be used to express the above predicate via t disjunctive proofs stating that each of the t tags is generated w.r.t the topic τ and the message m using one of the public keys $\mathbf{pk}_1, \dots, \mathbf{pk}_n$. This approach gives again a signature of size $O(tn)$ (i.e., t proofs of size nS , with S independent from t and n).

In this work, we start from the idea of publishing only t tags and we build new tools allowing us to instantiate the above template to get $O(n)$ -sized signatures providing both traceability and strong anonymity. In the next paragraphs, we give an high-level overview of such building blocks and how we combine them.

Extendable non-interactive proof systems (EP). We propose the notion of extendable non-interactive proof systems (EP). EP model proof systems for a generalized threshold relation and are inspired from the ENIWI of [8]. ENIWI are defined w.r.t. a threshold relation, which is in turn defined w.r.t. a poly-time relation $R_{\mathcal{L}}$ as $R_{\mathcal{L}_{tr}} = \{(x = (k, x_1, \dots, x_n), w = ((w_1, \alpha_1), \dots, (w_k, \alpha_k))) | 1 \leq \alpha_1 < \dots < \alpha_k \leq n \wedge \forall j \in [k] : (x_{\alpha_j}, w_j) \in R_{\mathcal{L}}\}$. Let $\mathcal{L}_{tr}$ be the corresponding NP-language and let us call *active* statements the statements that are in the set $\{x_{\alpha_j}\}_{j \in [k]}$; the other statements are called *inactive*. In words, the prover wants to prove that k *different* statements out of n are in the language $\mathcal{L}$. An ENIWI supports two kinds of transformations called *extend* and *add* operations:

³ In this work, we interchangeably use the word proof and argument. Generally, unless specified, soundness is assumed to hold against a PPT adversary.

⁴ Although using SNARKs in a recursive fashion one might get succinct extendable proofs [24,22], those proof systems are costly in terms of proving time and memory and rely on strong assumptions. We are instead interested in efficient provers and more standard assumptions.

- **Extend**: transform a proof for $(k, x_1, \dots, x_n) \in \mathcal{L}_{tr}$ into a proof for $(k, x_1, \dots, x_n, x_{n+1}) \in \mathcal{L}_{tr}$.
- **Add**: transform a proof for $(k, x_1, \dots, x_n) \in \mathcal{L}_{tr}$ into a proof for $(k+1, x_1, \dots, x_n) \in \mathcal{L}_{tr}$.

The extend operation can be executed without involving any private input from the previous provers. However, the same does not apply to the add operation. When a prover computes a proof Π for a statement $x = (k, x_1, \dots, x_n) \in \mathcal{L}_{tr}$, it generates auxiliary values $AUX = (\text{aux}_1, \dots, \text{aux}_n)$ alongside the proof. The auxiliary value aux_i is used later to perform the add operation through an additional algorithm named PAdd. Given an accepting proof Π for $(k, x_1, \dots, x_n) \in \mathcal{L}_{tr}$, a witness w_i for an unused index i where $(x_i, w_i) \in R_{\mathcal{L}}$, and the corresponding auxiliary value aux_i , PAdd outputs a proof Π' for $(k+1, x_1, \dots, x_n) \in \mathcal{L}_{tr}$. Similarly, the algorithm PExt is employed for the extend operation, which does not need any auxiliary value. When provided with an accepting proof for $(k, x_1, \dots, x_n) \in \mathcal{L}_{tr}$ and a statement x_{n+1} , PExt produces a proof Π' for $(k, x_1, \dots, x_{n+1}) \in \mathcal{L}_{tr}$ and the auxiliary value aux_{n+1} associated with the statement x_{n+1} . This auxiliary value aux_{n+1} can subsequently be used to perform an add operation using witness w_{n+1} s.t. $(x_{n+1}, w_{n+1}) \in R_{\mathcal{L}}$. In [8] the notion of extended witness indistinguishability is introduced. It is a witness indistinguishability (WI) notion which allows the adversary to additionally get *some of* the auxiliary values.

The notion of EP supports more expressive types of threshold relations (see Sec. 4), for example it also models shuffle arguments. Unlike ENIWI, EP also allow to transform (e.g., re-randomize) the statements. Namely, after an extend/add operation, the base statements $x_1, \dots, x_n$ can be transformed into different (but related according to a specific transformation) statements $x'_1, \dots, x'_n$. Additionally, we introduce a definition of extended zero knowledge with a similar spirit to extended witness indistinguishability and a mild, but useful, notion of extractability (reminiscent of F-extractability [28, Def. 3]) that allows to perfectly extract what are the active statements along with *a function of* (part of) the rest of the witness (see Sec. 4).

ENIWI internals. To give a better understanding of our tools, we briefly describe the ENIWI of [8], which is based on the remarkable malleability of the Groth-Sahai (GS) proof system [38,23]. GS proofs are a commit-and-prove framework to prove the satisfiability of several types of equations over bilinear groups. In GS, secret variables are committed, and the prover generates proof elements from the committed values and commitment randomnesses. The proof is verified based on the statement, commitments, and proof elements. More concretely, partial knowledge of satisfying assignments for k out of n equations is proved by introducing additional *binary* “switch” variables bit_i s.t. when $\text{bit}_i = 1$ the i -th equation is left unaltered, while when $\text{bit}_i = 0$ the i -th equation admits the trivial solution, thus allowing for simulation. Then, an additional equation proving that $\sum_{i=1}^n \text{bit}_i = k$ guarantees that only $n - k$ equations can be simulated, while the prover must hold a satisfying assignment for k of them.

The core idea of [8] is the observation that the proof elements of a GS proof are computed by linearly combining, along with some randomizers, the committed variables and the commitment randomnesses. This means that given (a function of) the values and the randomnesses of some committed variables, it is always possible to erase their contribution from a proof element in order to replace the old variables with freshly committed ones, assuming that the new assignment of the variables satisfies the new equation being proven⁵.

Roughly speaking, auxiliary values correspond to commitment openings for these switch variables bit_i allowing to update a proof for $\sum_{i=1}^n \text{bit}_i = k$ into a proof for $\sum_{i=1}^n \text{bit}_i = k+1$ without knowledge of the other bit_j with $j \neq i$.

A key observation that we use is that several extendable proof systems can be connected to each other by sharing the same switch variables; this guarantees that the active indices (i.e, $i \in [n]$ s.t. $\text{bit}_i = 1$) are the same in all proofs.

Extendable shuffle arguments. We introduce and construct an EP for the shuffle relation that we call extendable shuffle argument. The goal of an extendable shuffle argument is to prove that each element e_i of a public list $\{e_1, \dots, e_k\}$ of $k \leq n$ elements is the value committed in a *different* commitment in a list $\{c_1, \dots, c_n\}$ of n commitments. For $k = n$ the relation proved by an extendable shuffle argument coincides with a shuffle argument to known values (e.g., [3,44]), where one proves that a set of public values is obtained

⁵ Even though the techniques shown in [8] are applied only to prove k out of n Pairing-Product Equations (PPEs), the same techniques apply with minimal adjustments to all the other equation types supported by GS.

by first permuting and then opening a set of public commitments. Formally, the relation for the extendable shuffle argument is the following:

$$R_{SH} = \{(x = (c_1, \dots, c_n, e_1, \dots, e_k), w = (\phi, r_1, \dots, r_n) | \forall i \in [k] : \phi \text{ is an injective map } [k] \rightarrow [n] \wedge c_{\phi(i)} \leftarrow \text{Com}(e_i, r_i)\} \quad (1)$$

An extendable shuffle argument allows one to update previously generated proofs in the following two ways:

- **Extend:** On input a proof for $x = (c_1, \dots, c_n, e_1, \dots, e_k) \in \mathcal{L}_{sh}$, and a new commitment c_{n+1} , output a proof for $x' = (c'_1, \dots, c'_n, c'_{n+1}, e_1, \dots, e_k) \in \mathcal{L}_{sh}$.
- **Add:** On input a proof for $x = (c_1, \dots, c_n, e_1, \dots, e_k) \in \mathcal{L}_{sh}$, a new element e_{k+1} , a new commitment c'_i to e_{k+1} (along with its opening randomness) where position i has not been previously used, and an auxiliary value aux_i , output a proof for $x' = (c'_1, \dots, c'_n, e_1, \dots, e_k, e_{k+1}) \in \mathcal{L}_{sh}$.

Notice that we apply a further transformation to the statement when performing an add/extend operation by re-randomizing all the commitments (i.e., when we write c'_i we mean that c'_i is the re-randomization of c_i). This is an instantiation of our new notion of EP (see Sec. 4) that differs from the ENIWI of [8] where the base statements do not change after add/extend operations. This additional feature is crucial to guarantee the strong anonymity of our $\mathfrak{Tctris}$.

We build our extendable shuffle argument by modifying the shuffle argument by Groth and Lu [37] which is in turn based on GS proofs. We first adapt their work to asymmetric bilinear groups and then use the techniques of [8] to turn their regular shuffle into an extendable one by modifying the involved equations using switch variables (see Sec. 8 for more details). Our extendable shuffle satisfies our new notion of extended zero knowledge.

Doubly-authentication-preventing tags. We also define and construct new traceability tags that are compatible with our approach. We name such tags doubly-authentication-preventing tags (DAPT), inspired by doubly-authentication-preventing signatures (DAPS) [26,21]. A DAPS is a digital signature scheme with accountability. Messages consist of both an address and a payload component. If a signer signs two messages with identical addresses but different payloads, their secret key will be revealed. However, we cannot use DAPS as traceability tags since, being digital signatures, they are publicly verifiable and thus they reveal the public key w.r.t. they were generated.

Therefore, we define the related but different notion of DAPT. In a DAPT, a user is equipped with a public key and a secret key. There is a tag generation algorithm **Tag** that on input the secret key, a topic τ , and a message m deterministically generates a tag e . Unlike DAPS, an adversary who is only in possession of the public key cannot distinguish a valid DAPT from a random element of the tag space. Therefore, tags are not linkable to a specific signer. However, a DAPT is also provided with a tracing algorithm **TagTrace** that on input a public key and two tags generated on the same topic and different messages, outputs 1 if those tags were both generated w.r.t. the public key given in input. This property will allow us to trace malicious signers in our $\mathfrak{Tctris}$. Additionally, DAPT enjoy a non-frameability property stating that it is unfeasible to produce two tags that trace to a public key without having the corresponding secret key. This property will make our $\mathfrak{Tctris}$ exculpable.

Putting everything together. To build a $\mathfrak{Tctris}$, we combine an EP for a threshold relation with our extendable shuffle argument (SH). The public key of the $\mathfrak{Tctris}$ is composed by the public key of a DAPT and the public key of a public key encryption scheme, while the secret key simply includes the corresponding individual secret keys. To compute a signature on message m with tag τ , the first signer computes $e \leftarrow \text{Tag}(\text{sk}_i, \tau, m)$ and $c_i \leftarrow \text{Com}(e; r)$ with randomness r . Then, it generates other commitments c_j with $j \in [n], j \neq i$ by committing to 0. The signature includes e as well as all the commitments. The signer then proves using the EP that 1 out of the n commitments actually commits to a tag that was honestly generated starting from τ , m , and one of the DAPT keys in the ring. The resulting auxiliary values are individually encrypted under the public keys of the corresponding potential future signers as done in [8].

Nevertheless, this does not suffice to get traceability as the EP proof above is about the value in c_i and not the clear-text e , i.e., e might not be in any of the commitments and just be a random value. Therefore, the signer uses SH to prove that the clear-text tag e actually comes from the commitments list. However, this is still not enough. Indeed, the EP proof certifies that one commitment contains the correct tag, while the SH proof certifies that the clear-text tag e corresponds to the content of one of the commitments, but in principle there is no guarantee that the two proofs are talking about the same commitment. To solve this issue we make EP and SH share the same switch variables bit_i so that we are guaranteed that the two proofs talk about the same commitments. This assures that all the t clear-text tags of a valid $\mathfrak{T}\text{etris}$ actually correspond to honestly generated tags w.r.t. τ, m and t different keys in the ring.

The Trace algorithm of our $\mathfrak{T}\text{etris}$ simply runs the TagTrace algorithm of the DAPT on all possible triplets of tags and public keys. Finally, the extend and join operations exploit the extendability of the underlying proof systems.

1.3 Related Work

Ring signatures were introduced by Rivest et al. [46]. Several works have focused on improving the signature size [27,2,45,39,53,6,42,52,35,9], the minimal assumptions required [1,15,41,10], the security against quantum adversaries [2,13,39,19,14]. For some applications (e.g., e-voting) plain ring signatures are not enough and additional properties are required. Accountability properties based on trusted authorities were explored for both group and ring signatures [51,17,32,16]. Liu et al.[40] introduced likability allowing *anyone* to determine if multiple signature were made by the same user without revealing their identity. Fujisaki et al. [30] introduced traceable ring signatures allowing deanonymization of a signer that produces multiple signatures on different messages. Subsequent works focused on improving the size or the assumptions of traceable ring signatures [29,7,19,48]. Threshold ring signature were introduced by Bresson et al. [20], in which t members of a group of n parties can sign a message keeping their identity hidden. Aranha et al. [5] have enhanced the functionality of threshold ring signatures by proposing ETRS. Avitabile et al. [8] proposed a stricter anonymity definition for ETRS along with a new partially structure-preserving [33] ETRS based on Groth-Sahai (GS) proofs [38,34,28].

2 Notation and Preliminaries

Let $\mathcal{L}$ be an NP language, we call $R_{\mathcal{L}}$ the corresponding poly-time relation. We denote the security parameter as λ . We work over bilinear groups $\text{gk} = (p, \hat{\mathbb{G}}, \hat{\mathbb{H}}, \mathbb{T}, e, \hat{g}, \hat{h}) \leftarrow \mathcal{G}(1^\lambda)$. $\mathcal{G}(1^\lambda)$ is a generator algorithm that on input the security parameter, outputs the description of a bilinear group. We call such description group key gk . $\hat{\mathbb{G}}, \hat{\mathbb{H}}, \mathbb{T}$ are prime p order groups, $\hat{g}, \hat{h}$ are generators of $\hat{\mathbb{G}}, \hat{\mathbb{H}}$ respectively, and $e : \hat{\mathbb{G}} \times \hat{\mathbb{H}} \rightarrow \mathbb{T}$ is a non-degenerate bilinear map. In this paper, we will use additive notation for the group operations and multiplicative notation for the bilinear map e . Moreover, in some cases, we specify the randomness rand used by an algorithm A on an input x during the execution writing $A(x; \text{rand})$. For details on standard tools (e.g., commitment schemes, public key encryption, non-interactive proof systems, and Groth-Sahai proofs) see App. A.

Assumption 1 (DDH) *The Decisional Diffie-Hellman (DDH) holds in $\hat{\mathbb{G}}$ if for all PPT adversaries $\mathcal{A}$, the probability that $\mathcal{A}$ distinguishes the two distributions $(\hat{g}, \xi\hat{g}, \rho\hat{g}, \xi\rho\hat{g})$ and $(\hat{g}, \xi\hat{g}, \rho\hat{g}, \kappa\hat{g})$, where $\xi, \rho, \kappa \leftarrow \mathbb{Z}_p$ is negligible. Tuples of the form $(\hat{g}, \xi\hat{g}, \rho\hat{g}, \xi\rho\hat{g})$ are called Diffie-Hellman (DH) tuples. The DDH problem in $\hat{\mathbb{H}}$ is defined in a similar way.*

For the sake of convenience, we state the following assumption that we call Extended DDH. Notice that the Extended DDH assumption can be tightly reduced to DDH.

Assumption 2 (Extended DDH) *The Extended Decisional Diffie-Hellman (EDDH) holds in $\hat{\mathbb{G}}$ if for all PPT adversaries $\mathcal{A}$, the success probability in the experiment of figure Fig. 1 is*

$$\Pr \left[\text{Exp}_{\mathcal{A}, \hat{\mathbb{G}}}^{\text{ExtDDH}}(\lambda) = \text{win} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

$\text{Exp}_{\mathcal{A}, \hat{\mathbb{G}}}^{\text{ExtDDH}}(\lambda)$	$\mathcal{C}(b, a)$
1 : $b \leftarrow \{0, 1\}, a \leftarrow \mathbb{Z}_p, \hat{a} = a\hat{g}$	1 : $t \leftarrow \mathbb{Z}_p, \hat{t} = t\hat{g}$
2 : $b' \leftarrow \mathcal{A}^{C(b, a)}(\hat{a})$	2 : if $b = 0$:
3 : if $b' = b$:	3 : $\hat{z} = a\hat{t}$
4 : return win	4 : else :
5 : else : return lose	5 : $z \leftarrow \mathbb{Z}_p, \hat{z} = z\hat{g}$
	6 : return $(\hat{t}, \hat{z})$

Fig. 1. Extended DDH experiment.

Assumption 3 (Type 2 Extended DDH) *The Type 2 Extended Decisional Diffie-Hellman holds in $\hat{\mathbb{G}}$ if for all PPT adversaries $\mathcal{A}$, the success probability in the experiment of figure Fig. 2 is*

$$\Pr \left[\text{Exp}_{\mathcal{A}, \hat{\mathbb{G}}}^{\text{ExtDDH-2}}(\lambda) = \text{win} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

$\text{Exp}_{\mathcal{A}, \hat{\mathbb{G}}}^{\text{ExtDDH-2}}(\lambda)$	$\text{ODDH}(b, a, \text{honest})$
1 : $b \leftarrow \{0, 1\}, a \leftarrow \mathbb{Z}_p, \hat{a} = a\hat{g}$	1 : $t \leftarrow \mathbb{Z}_p, \hat{t} = t\hat{g}$
2 : $b' \leftarrow \mathcal{A}^{C(b, a, \cdot)}(\hat{a})$	2 : if $\text{honest} = 1 \vee b = 0$:
3 : if $b' = b$:	3 : $\hat{z} = a\hat{t}$
4 : return win	4 : else :
5 : else : return lose	5 : $z \leftarrow \mathbb{Z}_p, \hat{z} = z\hat{g}$
	6 : return $(\hat{t}, \hat{z})$

Fig. 2. Type 2 Extended DDH experiment.

Theorem 1. *If Ass. 2 holds w.r.t. $\hat{\mathbb{G}}$, then Ass. 3 also holds w.r.t. $\hat{\mathbb{G}}$.*

Proof. We construct an adversary $\mathcal{B}$ for Ass. 3 which internally runs $\mathcal{A}$ that breaks Ass. 2 with probability noticeably bigger than $\frac{1}{2}$. $\mathcal{B}$ forwards to $\mathcal{A}$ the $\hat{a}$ that it gets in input. Then, $\mathcal{B}$ replies to oracle queries made by $\mathcal{A}$ by making a query to its own $\mathcal{C}$ oracle. Finally, $\mathcal{B}$ outputs whatever $\mathcal{A}$ outputs. $\mathcal{B}$ perfectly simulates the view of an $\mathcal{A}$ playing in $\text{Exp}_{\mathcal{A}, \hat{\mathbb{G}}}^{\text{ExtDDH}}(\lambda)$ and thus wins in its game with the same advantage that $\mathcal{A}$ has in winning the $\text{Exp}_{\mathcal{A}, \hat{\mathbb{G}}}^{\text{ExtDDH}}(\lambda)$ game.

Assumption 4 (SXDH) *The Symmetric eXternal Diffie-Hellman (SXDH) assumption holds relative to $\mathcal{G}$ if there is no PPT adversary $\mathcal{A}$ that breaks the DDH problem in both $\hat{\mathbb{G}}$ and $\check{\mathbb{H}}$ for $\mathbf{gk} = (p, \hat{\mathbb{G}}, \check{\mathbb{H}}, \mathbb{T}, e, \hat{g}, \check{h}) \leftarrow \mathcal{G}(1^\lambda)$.*

Assumption 5 (Co-CDH) *The Co-Computational Diffie-Hellman assumption holds relative to $\mathcal{G}$ if for all PPT adversary $\mathcal{A}$ that are given $(\hat{g}, \xi\hat{g}, \rho\hat{g}, \rho\check{h})$, where $\xi, \rho \leftarrow \mathbb{Z}_p$, the probability that $\mathcal{A}$ outputs $\hat{s}$ s.t. $\xi\hat{g} \cdot \rho\check{h} = \hat{s} \cdot \check{h}$ is negligible.*

We also state the following two assumptions which we rely on in Sec. 8, namely the *subset* permutation pairing assumption and the *subset* simultaneous pairing assumption. They are inspired (and clearly implied)

by their non-subset (i.e., when $k = n$) analogues introduced in [37] for symmetric bilinear groups. The permutation pairing assumption was adapted to the asymmetric setting by González [36]. We adapt the simultaneous pairing assumption to the asymmetric setting as well.

Assumption 6 (Subset Permutation Pairing Assumption) *We say that the subset permutation pairing assumption holds if for $\mathbf{gk} = (p, \hat{\mathbb{G}}, \check{\mathbb{H}}, \mathbb{T}, e, \hat{g}, \check{h}) \leftarrow \mathcal{G}(1^\lambda)$, for all $n \in \mathbb{N}, k \leq n$, and for all PPT adversaries $\mathcal{A}$ we have:*

$$\Pr \left[\begin{array}{l} (\{\tilde{a}_i\}, \{\tilde{b}_i\}, \{\tilde{c}_i\}) \leftarrow \mathcal{A}(\mathbf{gk}, t, \{\hat{g}_i\}, \{\hat{\gamma}_i\}, \{\check{h}_i\}, \{\check{\delta}_i\}) \\ \sum_{i=1}^t \tilde{a}_i = \sum_{i=1}^t \tilde{h}_i, \sum_{i=1}^t \tilde{b}_i = \sum_{i=1}^t \tilde{\delta}_i \\ \forall i \in [t] \hat{g} \cdot \tilde{a}_i = \tilde{c}_i \cdot \check{h}, \hat{g} \cdot \tilde{b}_i = \tilde{c}_i \cdot \tilde{a}_i \\ \{\tilde{a}_i\}, \{\tilde{b}_i\} \text{ not a permutation of } \{\tilde{h}_i\}, \{\tilde{\delta}_i\} \end{array} \middle| \begin{array}{l} x_1, \dots, x_n \leftarrow Z_p, \\ \{\hat{g}_i\} = \{x_i \hat{g}\}, \{\hat{\gamma}_i\} = \{x_i^2 \hat{g}\} \\ \{\check{h}_i\} = \{x_i \check{h}\}, \{\check{\delta}_i\} = \{x_i^2 \check{\delta}\} \end{array} \right] \leq \text{negl}(\lambda).$$

Assumption 7 (Subset Simultaneous Pairing Assumption) *We say that the subset simultaneous pairing assumption holds if for $\mathbf{gk} = (p, \hat{\mathbb{G}}, \check{\mathbb{H}}, \mathbb{T}, e, \hat{g}, \check{h}) \leftarrow \mathcal{G}(1^\lambda)$, for all $n \in \mathbb{N}, k \leq n$, and for all PPT adversaries $\mathcal{A}$ we have:*

$$\Pr \left[\begin{array}{l} \{\hat{\mu}_i\} \leftarrow \mathcal{A}(\mathbf{gk}, t, \{\hat{g}_i\}, \{\hat{\gamma}_i\}, \{\check{h}_i\}, \{\check{\delta}_i\}) \\ \sum_{i=1}^t \hat{\mu}_i \cdot \check{h} = 0_{\mathbb{T}}, \sum_{i=1}^t \hat{\mu}_i \cdot \check{\delta}_i = 0_{\mathbb{T}} \\ \exists i : \hat{\mu}_i \neq 0 \end{array} \middle| \begin{array}{l} x_1, \dots, x_n \leftarrow Z_p \\ \{\hat{g}_i\} = \{x_i \hat{g}\}, \{\hat{\gamma}_i\} = \{x_i^2 \hat{g}\} \\ \{\check{h}_i\} = \{x_i \check{h}\}, \{\check{\delta}_i\} = \{x_i^2 \check{\delta}\} \end{array} \right] \leq \text{negl}(\lambda).$$

3 Traceable Extendable Threshold Ring Signatures

A $\mathfrak{Tetris}$ scheme consists of the following PPT algorithms and a poly-time relation $R_{\text{key}}^{\mathfrak{Tetris}}$. We model signatures as a pair $\sigma = (\mu, \text{tinfo})$, where we call tinfo tracing information as it is used by the Trace algorithm.

- $(\mathbf{pp}, \mathbf{td}) \leftarrow \text{Setup}(1^\lambda)$: outputs public parameters $\mathbf{pp}$ and trapdoor $\mathbf{td}$.⁶
- $(\mathbf{pk}, \mathbf{sk}) \leftarrow \text{KeyGen}()$: generates a new public and secret key pair.
- $\sigma \leftarrow \text{Sign}(\tau, m, \{\mathbf{pk}_i\}_{i \in \mathcal{R}}, \mathbf{sk})$: on input a topic τ , a message m , a secret key $\mathbf{sk}$ corresponding to a public key $\mathbf{pk}_i$ with $i \in \mathcal{R}$, returns a signature σ .
- $0/1 \leftarrow \text{Verify}(t, \tau, m, \{\mathbf{pk}_i\}_{i \in \mathcal{R}}, \sigma)$: on input a signature σ , a message m , a topic τ , a threshold t , and a set of public keys $\{\mathbf{pk}_i\}_{i \in \mathcal{R}}$; it outputs 1 to accept, and 0 to reject.
- $\sigma' \leftarrow \text{Join}(\tau, m, \{\mathbf{pk}_i\}_{i \in \mathcal{R}}, \mathbf{sk}, \sigma)$: on input a signature σ for message m and topic τ produced w.r.t. ring $\mathcal{R}$ with threshold t , and the new signer secret key $\mathbf{sk}$ (whose $\mathbf{pk}$ is in $\mathcal{R}$), outputs a new signature σ' with threshold $t + 1$.
- $\sigma' \leftarrow \text{Extend}(\tau, m, \sigma, \{\mathbf{pk}_i\}_{i \in \mathcal{R}}, \{\mathbf{pk}_i\}_{i \in \mathcal{R}'})$: extends σ with threshold t for ring $\mathcal{R}$ into a new signature σ' with threshold t for ring $\mathcal{R} \cup \mathcal{R}'$.
- $T \leftarrow \text{Trace}(\tau, m_1, m_2, \text{tinfo}_1, \text{tinfo}_2, \{\mathbf{pk}_i\}_{i \in \mathcal{R}_1}, \{\mathbf{pk}_i\}_{i \in \mathcal{R}_2})$: It is a *deterministic* algorithm. On input a topic τ , two messages and tracing information pairs $(m_1, \text{tinfo}_1), (m_2, \text{tinfo}_2)$, it outputs T that takes one of the following values: (i) *indep*; (ii) *(linked, n)*; (iii) a set $\mathbf{PK} = \{\mathbf{pk}_i\}_{i \in \mathcal{R} \subseteq \mathcal{R}_1 \cap \mathcal{R}_2}$. Where *indep* means that there are no common signers, *(linked, n)* means that $m_1 = m_2$ and there are n common signers, and when $m_1 \neq m_2$, $\mathbf{PK}$ is the set of traced keys.

We use the notion of ladder introduced in [5]. A ladder lad is a sequence of tuples $(\text{action}, \text{input})$, where action takes a value in the set $\{\text{Sign}, \text{Join}, \text{Extend}\}$ and the value of input depends on the value of action . If $\text{action} = \text{Sign}$, then input is a pair $(\mathcal{R}, i)$, where $\mathcal{R}$ is the ring for the signature and i is the signer's identity. If $\text{action} = \text{Join}$, then input is an identifier i that identifies the signer joining the signature. If $\text{action} = \text{Extend}$, then input is a ring $\mathcal{R}$ that is the ring to use to extend the previous one. We use $\text{lad}.\mathcal{S}$ to indicate the set of signers of a ladder lad . We notice that a ladder unequivocally determines a sequence of signatures, each one with a specific ring and threshold value. We also define the Proc algorithm, whose description is in (Fig. 3). Proc takes as input a topic, a message, a ladder, and a list of keys, and outputs the sequence of all the signatures corresponding to each step of the ladder. It outputs $\perp$ whenever the ladder is malformed. A $\mathfrak{Tetris}$ must satisfy all the following properties, in Fig. 4 we list the oracles used by our definitions.

⁶ The public parameters $\mathbf{pp}$ produced by Setup are implicitly available to all the other algorithms.


```

Proc( $\tau, m, \mathcal{L}_{\text{keys}}, \text{lad}$ )
1 :  $\Sigma \leftarrow \emptyset, t = 0, \text{Th} \leftarrow \emptyset$ 
2 : parse  $\text{lad} = ((\text{action}^1, \text{input}^1), \dots, (\text{action}^l, \text{input}^l))$ 
3 : if  $\text{action}^1 \neq \text{Sign}$  : return  $\perp$ 
4 : else :
5 :   parse  $\text{input}^1 = (\mathcal{R}^1, i^1)$ 
6 :   for  $j \in \mathcal{R}^1$ ;
7 :     if  $(j, \text{pk}_j, \cdot) \notin \mathcal{L}_{\text{keys}}$  : return  $\perp$ 
8 :   if  $\text{sk}_{i^1} = \perp \vee i^1 \notin \mathcal{R}^1$  : return  $\perp$ 
9 :    $\mathcal{R} \leftarrow \mathcal{R}^1, \mathcal{S} \leftarrow \{i^1\}$ 
10 :   $\sigma \leftarrow \text{Sign}(\tau, m, \{\text{pk}_j\}_{j \in \mathcal{R}}, \text{sk}_{i^1})$ 
11 :   $\Sigma \leftarrow \Sigma \cup \{\sigma\}, t \leftarrow 1, \text{Th} \leftarrow \text{Th} \cup \{t\}$ 
12 : for  $l' \in [2, \dots, l]$  :
13 :   if  $\text{action}^{l'} = \text{Sign}$  : return  $\perp$ 
14 :   else :
15 :     if  $\text{action}^{l'} = \text{Join}$  :
16 :       parse  $\text{input}^{l'} = (i^{l'})$ 
17 :       if  $i^{l'} \notin \mathcal{R} \vee i^{l'} \in \mathcal{S}$  : return  $\perp$ 
18 :        $\sigma \leftarrow \text{Join}(\tau, m, \{\text{pk}_j\}_{j \in \mathcal{R}}, \text{sk}_{i^{l'}}, \sigma)$ 
19 :        $\Sigma \leftarrow \Sigma \cup \{\sigma\}, \mathcal{S} \leftarrow \mathcal{S} \cup \{i^{l'}\}, t \leftarrow t + 1, \text{Th} \leftarrow \text{Th} \cup \{t\}$ 
20 :     if  $\text{action}^{l'} = \text{Extend}$  :
21 :       parse  $\text{input}^{l'} = (\mathcal{R}^{l'})$ 
22 :       for  $j \in \mathcal{R}^{l'}$  :
23 :         if  $(j, \text{pk}_j, \cdot) \notin \mathcal{L}_{\text{keys}}$  : return  $\perp$ 
24 :        $\sigma \leftarrow \text{Extend}(\tau, m, \sigma, \{\text{pk}_j\}_{j \in \mathcal{R}}, \{\text{pk}_j\}_{j \in \mathcal{R}^{l'}})$ 
25 :        $\mathcal{R} \leftarrow \mathcal{R} \cup \mathcal{R}^{l'}, \Sigma \leftarrow \Sigma \cup \{\sigma\}, \text{Th} \leftarrow \text{Th} \cup \{t\}$ 
26 :     else : return  $\perp$ 
27 : return  $(\Sigma, \text{Th}, \mathcal{R})$ 

```

Fig. 3. The algorithm Proc for $\mathfrak{Tetris}$.

OSign ($\tau, m, \mathcal{R}, i$)	OKey (i, pk)
1 : if $i \in \mathcal{L}_{\text{corr}} \vee i \notin \mathcal{R}$: return $\perp$ 2 : for $j \in \mathcal{R}$: 3 : if $(j, \text{pk}_j, \cdot) \notin \mathcal{L}_{\text{keys}}$: return $\perp$ 4 : $\sigma \leftarrow \text{Sign}(\tau, m, \{\text{pk}_j\}_{j \in \mathcal{R}}, \text{sk}_i)$ 5 : $\mathcal{L}_{\text{sign}} \leftarrow \mathcal{L}_{\text{sign}} \cup \{(\tau, m, \mathcal{R}, i, \sigma)\}$ 6 : return σ	1 : if $\text{pk} = \perp$: 2 : $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}()$ 3 : $\mathcal{L}_{\text{keys}} \leftarrow \mathcal{L}_{\text{keys}} \cup \{(i, \text{pk}_i, \text{sk}_i)\}$ 4 : else : 5 : $\mathcal{L}_{\text{corr}} \leftarrow \mathcal{L}_{\text{corr}} \cup \{i\}, \text{pk}_i \leftarrow \text{pk}$ 6 : $\mathcal{L}_{\text{keys}} \leftarrow \mathcal{L}_{\text{keys}} \cup \{(i, \text{pk}_i, \perp)\}$ 7 : return pk_i
OJoin ($\tau, m, \mathcal{R}, i, \sigma$)	OCorr (i)
1 : if $i \in \mathcal{L}_{\text{corr}}$: return $\perp$ 2 : for $j \in \mathcal{R}$: 3 : if $(j, \text{pk}_j, \cdot) \notin \mathcal{L}_{\text{keys}}$: return $\perp$ 4 : $\sigma' \leftarrow \text{Join}(\tau, m, \{\text{pk}_j\}_{j \in \mathcal{R}}, \text{sk}_i, \sigma)$ 5 : $\mathcal{L}_{\text{join}} \leftarrow \mathcal{L}_{\text{join}} \cup \{(\tau, m, \mathcal{R}, i, \sigma)\}$ 6 : return σ'	1 : if $(i, \text{pk}_i, \text{sk}_i) \in \mathcal{L}_{\text{keys}} \wedge \text{sk}_i \neq \perp$: 2 : $\mathcal{L}_{\text{corr}} \cup \{i\}$ 3 : return $(\text{pk}_i, \text{sk}_i)$ 4 : return $\perp$

Fig. 4. Oracles for the security definitions of $\mathfrak{T}\text{etris}$.

Definition 1 (Correctness). For all $\lambda \in \mathbb{N}$, any message m , any topic τ , any ladder lad of polynomial size ℓ identifying a ring $\mathcal{R}$:

$$\Pr \left[\left(\bigwedge_{j=1}^{\ell} \text{Verify}(t_j, \tau, m, \{\text{pk}_i\}_{i \in \mathcal{R}}, \sigma_j) = 1 \right) \wedge \left(\begin{array}{l} (\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda); \\ \mathcal{L}_{\text{keys}} \leftarrow \{\text{KeyGen}()\}_{i \in \mathcal{R}}; \\ (\Sigma, \text{Th}, \mathcal{R}) \leftarrow \text{Proc}(\tau, m, \mathcal{L}_{\text{keys}}, \text{lad}); \\ \{\sigma_1, \dots, \sigma_\ell\} = \Sigma; \\ \{t_1, \dots, t_\ell\} = \text{Th}; \end{array} \right) \right] = 1.$$

Definition 2 (Verifiability of Keys). For all $\lambda \in \mathbb{N}$, $(\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda)$, it holds that $(\text{pk}, \text{sk}) \in \text{KeyGen}()$ iff $(\text{pk}, \text{sk}) \in R_{\text{key}}^{\mathfrak{T}\text{etris}}$. We also require that for all pk there exists a unique sk s.t. $(\text{pk}, \text{sk}) \in R_{\text{key}}^{\mathfrak{T}\text{etris}}$.

We define tracing correctness to model that, on honestly generated signatures, Trace should return the expected output (e.g., the set of common public keys if $m_1 \neq m_2$). Precisely, below we give a strong version of this notion that considers adversarially generated keys. This strong notion will be useful in our constructions and clearly implies the weaker one using honestly generated keys.

Definition 3 (Tracing Correctness). For all PPT $\mathcal{A}$ and all $\lambda \in \mathbb{N}$ the success probability in the experiment $\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{etris}}^{\text{Tcorr}}(\lambda)$ (Fig. 5) is $\Pr[\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{etris}}^{\text{Tcorr}}(\lambda) = \text{win}] \leq \text{negl}(\lambda)$.

On defining traceability. Previous definitions for $t = 1$ [30] require the adversary to output a topic τ , a ring $\mathcal{R}$ of size n , and $n + 1$ accepting message/signature pairs w.r.t. τ and $\mathcal{R}$. $\mathcal{A}$ generates all the keys by itself and wins the game if the signatures it produces look signed by different signers according to Trace . If the scheme is traceable, the adversary should not be able to win as there must exist at least a pair of signatures sharing the same signer. However, modeling traceability for general $t \geq 1$ is far more complicated. One may adapt the definition of [30] requiring $\mathcal{A}$ to provide enough signatures so that at least two of them must have at least c overlapping signers. However, unlike the $t = 1$ case where the size of the overlap is fixed, $\mathcal{A}$ may produce signatures with $c' > c$ overlapping signers. Therefore, such a definition would allow schemes tracing c keys while we want to identify *all* the $c' > c$ malicious signers. We solve this issue proposing a definition featuring an extractor Ext which is able to perfectly identify the guilty signers and whose output should match the one of Trace . In the following definition, we require the existence of such extractor.

$\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{etris}}^{\text{TCorr}}(\lambda)$
<pre> 1 : (pp, td) ← Setup(λ) 2 : (τ, m₀, m₁, lad₀, lad₁, L_{keys}) ← A(pp) 3 : ∀(i, pk_i, sk_i) ∈ L_{keys} ∧ sk_i ≠ ⊥ : 4 : if (sk_i, pk_i) ∉ R_{key}^{etris} : return lose 5 : ∀(i, pk_i, sk_i), (j, pk_j, sk_j) ∈ L_{keys} ∧ sk_i, sk_j ≠ ⊥ ∧ i ≠ j : 6 : if pk_i = pk_j : return lose 7 : ∀b ∈ {0, 1} : val_b ← Proc(τ, m_b, L_{keys}, lad_b) 8 : if ∃b ∈ {0, 1} : val_b = ⊥ return lose 9 : ∀b ∈ {0, 1} : parse val_b = (Σ_b, Th_b, R_b) 10 : ∀b ∈ {0, 1} : parse Σ_b = {σ₁^b, ..., σ_{ℓ_b}^b = (μ_b, tinfo_b)} 11 : T ← Trace(τ, m₀, m₁, tinfo₀, tinfo₁, {pk_j}_{j∈R₀}, {pk_k}_{k∈R₁}) 12 : if lad₀.S ∩ lad₁.S = ∅ ∧ T ≠ indep : return win 13 : if m₀ ≠ m₁ ∧ T ≠ lad₀.S ∩ lad₁.S : return win 14 : if m₀ = m₁ ∧ T ≠ (linked, (lad₀.S ∩ lad₁.S)) : return win 15 : return lose </pre>

Fig. 5. Tracing correctness game of $\mathfrak{T}\text{etris}$. Tracing correctness game of $\mathfrak{T}\text{etris}$. In line 6, if public keys are tuples, we say that two public keys are equal if they are equal in at least one position. This is justified as sampling the same value in the public key tuple basically requires sampling the same value in the secret key tuple (e.g., DL of a group element).

In the following definition, we require the existence of such extractor. On input the trapdoor and a valid signature, Ext must extract the signers of the signature. The existence of Ext is crucial to design a traceability experiment which is able to determine the right winning condition for its adversary.

Definition 4 (Signers Extraction). *There exists a PPT algorithm Ext that for all $\lambda \in \mathbb{N}$, $(\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda)$, $\Lambda = (t, \tau, m, \{\text{pk}_i\}_{i \in \mathcal{R}}, \sigma = (\mu, \text{tinfo}))$ s.t. $\text{Verify}(\text{pp}, \Lambda) = 1$ works as follows:*

$$\text{Ext}(\text{td}, \Lambda) = \begin{cases} \perp \\ (\text{PK}, \text{Ind}). \end{cases}$$

Additionally, we require the following two properties to hold:

1. Whenever Ext does not return $\perp$, it holds that:

$$\Pr[(\Lambda, \text{PK}) \in \mathcal{L}_{\text{Ext}} \mid (\text{PK}, \text{Ind}) \leftarrow \text{Ext}(\text{td}, \Lambda)] = 1.$$

Where $\text{PK}[i] = \text{pk}_{\text{Ind}[i]}$ for $i \in [t]$, and $\text{pk}_{\text{Ind}[i]} \in \{\text{pk}_i\}_{i \in \mathcal{R}}$. Moreover, the language $\mathcal{L}_{\text{Ext}} = \{x = (\Lambda, \text{PK}) \mid \exists w = (\text{sk}_{\text{Ind}[1]}, \dots, \text{sk}_{\text{Ind}[t]}, \text{lad}, \text{rand}) \text{ s.t. } (\forall i \in [t]) (\text{sk}_{\text{Ind}[i]}, \text{pk}_{\text{Ind}[i]}) \in R_{\text{key}}^{\mathfrak{T}\text{etris}}) \wedge \text{P}(x, w) = 1\}$, where $\text{P}(x, w)$ computes:

(a) $\text{L}_{\text{keys}} = [(j, \text{pk}_j, \text{sk}_j)_{j \in \mathcal{R}}]$ as

$$(j, \text{pk}_j, \text{sk}_j) = \begin{cases} (j, \text{pk}_j, \text{sk}_{\text{Ind}[j]}) & \text{for } j \in \text{Ind} \\ (j, \text{pk}_j, \perp) & \text{for } j \in \mathcal{R} \setminus \text{Ind}, \end{cases}$$

(b) $(\Sigma, \text{Th}, \mathcal{R}) = \text{Proc}(\tau, m, \text{L}_{\text{keys}}, \text{lad}; \text{rand})$.

(c) Return 1 iff $\Sigma = [\cdot, \dots, (\mu', \text{tinfo}')] and $\text{tinfo}' = \text{tinfo}$, where $\sigma = (\mu, \text{tinfo})$.$

2. For all $\lambda \in \mathbb{N}$, $(\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda)$, for all PPT $\mathcal{A}$ it holds that:

$$\Pr \left[\begin{array}{c} \text{Verify}(\text{pp}, \Lambda) = 1 \wedge \\ \text{Ext}(\text{td}, \Lambda) = \perp \end{array} \middle| (\Lambda) \leftarrow \mathcal{A}(\text{pp}) \right] \leq \text{negl}(\lambda).$$

Intuitively, the property (1) above ensures that the extractor finds valid public keys, and that from their unique secret keys it is possible to generate signatures with the same tracing information tinfo' as the one in input, tinfo . Property (2) instead models that a PPT adversary cannot find a tuple that correctly verifies and makes the extractor fail.

We define traceability with a simple game that requires the adversary to output a pair of signatures and keys of its choice. The adversary wins the game if the output of Trace does not agree with the information derived from running Ext on both signatures.⁷ Notice that $\mathcal{A}$ does not need any oracle in this game as it can adversarially sample keys with the knowledge of their secret key.

Definition 5 (Traceability). *There exists a PPT algorithm Ext s.t. for all PPT $\mathcal{A}$ and all $\lambda \in \mathbb{N}$, the success probability in $\text{Exp}_{\text{Ext}, \mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{Trace}}(\lambda)$ (Fig. 6) is $\Pr[\text{Exp}_{\text{Ext}, \mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{Trace}}(\lambda) = \text{win}] \leq \text{negl}(\lambda)$.*

Finally, we define anonymity and exculpability. The former models that an adversary should not learn which keys were used to generate a signature. Notably, we do this by letting the adversary choose two ladders and see the full evolution of the signature. The latter models that an adversary cannot falsely accuse a honest user of producing two signatures on the same topic, if the user did not do it. Slightly below we give more details on the checks performed in the experiments.

Definition 6 (Anonymity). *For all PPT $\mathcal{A}$ and all $\lambda \in \mathbb{N}$ the success probability in $\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{Anon}}(\lambda)$ (Fig. 6) is $\Pr[\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{Anon}}(\lambda) = \text{win}] \leq \frac{1}{2} + \text{negl}(\lambda)$. The ladders output by $\mathcal{A}$ are well-formed if all the actions in the ladders are pairwise of the same type and they have the same ring as input.*

Definition 7 (Exculpability). *For all PPT $\mathcal{A}$ and all $\lambda \in \mathbb{N}$ the success probability in $\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{Exculp}}(\lambda)$ (Fig. 7) is $\Pr[\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{Exculp}}(\lambda) = \text{win}] \leq \text{negl}(\lambda)$.*

On the admissible adversaries of anonymity and exculpability. In our definitions there are some obvious checks that are performed by the experiment to exclude adversaries winning the game by just making use of the oracles. However, there is a less obvious check (line 10 of $\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{Exculp}}(\lambda)$ (Fig. 7) and line 11 of $\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{Anon}}(\lambda)$ (Fig. 6). Namely, any key for which $\mathcal{A}$ asked two queries over the same topic τ and two different messages $m_0 \neq m_1$ is considered as if it was corrupted. This is because, since the aim of traceable signatures is to prevent such behavior, we allow for constructions with the additional feature that every signer who signs different messages loses any security guarantee. Finally, the check of line 10 of $\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{Anon}}(\lambda)$ (Fig. 6) excludes adversary that can easily distinguish both ladders using Trace .

Additional definitions and lemmas. We now give the definition of unforgeability and show that a $\mathfrak{T}\text{ctris}$ which is both traceable and exculpable is also unforgeable. Additionally, we show that tracing correctness and signers extraction imply traceability. As [5,8], we do not model malicious signers trying to prevent others from joining a signature.

Definition 8 (Unforgeability). *For all PPT adversaries $\mathcal{A}$ and for all $\lambda \in \mathbb{N}$ the success probability in the experiment $\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{cmEUF}}(\lambda)$ of Fig. 8 is*

$$\Pr[\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{ctris}}^{\text{cmEUF}}(\lambda) = \text{win}] \leq \text{negl}(\lambda).$$

⁷ Notice that the Ext required in this definition operates as the extractor of signers extraction (Def. 4).

$\text{Exp}_{\mathcal{A}, \mathfrak{Tetris}}^{\text{Anon}}(\lambda)$	$\text{Chal}_b(\tau^*, m^*, \text{lad}_0^*, \text{lad}_1^*)$
<pre> 1 : $b \leftarrow \{0, 1\}, \mathsf{L}_{\text{keys}}, \mathsf{L}_{\text{corr}}, \mathsf{L}_{\text{sign}}, \mathsf{L}_{\text{join}} \leftarrow \emptyset$ 2 : $(\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda)$ 3 : $\mathsf{O} \leftarrow \{\text{OSign}, \text{OKey}, \text{OCorr}, \text{OJoin}\}$ 4 : $(\tau^*, m^*, \text{lad}_0^*, \text{lad}_1^*) \leftarrow \mathcal{A}^{\mathsf{O}}(\text{pp})$ 5 : $\Sigma \leftarrow \text{Chal}_b(\tau^*, m^*, \text{lad}_0^*, \text{lad}_1^*)$ 6 : $b^* \leftarrow \mathcal{A}^{\mathsf{O}}(\Sigma)$ 7 : if $\exists i \in \text{lad}_0^*. \mathcal{S} \cup \text{lad}_1^*. \mathcal{S}$ s.t. $i \in \mathsf{L}_{\text{corr}}$: 8 : return <i>lose</i> 9 : for $i \in \text{lad}_0^*. \mathcal{S} \cup \text{lad}_1^*. \mathcal{S}$: 10 : if $\exists (\tau^*, \cdot, \cdot, i, \cdot) \in \mathsf{L}_{\text{sign}} \cup \mathsf{L}_{\text{join}}$: return <i>lose</i> 11 : if $\exists (\tau, m_1, \cdot, i, \cdot), (\tau, m_2, \cdot, i, \cdot) \in$ $\mathsf{L}_{\text{sign}} \cup \mathsf{L}_{\text{join}} \wedge m_1 \neq m_2$: return <i>lose</i> 12 : if $b^* \neq b$: return <i>lose</i> else : return <i>win</i></pre>	<pre> 1 : if $(\text{lad}_0^*, \text{lad}_1^*)$ is not well-formed : 2 : return $\perp$ 3 : if $\exists i \in \text{lad}_0^*. \mathcal{S}$ s.t. $i \in \mathsf{L}_{\text{corr}}$: 4 : return $\perp$ 5 : if $\exists i \in \text{lad}_1^*. \mathcal{S}$ s.t. $i \in \mathsf{L}_{\text{corr}}$: 6 : return $\perp$ 7 : $\text{val}_0 \leftarrow \text{Proc}(\tau^*, m^*, \mathsf{L}_{\text{keys}}, \text{lad}_0^*)$ 8 : $\text{val}_1 \leftarrow \text{Proc}(\tau^*, m^*, \mathsf{L}_{\text{keys}}, \text{lad}_1^*)$ 9 : if $\text{val}_0 = \perp \vee \text{val}_1 = \perp$: 10 : return $\perp$ 11 : parse $\text{val}_0 = (\Sigma_0, \text{Th}_0, \mathcal{R}_0)$ 12 : parse $\text{val}_1 = (\Sigma_1, \text{Th}_1, \mathcal{R}_1)$ 13 : return Σ_b</pre>

$\text{Exp}_{\text{Ext}, \mathcal{A}, \mathfrak{Tetris}}^{\text{Trace}}(\lambda)$
<pre> 1 : $(\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda)$ 2 : $(\tau, t_0, t_1, \{\text{pk}_i\}_{i \in \mathcal{R}_0}, \{\text{pk}_i\}_{i \in \mathcal{R}_1}, m_0, m_1, \sigma_0, \sigma_1) \leftarrow \mathcal{A}(\text{pp}, \text{td})$ 3 : if $\exists b \in \{0, 1\}$ s.t. $\text{Verify}(t_b, \tau, m_b, \{\text{pk}_i\}_{i \in \mathcal{R}_b}, \sigma_b) = 0$: return <i>lose</i> 4 : $\forall b \in \{0, 1\}$: $\text{out}_b \leftarrow \text{Ext}(\text{td}, \tau, m_b, t_b, \{\text{pk}_i\}_{i \in \mathcal{R}_b}, \sigma_b)$ 5 : if $\exists b \in \{0, 1\}$ s.t. $\text{out}_b = \perp$: return <i>win</i> 6 : $\forall b \in \{0, 1\}$: parse $\text{out}_b = (\text{PK}_b, \text{Ind}_b)$ 7 : $\forall b \in \{0, 1\}$: parse $\sigma_b = (\mu_b, \text{tinfo}_b)$ 8 : $T \leftarrow \text{Trace}(\tau, m_0, m_1, \text{tinfo}_0, \text{tinfo}_1, \{\text{pk}_j\}_{j \in \mathcal{R}_0}, \{\text{pk}_k\}_{k \in \mathcal{R}_1})$ 9 : if $\text{PK} = \text{PK}_0 \cap \text{PK}_1 = \emptyset \wedge T \neq \text{indep}$: return <i>win</i> else : return <i>lose</i> 10 : if $m_0 \neq m_1$: 11 : if $\text{PK} \neq T$: return <i>win</i> else : return <i>lose</i> 12 : else : 13 : parse $T = (\text{linked}, n)$ 14 : if $n \neq \text{PK}$: return <i>win</i> else return <i>lose</i></pre>

Fig. 6. Anonymity and traceability games of $\mathfrak{Tetris}$.

$\text{Exp}_{\mathcal{A}, \mathfrak{T}\text{etris}}^{\text{Exculp}}(\lambda)$
1 : $\mathsf{L}_{\text{keys}}, \mathsf{L}_{\text{corr}}, \mathsf{L}_{\text{sign}}, \mathsf{L}_{\text{join}} \leftarrow \emptyset, \mathsf{O} \leftarrow \{\text{OSign}, \text{OKey}, \text{OCorr}, \text{OJoin}\}$
2 : $(\mathsf{pp}, \mathsf{td}) \leftarrow \text{Setup}(1^\lambda)$
3 : $(\tau^*, t_0^*, t_1^*, \mathcal{R}_0^*, \mathcal{R}_1^*, m_0^*, m_1^*, \sigma_0^*, \sigma_1^*) \leftarrow \mathcal{A}^{\mathsf{O}}(\mathsf{pp})$
4 : if $\exists b \in \{0, 1\}$ Verify $(t_b^*, \tau^*, m_b^*, \{\mathsf{pk}_i\}_{i \in \mathcal{R}_b^*}, \sigma_b^*) = 0$: return lose
5 : $\mathsf{PK} \leftarrow \text{Trace}(\tau^*, m_0^*, m_1^*, \sigma_0^*, \sigma_1^*, \{\mathsf{pk}_i\}_{i \in \mathcal{R}_0^*}, \{\mathsf{pk}_i\}_{i \in \mathcal{R}_1^*})$
6 : if $\mathsf{PK} = \text{indep} \vee \mathsf{PK} = (\text{linked}, \cdot)$:
7 : return lose
8 : $\forall \mathsf{pk}_i \in \mathsf{PK}$:
9 : if $\mathsf{pk}_i \in \mathsf{L}_{\text{corr}}$: $\mathsf{PK} = \mathsf{PK} \setminus \mathsf{pk}_i$
10 : elseif $\exists(\tau, m_0, \mathcal{R}', \sigma', j), (\tau, m_1, \mathcal{R}'', \sigma'', j) \in \mathsf{L}_{\text{sign}} \cup \mathsf{L}_{\text{join}}$ s.t. $m_0 \neq m_1 \wedge j \in \mathcal{R}_0^* \wedge j \in \mathcal{R}_1^* \wedge \mathsf{pk}_i = \mathsf{pk}_j$:
11 : $\mathsf{PK} = \mathsf{PK} \setminus \mathsf{pk}_i$
12 : if $\mathsf{PK} = \emptyset$: return lose else : return win

Fig. 7. Exculpability game of $\mathfrak{T}\text{etris}$.

Lemma 1. *If a $\mathfrak{T}\text{etris}$ is both exculpable (Def. 7) and traceable (Def. 5) then it is also unforgeable.*

Proof. Let us assume that there exists an adversary $\mathcal{A}$ that breaks the unforgeability property with non-negligible probability. We build an adversary $\mathcal{B}$ that wins the exculpability game with the same probability. Let $\mathcal{C}$ be the challenger of the exculpability game, $\mathcal{B}$ replies to all the oracle calls made by $\mathcal{A}$ by forwarding such calls to $\mathcal{C}$. $\mathcal{B}$ stores all the queries made by $\mathcal{A}$ along with the corresponding replies from $\mathcal{C}$. Let $(t^*, \tau^*, m^*, \mathcal{R}^*, \sigma^*)$ be the forgery output by $\mathcal{A}$. Let us focus on the queries that $\mathcal{A}$ performs on the public keys contained in $\mathcal{R}^*$.⁸ The number of these queries is given by $q = c + q_1 + q_2$ where c is the number of corrupted keys in $\mathcal{R}^*$, q_1 is the number of Join/Sign queries over (τ^*, m^*) , and q_2 is the number of pairs of Join/Sign queries over the same topic and different messages made w.r.t. the same public key. Let us call any of the above queries a compromising query, and let us refer to the public key of such a query as a compromised key. Notice that $q < t^*$, therefore, the maximum number of public keys in $\mathcal{R}^*$ on which $\mathcal{A}$ performed a compromising query is $t^* - 1$, thus there must exist a set $P \subseteq \mathcal{R}^*$ containing at least one *not* compromised key. Since the signature scheme achieves signer extraction (Def. 4), the probability that there are no public keys in P which is also in the set of signers⁹ of σ^* is negligible (the reduction to signers extraction is very straightforward). For each $\mathsf{pk}_j \in P$, $\mathcal{B}$ looks at the queries made by $\mathcal{A}$ for a Join/Sign query involving (τ^*, m, j) , with $m \neq m^*$. If such a query exists, let $\sigma_j, \mathcal{R}_j, t_j$ be the signature, the ring, and the threshold resulting from the reply of $\mathcal{C}$ to such query. Otherwise, $\mathcal{B}$ queries $\mathcal{C}$ to generate a signature on $(\tau^*, m, \mathcal{R}^*, j)$, for arbitrary $m \neq m^*$ receiving back a signature σ_j over ring $\mathcal{R}_j = \mathcal{R}^*$ with threshold $t_j = 1$. For each $\mathsf{pk}_j \in P$, $\mathcal{B}$ runs $\mathsf{PK}_j \leftarrow \text{Trace}(\tau^*, m^*, m, \text{tinfo}^*, \text{tinfo}_j, \mathcal{R}^*, \mathcal{R}_j)$, where tinfo^* and tinfo_j are taken from σ^* and σ_j respectively. Thanks to traceability (the reduction is very straightforward), there must exist at least one $\mathsf{pk}_j \in P$ such that $\mathsf{pk}_j \in \mathsf{PK}_j$, therefore $\mathcal{B}$ sends $(\tau^*, t^*, t_j, \mathcal{R}^*, \mathcal{R}^*, m^*, m, \sigma^*, \sigma_j)$ to $\mathcal{C}$. Assuming that the queries made by $\mathcal{B}$ do not cause the removal of any $\mathsf{pk}_j \in P$ from PK in lines 9 and 10 of the exculpability experiment (Fig. 7), then $\mathcal{B}$ wins the exculpability game with the same probability of $\mathcal{A}$ winning the unforgeability game. It remains to show that the queries carried out by $\mathcal{B}$ do not cause the removal of any $\mathsf{pk}_j \in P$ from PK in lines 9 and 10 of the exculpability experiment (Fig. 7). To show this, we observe that since $\mathsf{pk}_j \in P$ the following facts about the queries performed by $\mathcal{A}$ over pk_j hold:

⁸ For simplicity, we say that a public key belongs to the ring instead of saying that it belongs to the set $\{\mathsf{pk}_j\}_{j \in \mathcal{R}^*}$.

⁹ More precisely, there exists a ladder that includes this public key in the set of signers and leads to the tinfo^* contained in σ^* .

Exp_{$\mathcal{A}, \mathfrak{Tetris}$}^{cmEUF}($\lambda$)	OKey(i, pk)
1 : $L_{\text{keys}}, L_{\text{corr}}, L_{\text{sign}}, L_{\text{join}} \leftarrow \emptyset$ 2 : $(pp, td) \leftarrow \text{Setup}(1^\lambda)$ 3 : $O \leftarrow \{\text{OSign}, \text{OKey}, \text{OCorr}, \text{OJoin}\}$ 4 : $(t^*, \tau^*, m^*, \mathcal{R}^*, \sigma^*) \leftarrow \mathcal{A}^O(pp)$ 5 : $L \leftarrow L_{\text{sign}} \cup L_{\text{join}}$ 6 : $q_1 \leftarrow \{(\tau^*, m^*, \cdot, i, \cdot) \in L : i \in \mathcal{R}^*\} $ 7 : $q_2 \leftarrow \{(\tau, m_1, \cdot, i, \cdot), (\tau, m_2, \cdot, i, \cdot) \in L : m_1 \neq m_2 \wedge i \in \mathcal{R}^*\} $ 8 : if $ \mathcal{R}^* \cap L_{\text{corr}} + q_1 + q_2 \geq t^*$: 9 : return <i>lose</i> 10 : if $\text{Verify}(t^*, \tau^*, m^*, \{pk_j\}_{j \in \mathcal{R}^*}, \sigma^*) = 0$: 11 : return <i>lose</i> 12 : return <i>win</i>	1 : if $pk = \perp$; 2 : $(pk_i, sk_i) \leftarrow \text{KeyGen}()$ 3 : $L_{\text{keys}} \leftarrow L_{\text{keys}} \cup \{(i, pk_i, sk_i)\}$ 4 : else : 5 : $L_{\text{corr}} \leftarrow L_{\text{corr}} \cup \{i\}$ 6 : $pk_i \leftarrow pk$ 7 : $L_{\text{keys}} \leftarrow L_{\text{keys}} \cup \{(i, pk_i, \perp)\}$ 8 : return pk_i
OSign($\tau, m, \mathcal{R}, i$)	OCorr(i)
1 : if $(i \in L_{\text{corr}} \vee i \notin \mathcal{R})$: return $\perp$ 2 : for $j \in \mathcal{R}$: 3 : if $(j, pk_j, \cdot) \notin L_{\text{keys}}$: 4 : return $\perp$ 5 : $\sigma \leftarrow \text{Sign}(\tau, m, \{pk_j\}_{j \in \mathcal{R}}, sk_i)$ 6 : $L_{\text{sign}} \leftarrow L_{\text{sign}} \cup \{(\tau, m, \mathcal{R}, i, \sigma)\}$ 7 : return σ	1 : if $(i, pk_i, sk_i) \in L_{\text{keys}} \wedge sk_i \neq \perp$: 2 : $L_{\text{corr}} \leftarrow L_{\text{corr}} \cup \{i\}$ 3 : return (pk_i, sk_i) 4 : return $\perp$
	OJoin($\tau, m, \mathcal{R}, i, \sigma$)
	1 : if $i \in L_{\text{corr}}$: return $\perp$ 2 : for $j \in \mathcal{R}$: 3 : if $(j, pk_j, \cdot) \notin L_{\text{keys}}$: 4 : return $\perp$ 5 : $\sigma' \leftarrow \text{Join}(\tau, m, \{pk_j\}_{j \in \mathcal{R}}, sk_i, \sigma)$ 6 : $L_{\text{join}} \leftarrow L_{\text{join}} \cup \{(\tau, m, \mathcal{R}, i, \sigma)\}$ 7 : return σ'

Fig. 8. Unforgeability game for $\mathfrak{Tetris}$ (security experiment and oracles).

- pk_j was never corrupted;
- no query over (τ^*, m^*) was ever asked for pk_j ;
- no pair of Join/Sign queries over the same topic and different messages were ever made for pk_j .

$\mathcal{B}$ does not perform any corruption query and it only asks, if not previously asked by $\mathcal{A}$, a query over (τ^*, m) with $m \neq m^*$. As a result, pk_j will not have pairs of Join/Sign queries over the same topic and different messages and it will not be removed from PK in lines 9 and 10 of the exculpability experiment (Fig. 7).

Lemma 2. *Every $\mathfrak{T}\text{etris}$ which has verifiability of keys (Def. 2), tracing correctness (Def. 3) and signers extraction (Def. 4) is traceable (Def. 5).*

Proof. The probability of $\mathcal{A}$ winning $\text{Exp}_{\text{Ext}, \mathcal{A}, \mathfrak{T}\text{etris}}^{\text{Trace}}(\lambda)$ is at most the sum of the probabilities of $\mathcal{A}$ being able to satisfy one of the following conditions:

1. $\mathcal{A}$ makes Ext output $\perp$ (i.e., line 4 of $\text{Exp}_{\text{Ext}, \mathcal{A}, \mathfrak{T}\text{etris}}^{\text{Trace}}(\lambda)$ (Fig. 6));
2. $\mathcal{A}$ outputs two signatures σ_0, σ_1 s.t. PK extracted by Ext is empty but Trace does not output indep (i.e., line 9 of $\text{Exp}_{\text{Ext}, \mathcal{A}, \mathfrak{T}\text{etris}}^{\text{Trace}}(\lambda)$).
3. $\mathcal{A}$ outputs two signatures σ_0, σ_1 on different messages $m_0 \neq m_1$ which trace to a set of keys T different from the set of keys PK extracted by Ext (i.e., lines 10 and 11 of $\text{Exp}_{\text{Ext}, \mathcal{A}, \mathfrak{T}\text{etris}}^{\text{Trace}}(\lambda)$);
4. $\mathcal{A}$ outputs two signatures σ_0, σ_1 on the same message (i.e. $m_0 = m_1$) on which Trace returns (linked, n) , but n is not equal to $|\text{PK}|$, where PK is the set of keys extracted by Ext (i.e., lines 13-14 of $\text{Exp}_{\text{Ext}, \mathcal{A}, \mathfrak{T}\text{etris}}^{\text{Trace}}(\lambda)$).

We argue that $\mathcal{A}$ is able to satisfy each of the above conditions only with negligible probability.

Let us consider case 1. The probability of $\mathcal{A}$ satisfying case 1 is negligible, otherwise we can break the signers extraction of $\mathfrak{T}\text{etris}$ (Def. 4) by simply using one of the two signatures given in output by $\mathcal{A}$.

Let us now analyze the remaining cases. Since $\mathfrak{T}\text{etris}$ satisfies signers extraction, we have that, there must exist two valid ladders, that, generate (via Proc), using the secret keys corresponding to the public keys extracted by Ext, two signatures containing *exactly* tinfo_0 and tinfo_1 . Thus, due to tracing correctness of $\mathfrak{T}\text{etris}$ (Def. 3) and the fact that Trace is deterministic, given σ_0 and σ_1 output by $\mathcal{A}$, the output of Trace is consistent with the one derived from Ext with overwhelming probability. Therefore, the probability of $\mathcal{A}$ satisfying one of case 2, 3, 4 is negligible.

4 Extendable Non-Interactive Proof Systems

We propose a more expressive poly-time relation than [8] called generalized threshold relation R_{GT} containing triples (ck, x, w) , where Com_{ck} is a commitment scheme with parameters ck and F is a (hard) predicate.

$$R_{\text{GT}} = \{(\text{ck}, x = (k, x_1, \dots, x_n, \tilde{c}_1, \dots, \tilde{c}_n, e_1, \dots, e_k), w = (\phi, w_1, \dots, w_k, \tilde{r}_1, \dots, \tilde{r}_n)) \mid \\ \forall i \in [n] \exists \text{bit}_i : \tilde{c}_i \leftarrow \text{Com}_{\text{ck}}(\text{bit}_i; \tilde{r}_i) \wedge \text{bit}_i \in \{0, 1\} \wedge \sum_{i=1}^n \text{bit}_i = k \wedge \\ \forall_{j=1}^k (\text{bit}_{\phi(j)} = 1 \wedge F(\text{ck}, x_{\phi(j)}, e_j, w_j) = 1) \wedge \phi : [k] \rightarrow [n] \wedge \phi \text{ is injective}\}.$$

Two important specific cases are:

- By setting $\forall i \in [k] e_i$ to be empty strings and $F(\text{ck}, x, \cdot, w) = R_{\mathcal{L}'}(x, w)$, we retrieve the threshold relation over $R_{\mathcal{L}'}$.
- By setting $F(\text{ck}, x, e, w)$ to be the predicate that returns 1 iff $x = \text{Com}_{\text{ck}}(e; w)$, we retrieve the extendable shuffle relation (see relation (1)).

We call $\{\tilde{c}_i\}_{i \in [n]}$ commitments to *switch variables* and $\{e_i\}_{i \in [k]}$ *clear-text* elements. Additionally, we call the set $\{x_{\phi(j)}\}_{j \in [k]}$ *active* statements, while $\{x_i\}_{i \in [n]} \setminus \{x_{\phi(j)}\}_{j \in [k]}$ *inactive*. Let $R_{\mathcal{L}}$ be an instantiation of R_{GT} where Com_{ck} and F are concretely specified. An extendable non-interactive proof system EP also comes with *addition* ($\text{AddT} = (\text{AddT}_x, \text{AddT}_w)$) and *extension* ($\text{ExtT} = (\text{ExtT}_x, \text{ExtT}_w)$) transformations.

For example, $\text{AddT}_x(\text{ck}, x, e, \alpha, w_\alpha, \widetilde{r}_\alpha; r)$ is a transformation, with randomness r , that takes a statement $x = (k, x_1, \dots, x_n, \widetilde{c}_1, \dots, \widetilde{c}_n, e_1, \dots, e_{k-1})$, and transforms it to $x' = (k+1, x'_1, \dots, x'_n, \widetilde{c}'_1, \dots, \widetilde{c}'_n, e_1, \dots, e_{k-1}, e)$. Namely, after an addition (or extension) operation the parts of the statement related to the threshold and the clear-text elements are deterministically modified independently from the randomness in input to the transformation, while the commitments and the base statements can be updated depending on the randomness (e.g., re-randomized). Analogously, AddT_w , on input the same transformation randomness given to AddT_x , transforms a witness w s.t. $(\text{ck}, x, w) \in R_{\mathcal{L}}$ into a witness w' s.t. $(\text{ck}, x', w') \in R_{\mathcal{L}}$. An EP for $R_{\mathcal{L}}$ consists of the following PPT algorithms along with transformations $\text{ExtT}_x, \text{ExtT}_w, \text{AddT}_x, \text{AddT}_w$. The group key $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$ is considered as an implicit input to all algorithms:

- $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$: generates crs and trapdoor td .
- $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$: generates the commitment key ck .
- $c \leftarrow \text{Com}_{\text{ck}}(m; r)$: on input a message m and a randomness r , given the commitment key ck , returns a commitment c .
- $(\Pi, (\text{aux}_1, \dots, \text{aux}_n)) \leftarrow \text{Prv}(\text{crs}, \text{ck}, x, w)$: on input statement $x = (k, x_1, \dots, x_n, \widetilde{c}_1, \dots, \widetilde{c}_n, e_1, \dots, e_k)$, witness $w = (\phi, w_1, \dots, w_k, \widetilde{r}_1, \dots, \widetilde{r}_n)$, s.t. $(\text{ck}, x, w) \in R_{\mathcal{L}}$ outputs a proof Π and auxiliary values $(\text{aux}_1, \dots, \text{aux}_n)$. Each aux_i is used later on to perform an add operation using a witness for a not previously used statement x_i .
- $0/1 \leftarrow \text{PVfy}(\text{crs}, \text{ck}, x, \Pi)$: on input statement x , and a proof Π , outputs 1 to accept and 0 to reject.
- $(\Pi', x', \widetilde{\text{aux}}_{n+1}, (r_1, \dots, r_{n+1})) \leftarrow \text{PExt}(\text{crs}, \text{ck}, x, x_{n+1}, \widetilde{r}_{n+1}, \Pi, r)$: on input statements x, x_{n+1} , randomness $\widetilde{r}_{n+1}$ (used to commit to $\text{bit}_{n+1} = 0$), a proof Π for $x \in \mathcal{L}$, and transformation randomness r , outputs an updated statement $x' = (k, x'_1, \dots, x'_{n+1}, \widetilde{c}'_1, \dots, \widetilde{c}'_n, e_1, \dots, e_k)$, an updated proof Π' for $x' \in \mathcal{L}$, and randomnesses $r_1, \dots, r_{n+1}$.
- $(\Pi', x', \text{aux}'_\alpha, (r_1, \dots, r_n)) \leftarrow \text{PAdd}(\text{crs}, \text{ck}, x, w', \alpha, e, \widetilde{r}_\alpha, \text{aux}_\alpha, \Pi, r)$: on input statement x , witness w' , index α , element e , randomness $\widetilde{r}_\alpha$ (used to commit to $\text{bit}_\alpha = 1$), auxiliary value aux_α , proof Π for $x \in \mathcal{L}$, and transformation randomness r , outputs an updated statement $x' = (k+1, x'_1, \dots, x'_{n+1}, \widetilde{c}'_1, \dots, \widetilde{c}'_n, e_1, \dots, e_k, e)$, an updated proof Π' for $x' \in \mathcal{L}$, an updated auxiliary value aux'_α , and randomnesses $(r_1, \dots, r_n)$.
- $\text{aux}'_i \leftarrow \text{AuxUpd}(\text{crs}, \text{ck}, \Pi, \text{aux}_i, r_i)$: on input proof Π previous to addition or extension, the auxiliary value aux_i , and update randomness r_i , outputs updated auxiliary value aux'_i . AuxUpd is used to update the auxiliary values after an extend/add operation. The randomness given in input is the one given in output by either PExt or PAdd . To simplify the notation, we write $\text{AUX}' \leftarrow \text{AuxUpd}(\text{crs}, \text{ck}, x, \Pi, \text{AUX}, r)$ to indicate that a list of auxiliary values is updated by appropriately parsing AUX and r and running the update operation on each element of the list.
- $0/1 \leftarrow \text{AuxVerify}(\text{crs}, \text{ck}, x, w, (\text{aux}_1, \dots, \text{aux}_n), \Pi)$: on input statement x , witness w , auxiliary values $(\text{aux}_1, \dots, \text{aux}_n)$, and proof Π , outputs 1 if the auxiliary values are consistent with the statement, the proof, and the witness. Returns 0 otherwise. If AuxVerify returns 1, we are guaranteed that the subsequent extend/add operations can be correctly performed¹⁰.

An EP has to satisfy: (1) *completeness*, i.e., honestly generated proofs are always accepting and result in correct auxiliary values (w.r.t. AuxVerify); (2) *soundness*, i.e., it is unfeasible to produce accepting proofs for false statements; (3) *admissible transformations*, i.e. transforming $(\text{ck}, x, w) \in R_{\mathcal{L}}$ using AddT or ExtT with the intended inputs always gives $(\text{ck}, x', w') \in R_{\mathcal{L}}$; (4) *transformation completeness*, i.e., addition and extension operations always give an accepting proof and correct auxiliary values when executed with the intended inputs. Furthermore, AddT_x (ExtT_x) and PAdd (PExt) must give the same x' in output when given in input the same r and x . Finally, Com_{ck} must be perfectly binding and computationally hiding.

Definition 9 (Binding and Hiding of Com_{ck}). For all $\lambda \in \mathbb{N}$, $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, Com_{ck} is a perfectly binding and computationally hiding commitment scheme.

¹⁰ We introduce AuxVerify merely as an internal utility to simplify the description of our definitions.

Definition 10 (Completeness). An EP for $R_{\mathcal{L}}$ is complete if $\forall \lambda \in \mathbb{N}$, $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$, $(\text{ck}, x, w) \in R_{\mathcal{L}}$, and $(\Pi, \text{AUX}) \leftarrow \text{Prv}(\text{crs}, \text{ck}, x, w)$ it holds that

$$\Pr[\text{PVfy}(\text{crs}, \text{ck}, x, \Pi) = 1 \wedge \text{AuxVerify}(\text{crs}, \text{ck}, x, w, \text{AUX}, \Pi) = 1] = 1$$

Definition 11 (Admissible Transformations). The transformations AddT_x , AddT_w , ExtT_x and ExtT_w have the following properties:

- AddT_x is a transformation that on input $(\text{ck}, x, e, \alpha, w^*, \widetilde{r}_\alpha^*; r)$, where $x = (k, x_1, \dots, x_n, \widetilde{c}_1, \dots, \widetilde{c}_n, e_1, \dots, e_{k-1})$, e is the added element, α is an index, w^* is the witness for position α , $\widetilde{r}_\alpha^*$ is a randomness, and r is the randomness of the transformation, output $x' = (k+1, x'_1, \dots, x'_n, \widetilde{c}'_1, \dots, \widetilde{c}'_n, e_1, \dots, e_{k-1}, e)$.
- AddT_w is a transformation that on input $(\text{ck}, w, w^*, \alpha, \widetilde{r}_\alpha^*; r)$, where $w = (\phi, w_1, \dots, w_{k-1}, \widetilde{r}_1, \dots, \widetilde{r}_n)$, output $w' = (\phi', w'_1, \dots, w'_{k-1}, w^*, \widetilde{r}'_1, \dots, \widetilde{r}'_n)$, where $\phi' = \phi \cup \{k+1 \rightarrow \alpha\}$ ¹¹.
- Given $\text{ck}, x, e, w, w^*, \alpha, \widetilde{r}_\alpha^*, r$, if $(\text{ck}, x, w) \in R_{\mathcal{L}}$, $x' \leftarrow \text{AddT}_x(\text{ck}, x, e, \alpha, w^*, \widetilde{r}_\alpha^*; r)$, and $w' \leftarrow \text{AddT}_w(\text{ck}, w, w^*, \alpha, \widetilde{r}_\alpha^*; r)$, $F(\text{ck}, x'_\alpha, e, w'^*) = 1$, then it holds that $(\text{ck}, x', w') \in R_{\mathcal{L}}$.
- ExtT_x is a transformation that on input $(\text{ck}, x, x_{n+1}, \widetilde{r}_{n+1}; r)$ output $x' = (\text{ck}, k, x'_1, \dots, x'_n, x'_{n+1}, \widetilde{c}'_1, \dots, \widetilde{c}'_n, \widetilde{c}'_{n+1}, e_1, \dots, e_k)$ where $x = (\text{ck}, k, x_1, \dots, x_n, \widetilde{c}_1, \dots, \widetilde{c}_n, e_1, \dots, e_k)$.
- ExtT_w is a transformation that on input $(\text{ck}, w, \widetilde{r}_{n+1}; r)$ output $w' = (\phi, w'_1, \dots, w'_k, \widetilde{r}'_1, \dots, \widetilde{r}'_{n+1})$ where $w = (\phi, w_1, \dots, w_k, \widetilde{r}_1, \dots, \widetilde{r}_{n+1})$.
- Given $\text{ck}, x, x_{n+1}, \widetilde{c}_{n+1}, \widetilde{r}_{n+1}, r$, if $(\text{ck}, x, w) \in R_{\mathcal{L}}$, $x' \leftarrow \text{ExtT}_x(\text{ck}, x, x_{n+1}, \widetilde{r}_{n+1}; r)$ and $w' \leftarrow \text{ExtT}_w(\text{ck}, w, \widetilde{r}_{n+1}; r)$, then it holds that $(\text{ck}, x', w') \in R_{\mathcal{L}}$.

Definition 12 (Transformation Completeness). An EP for $R_{\mathcal{L}}$ is transformation complete if $\forall \lambda \in \mathbb{N}$, $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$, $(\text{ck}, x, w) \in R_{\mathcal{L}}$, where $x = (k, x_1, \dots, x_n, \widetilde{c}_1, \dots, \widetilde{c}_n, e_1, \dots, e_k)$ and $w = (\phi, w_1, \dots, w_k, \widetilde{r}_1, \dots, \widetilde{r}_n)$, for all randomnesses r , and (Π, AUX) such that $\text{PVfy}(\text{crs}, \text{ck}, x, \Pi) = 1$ and $\text{AuxVerify}(\text{crs}, \text{ck}, x, w, \text{AUX}, \Pi) = 1$ the following holds with probability 1:

- For all $e, w^*, \alpha, \widetilde{r}_\alpha^*$ s.t. $\alpha \in [n]$, $\phi(i) \neq \alpha$ for all $i \in [k]$, if $(\Pi', x', \text{aux}'_\alpha, r') \leftarrow \text{PAdd}(\text{crs}, \text{ck}, x, w^*, \alpha, e, \widetilde{r}_\alpha^*, \text{AUX}[\alpha], \Pi, r)$ and $F(\text{ck}, x'_\alpha, e, w') = 1$ then $\text{PVfy}(\text{crs}, \text{ck}, x', \Pi') = 1$ and $\text{AuxVerify}(\text{crs}, \text{ck}, x', w', \text{AUX}', \Pi') = 1$. The inputs to AuxVerify and F are obtained as follows:
 - $x' \leftarrow \text{AddT}_x(\text{ck}, x, e, \alpha, w^*, \widetilde{r}_\alpha^*; r)$
 - $w' \leftarrow \text{AddT}_w(\text{ck}, w, w^*, \alpha, \widetilde{r}_\alpha^*; r)$
 - AUX' is obtained as follows: AUX'' is obtained by replacing aux_α with aux'_α in AUX , and $\text{AUX}' \leftarrow \text{AuxUpd}(\text{crs}, \text{ck}, x', \Pi', \text{AUX}'', r')$
- For all $x_{n+1}, \widetilde{r}_{n+1}, r$, if $(\Pi', x', \text{aux}_{n+1}, r') \leftarrow \text{PExt}(\text{crs}, \text{ck}, x, x_{n+1}, \widetilde{r}_{n+1}, \Pi, r)$, then $\text{PVfy}(\text{crs}, \text{ck}, x', \Pi') = 1$, and $\text{AuxVerify}(\text{crs}, \text{ck}, x', w', \text{AUX}', \Pi') = 1$. The inputs to AuxVerify are obtained as follows:
 - $x' \leftarrow \text{ExtT}_x(\text{ck}, x, x_{n+1}, \widetilde{r}_{n+1}; r)$
 - $w' \leftarrow \text{ExtT}_w(\text{ck}, w, \widetilde{r}_{n+1}; r)$
 - AUX' is obtained as follows: AUX'' is obtained by adding aux_{n+1} to AUX , and $\text{AUX}' \leftarrow \text{AuxUpd}(\text{crs}, \text{ck}, x', \Pi', \text{AUX}'', r')$.

Definition 13 (Soundness). An EP for $R_{\mathcal{L}}$ is sound if for all PPT $\mathcal{A}$ and $\forall \lambda \in \mathbb{N}$, $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$, the probability that $\mathcal{A}(\text{crs}, \text{ck})$ outputs (x, Π) such that $x \notin \mathcal{L}$ but $\text{PVfy}(\text{crs}, \text{ck}, x, \Pi) = 1$ is negligible.

We also require addition and extension privacy. That is, proofs updated with PAdd (or PExt) are indistinguishable from proofs done from scratch using Prv .

Definition 14 (Addition Privacy). Consider the following experiment:

- $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$
- $(x, w, e^*, w^*, \alpha, \widetilde{r}_\alpha^*, \Pi^*, \text{AUX}^*, r) \leftarrow \mathcal{A}(\text{crs}, \text{ck})$

¹¹ Notice that it must hold that for each $i \in [k]$, $\phi(i) \neq \alpha$.

- If $(\text{ck}, x, w) \notin R_{\mathcal{L}}$ or $\text{PVfy}(\text{crs}, \text{ck}, x, \Pi^*) = 0$ or $\text{AuxVerify}(\text{crs}, \text{ck}, x, w, \text{AUX}^*, \Pi^*) = 0$ or $\alpha \in \text{Im}(\phi)$ (with ϕ taken from w) output $\perp$ and abort. Otherwise, sample $b \leftarrow \{0, 1\}$ and do the following:
 - If $b = 0$, $x' \leftarrow \text{AddT}_x(\text{ck}, x, e^*, \alpha, w^*, r_\alpha^*, r)$, $w' \leftarrow \text{AddT}_w(\text{ck}, w, w^*, \alpha, \widetilde{r}_\alpha^*, r)$ $(\Pi, \text{AUX}) \leftarrow \text{Prv}(\text{crs}, \text{ck}, x', w')$
 - If $b = 1$, $(\Pi, x', \text{aux}^*, r') \leftarrow \text{PAdd}(\text{crs}, \text{ck}, x, w^*, \alpha, e^*, \widetilde{r}_\alpha^*, \text{AUX}^*[\alpha], \Pi^*, r)$. Replace in AUX^* the value aux_α with aux^* . $\text{AUX} \leftarrow \text{AuxUpd}(\text{crs}, \text{ck}, x, \Pi, \text{AUX}^*, r')$
- $b' \leftarrow \mathcal{A}(x', \Pi, \text{AUX})$.

For every PPT $\mathcal{A}$ and $\lambda \in \mathbb{N}$, $\Pr[b = b'] \leq 1/2 + \text{negl}(\lambda)$.

Definition 15 (Extension Privacy). Consider the following experiment:

- $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$
- $(x, w, x_{n+1}, \widetilde{r}_{n+1}, \Pi^*, \text{AUX}^*, r) \leftarrow \mathcal{A}(\text{crs}, \text{ck})$
- If $(\text{ck}, x, w) \notin R_{\mathcal{L}}$ or $\text{PVfy}(\text{crs}, \text{ck}, x, \Pi^*) = 0$ or $\text{AuxVerify}(\text{crs}, \text{ck}, x, w, \text{AUX}^*, \Pi^*) = 0$ output $\perp$ and abort. Otherwise, sample $b \leftarrow \{0, 1\}$ and do the following:
 - If $b = 0$, $x' \leftarrow \text{ExtT}_x(\text{ck}, x, x_{n+1}, \widetilde{r}_{n+1}; r)$, $w' \leftarrow \text{ExtT}_w(\text{ck}, w, \widetilde{r}_{n+1}; r)$ $(\Pi, \text{AUX}) \leftarrow \text{Prv}(\text{crs}, \text{ck}, x', w')$
 - If $b = 1$ $(\Pi, x', \text{aux}^*, r') \leftarrow \text{PExt}(\text{crs}, \text{ck}, x, x_{n+1}, \widetilde{r}_{n+1}, \Pi^*, r)$. Append the value aux^* to AUX^* , $\text{AUX} \leftarrow \text{AuxUpd}(\text{crs}, \text{ck}, x', \Pi, \text{AUX}^*, r')$
- $b' \leftarrow \mathcal{A}(x', \Pi, \text{AUX})$.

For every PPT $\mathcal{A}$ and $\lambda \in \mathbb{N}$, $\Pr[b = b'] \leq 1/2 + \text{negl}(\lambda)$.

An EP may also satisfy (some of) the following additional properties.

Perfect Δ -Extraction is a perfect but mild extraction property. Indeed, instead of extracting the full witness, there exists an extractor which perfectly extracts the active indices and a function Δ of the rest of the witness.

Definition 16 (Perfect Δ -Extraction). There exists a PPT extractor Ext that for all $\lambda \in \mathbb{N}$, $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$, and all (x, Π) s.t. $\text{PVfy}(\text{crs}, \text{ck}, x, \Pi) = 1$ it hold that:

$$\Pr[(\text{ck}, (x, \phi', w')) \in \mathcal{L}' \mid (\phi', w' = (w'_1, \dots, w'_k))] \leftarrow \text{Ext}(\text{crs}, \text{td}, \text{ck}, x, \Pi)] = 1.$$

$$\begin{aligned} \text{Where } \mathcal{L}' = \{ & (\text{ck}, (k, x_1, \dots, x_n, \widetilde{c}_1, \dots, \widetilde{c}_n, e_1, \dots, e_k, \phi', w')) \mid \exists w = (w_1, \dots, w_k, \\ & \text{bit}_1, \dots, \text{bit}_n, \widetilde{r}_1, \dots, \widetilde{r}_n) \text{ s.t. } \forall i \in [n] \text{ bit}_i \in \{0, 1\} \wedge \widetilde{c}_i \leftarrow \text{Com}_{\text{ck}}(\text{bit}_i; \widetilde{r}_i) \wedge \\ & \sum_{i=1}^n \text{bit}_i = k \wedge \forall_{j=1}^k (\text{bit}_{\phi'(j)} = 1 \wedge F(\text{ck}, x_{\phi'(j)}, e_j, w_j) = 1) \wedge w' = \Delta(w_1, \dots, w_k) \}. \end{aligned}$$

We now define three WI variants useful in security proofs. Such flavors have some similarities but do not seem to imply each other.

Witness addition indistinguishability (WAI) ensures that two addition operations over the same α , but with different witnesses for x_α are indistinguishable.

Definition 17 (WAI). Consider the following experiment:

- $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$
- $(x, \Pi, \alpha, \text{aux}_\alpha, e, w_0, w_1, \widetilde{r}'_\alpha, r) \leftarrow \mathcal{A}(\text{crs}, \text{ck})$
- If $F(\text{ck}, x_\alpha, e, w_0) = 0$ or $F(\text{ck}, x_\alpha, e, w_1) = 0$ or $\text{PVfy}(\text{crs}, \text{ck}, x, \Pi) = 0$ output $\perp$ and abort. Otherwise, sample $b \leftarrow \{0, 1\}$ and do the following:
 - Parse $x = (k, x_1, \dots, x_n, \widetilde{c}_1, \dots, \widetilde{c}_n, e_1, \dots, e_n)$ and compute:
 - $(\Pi', x', \text{aux}'_\alpha, (r_1, \dots, r_n)) \leftarrow \text{PAdd}(\text{crs}, \text{ck}, x, w_b, \alpha, e, \widetilde{r}'_\alpha, \text{aux}_\alpha, \Pi, r)$
- $b' \leftarrow \mathcal{A}(x', \Pi', \text{aux}'_\alpha, (r_1, \dots, r_n))$.

For every PPT $\mathcal{A}$ and $\lambda \in \mathbb{N}$, $\Pr[b = b'] \leq 1/2 + \text{negl}(\lambda)$.

Fixed position witness indistinguishability (FPWI) guarantees that two proofs done for the same statement with two different witnesses sharing the same injective map ϕ are indistinguishable.

Definition 18 (FPWI). Consider the following experiment:

- $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$
- $(x, w_0, w_1) \leftarrow \mathcal{A}(\text{crs}, \text{ck})$
- Parse $w_i = (\phi^i, w_1^i, \dots, w_k^i, \tilde{r}_1^i, \dots, \tilde{r}_n^i)$ for $i \in \{0, 1\}$.
- If $(\text{ck}, x, w_0) \notin R_{\mathcal{L}}$ or $(\text{ck}, x, w_1) \notin R_{\mathcal{L}}$ or $\phi^0 \neq \phi^1$ output $\perp$ and abort. Otherwise, sample $b \leftarrow_{\$} \{0, 1\}$ and compute $(\Pi, \text{AUX}) \leftarrow \text{Prv}(\text{crs}, \text{ck}, x, w_b)$.
- $b' \leftarrow \mathcal{A}(\Pi, \text{AUX})$.

For every PPT $\mathcal{A}$ and $\lambda \in \mathbb{N}$, $\Pr[b = b'] \leq 1/2 + \text{negl}(\lambda)$.

Extended witness indistinguishability (EWI) guarantees that two proofs for the same statement generated using different witnesses are indistinguishable. Importantly, this holds even if the adversary gets the auxiliary values corresponding to the statements that are *inactive* w.r.t. both witnesses. Additionally, the switch variables are committed by the experiment.

Definition 19 (EWI). Consider the following experiment:

- $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$
- $(x', w'_0, w'_1) \leftarrow \mathcal{A}(\text{crs}, \text{ck})$
- Parse $x' = (k, x_1, \dots, x_n, e_1, \dots, e_k)$
- Parse $w'_b = (\phi^b, w_1^b, \dots, w_k^b)$ with $b \in \{0, 1\}$
- If $\exists b \in \{0, 1\}, j \in [k] : F(\text{ck}, x_{\phi^b(j)}, e_j, w_j^b) = 0$ output $\perp$ and abort.
- Sample $b' \leftarrow_{\$} \{0, 1\}$ and do the following:
 - Construct $x_{b'} = (k, x_1, \dots, x_n, \tilde{c}_1^{b'}, \dots, \tilde{c}_k^{b'}, e_1, \dots, e_k)$, and $w_{b'} = (\phi^{b'}, w_1^{b'}, \dots, w_k^{b'}, \tilde{r}_1^{b'}, \dots, \tilde{r}_n^{b'})$, where, for $i \in [n]$, for randomly sampled $\tilde{r}_i^{b'}$, set $\tilde{c}_i^{b'} \leftarrow \text{Com}_{\text{ck}}(1; \tilde{r}_i^{b'})$ if $i \in \text{Im}(\phi^{b'}(i))$, and $\tilde{c}_i^{b'} \leftarrow \text{Com}_{\text{ck}}(0; \tilde{r}_i^{b'})$
 - $(\Pi, (\text{aux}_1, \dots, \text{aux}_n)) \leftarrow \text{Prv}(\text{crs}, \text{ck}, x_{b'}, w_{b'})$
 - Set $S = ([n] \setminus (\text{Im}(\phi^0) \cup \text{Im}(\phi^1)))$, and $\text{AUX} = \{\text{aux}_i\}_{i \in S}$
- $b^* \leftarrow \mathcal{A}(x_{b'}, \Pi, \text{AUX})$.

For every PPT $\mathcal{A}$ and $\lambda \in \mathbb{N}$, $\Pr[b^* = b'] \leq 1/2 + \text{negl}(\lambda)$.

Extended zero knowledge (EZK) requires the existence of a simulator that, using a trapdoor, can simulate a proof and the auxiliary values of inactive statements.

Definition 20 (EZK). Let $\text{OEZK}(b, \text{crs}, \text{ck}, x, w)$ be defined as follows:

- Parse w as $(\phi, w_1, \dots, w_k, \tilde{r}_1, \dots, \tilde{r}_n)$
- If $(\text{ck}, x, w) \notin R_{\mathcal{L}}$ return $\perp$
- Else:
 - If $b = 0$, $(\Pi, (\text{aux}_1, \dots, \text{aux}_n)) \leftarrow \text{Prv}(\text{crs}, \text{ck}, x, w)$
 - If $b = 1$, $(\Pi, (\text{aux}_1, \dots, \text{aux}_n)) \leftarrow \text{Sim}_1(\text{crs}, \text{ck}, \text{td}, x)$
- Set $S = [n] \setminus \text{Im}(\phi)$ and $\text{AUX} = \{\text{aux}_i\}_{i \in S}$, with ϕ taken from w
- Return (Π, AUX) .

There exists a polynomial time simulator $\text{Sim} = (\text{Sim}_0, \text{Sim}_1)$ such that for every PPT $\mathcal{A}$ and $\lambda \in \mathbb{N}$, considering the following game:

- $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, $(\text{crs}, \text{td}) \leftarrow \text{Sim}_0(\text{gk})$, $b \leftarrow_{\$} \{0, 1\}$
- $b' \leftarrow \mathcal{A}^{\text{OEZK}(b, \text{crs}, \text{ck}, \cdot, \cdot)}(\text{crs}, \text{ck})$

$\Pr[b = b'] \leq 1/2 + \text{negl}(\lambda)$. Moreover, the crs output by Sim_0 is computationally indistinguishable to the crs output by CRSSetup .

5 Doubly-Authentication-Preventing Tags

A doubly-authentication-preventing tag (DAPT) consists of the following PPT algorithms and a poly-time relation R_{key} . Let $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$ be an implicit input to all algorithms. Given gk , a topic space $\mathcal{T}$, a message space $\mathcal{M}$, and a tag space $\mathcal{E}$ are implicitly defined.

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{gk})$: outputs a pair of public and secret keys (pk, sk) .
- $e \leftarrow \text{Tag}(\text{sk}, \tau, m)$: it is a *deterministic* algorithm that on input the secret sk , topic $\tau \in \mathcal{T}$, and message $m \in \mathcal{M}$, outputs a tag $e \in \mathcal{E}$.
- $0/1 \leftarrow \text{TagTrace}(e_0, e_1, m_0, m_1, \text{pk})$: it is a *deterministic* algorithm that on input two tags $e_0, e_1 \in \mathcal{E}$ on the same topic $\tau \in \mathcal{T}$ and different messages $m_0 \neq m_1 \in \mathcal{M}$, outputs 1 iff both tags were generated w.r.t. public key pk .

A DAPT has to satisfy all the following properties.

Definition 21 (Verifiability of Keys). *For all $\lambda \in \mathbb{N}$ and $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, it holds that $(\text{pk}, \text{sk}) \in \text{KeyGen}(\text{gk})$ iff $(\text{pk}, \text{sk}) \in R_{\text{key}}$. Additionally, we require that for all pk there exists a unique sk s.t. $(\text{pk}, \text{sk}) \in R_{\text{key}}$.*

Definition 22 (Tag Traceability). *For all $\lambda \in \mathbb{N}$, for any topic $\tau \in \mathcal{T}$, for any pair of messages $m_0 \neq m_1 \in \mathcal{M}$ it holds that, for any $(\text{pk}, \text{sk}) \in R_{\text{key}}$:*

$$\Pr \left[1 \leftarrow \text{TagTrace}(e_0, e_1, m_0, m_1, \text{pk}) \mid \begin{array}{l} e_0 \leftarrow \text{Tag}(\text{sk}, \tau, m_0) \\ e_1 \leftarrow \text{Tag}(\text{sk}, \tau, m_1) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Additionally, for all PPT $\mathcal{A}$ and $\lambda \in \mathbb{N}$ let us consider the following game:

$$\begin{aligned} (\tau, m_0, m_1, \text{pk}_0, \text{pk}_1, \text{sk}_0, \text{sk}_1) &\leftarrow \mathcal{A}(\lambda) \\ e_0 &\leftarrow \text{Tag}(\text{sk}_0, \tau, m_0) \\ e_1 &\leftarrow \text{Tag}(\text{sk}_1, \tau, m_1) \end{aligned}$$

We say that $\mathcal{A}$ wins the above game if $\text{sk}_0 \neq \text{sk}_1 \wedge (\text{pk}_0, \text{sk}_0) \in R_{\text{key}} \wedge (\text{pk}_1, \text{sk}_1) \in R_{\text{key}} \wedge (1 = \text{TagTrace}(e_0, e_1, m_0, m_1, \text{pk}_0)) \vee (1 = \text{TagTrace}(e_0, e_1, m_0, m_1, \text{pk}_1))$. We require that the probability of $\mathcal{A}$ winning the above game be $\text{negl}(\lambda)$.

Definition 23 (Pseudo-randomness). *For every PPT $\mathcal{A}$ and $\lambda \in \mathbb{N}$, the success probability in $\text{Exp}_{\mathcal{A}, \text{DAPT}}^{\text{TagPR}}(\lambda)$ (Fig. 9) is $\Pr[\text{Exp}_{\mathcal{A}, \text{DAPT}}^{\text{TagPR}}(\lambda) = \text{win}] \leq 1/2 + \text{negl}(\lambda)$.*

Notice that, in Fig. 9, $\mathcal{A}$ can ask for honestly computed tags via the flag $\text{honest} = 1$, independently of the value of b . The flag abrt is defined for convenience and can be ignored (i.e., $\mathcal{A}$ loses as soon as it makes an oracle call with $\text{abrt} = 1$).

Definition 24 (Non-Frameability). *For every PPT $\mathcal{A}$ and $\lambda \in \mathbb{N}$, the success probability in $\text{Exp}_{\mathcal{A}, \text{DAPT}}^{\text{TagNF}}(\lambda)$ (Fig. 10) is $\Pr[\text{Exp}_{\mathcal{A}, \text{DAPT}}^{\text{TagNF}}(\lambda) = \text{win}] \leq \text{negl}(\lambda)$.*

6 Our Tetrax

Before giving our construction, we define some useful poly-time relations. We first define the disjunctive relation R_D . The statement contains a commitment c , a DAPT public key pk_T , a topic τ , a message m , and a trapdoor theorem x_{trap} . A witness for such relation may be a tuple (e, r, sk_T) such that (e, r) is an opening of c and e is a correctly computed tag over the pair (τ, m) and the secret key sk_T . Alternatively, the witness

$\text{Exp}_{\mathcal{A}, \text{DAPT}}^{\text{TagPR}}(\lambda)$	$\text{OTag}(b, \text{sk}, \tau, m, \text{abrt}, \text{honest})$
1 : $\text{L}_{\text{tag}} \leftarrow \emptyset, b \leftarrow_{\$} \{0, 1\}$	1 : if $\text{abrt} = 1$:
2 : $\text{SKcorr} = 0, \text{gk} \leftarrow \mathcal{G}(1^\lambda)$	2 : $\text{SKcorr} = 1$
3 : $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{gk})$	3 : return sk
4 : $b' \leftarrow \mathcal{A}^{\text{OTag}(b, \text{sk}, \cdot, \cdot, \cdot)}(\text{pk})$	4 : if $\exists(\tau', m', e) \in \text{L}_{\text{tag}} \wedge \tau' = \tau \wedge m' = m$:
5 : if $\text{SKcorr} = 1$:	5 : return e
6 : return <i>lose</i>	6 : if $\text{honest} = 1$:
7 : $\forall(\tau', m', \cdot), (\tau, m, \cdot) \in \text{L}_{\text{tag}}$:	7 : $e \leftarrow \text{Tag}(\text{sk}, \tau, m)$
8 : if $\tau' = \tau \wedge m' \neq m$:	8 : $\text{L}_{\text{tag}} \leftarrow \text{L}_{\text{tag}} \cup \{(\tau, m, e)\}$
9 : return <i>lose</i>	9 : return e
10 : if $b' = b$:	10 : if $b = 0$: $e \leftarrow \text{Tag}(\text{sk}, \tau, m)$ else : $e \leftarrow_{\$} \mathcal{E}$
11 : return <i>win</i>	11 : $\text{L}_{\text{tag}} \leftarrow \text{L}_{\text{tag}} \cup \{(\tau, m, e)\}$
12 : else : return <i>lose</i>	12 : return e

Fig. 9. Tag pseudo-randomness game of DAPT.

can just be w_{trap} such that $(x_{\text{trap}}, w_{\text{trap}}) \in R_{\mathcal{L}_{\text{trap}}}$, where $\mathcal{L}_{\text{trap}}$ is a language in $\text{NP} \cap \text{co-NP}$. The introduction of x_{trap} is instrumental to prove security of our $\mathfrak{T}\text{etris}$.

$$R_D = \{(\text{ck}, x = (c, \text{pk}_T, \tau, m, x_{\text{trap}}), w) \mid (w = (e, r, \text{sk}_T) \wedge e \leftarrow \text{Tag}(\text{sk}_T, \tau, m) \wedge c \leftarrow \text{Com}_{\text{ck}}(e; r) \wedge (\text{pk}_T, \text{sk}_T) \in R_{\text{key}}) \vee (w = w_{\text{trap}} \wedge (x_{\text{trap}}, w_{\text{trap}}) \in R_{\mathcal{L}_{\text{trap}}})\}. \quad (2)$$

Then, we define the threshold relation over R_D as follows.

$$R_{D_{\text{tr}}} = \{(\text{ck}, x = (k, x_1, \dots, x_n, \tilde{c}_1, \dots, \tilde{c}_n), w = (\phi, w_1, \dots, w_k, \tilde{r}_1, \dots, \tilde{r}_n)) \mid \forall i \in [n] \exists \text{bit}_i : \tilde{c}_i \leftarrow \text{Com}_{\text{ck}}(\text{bit}_i; \tilde{r}_i) \wedge \text{bit}_i \in \{0, 1\} \wedge \sum_{i=1}^n \text{bit}_i = k \wedge \forall_{i=1}^k (\text{bit}_{\phi(i)} = 1 \wedge (\text{ck}, x_{\phi(i)}, w_i) \in R_D) \wedge \phi \text{ is an injective map } [t] \rightarrow [n]\} \quad (3)$$

Additionally, we define the extendable shuffle relation as

$$R_{\text{SH}} = \{(\text{ck}, x = (k, c_1, \dots, c_n, \tilde{c}_1, \dots, \tilde{c}_n, e_1, \dots, e_k), w = (\phi, r_1, \dots, r_k, \tilde{r}_1, \dots, \tilde{r}_n)) \mid \forall i \in [n] \exists \text{bit}_i : \tilde{c}_i \leftarrow \text{Com}_{\text{ck}}(\text{bit}_i; \tilde{r}_i) \wedge \text{bit}_i \in \{0, 1\} \wedge \sum_{i=1}^n \text{bit}_i = k \wedge \forall_{i=1}^k (\text{bit}_{\phi(i)} = 1 \wedge (c_{\phi(i)} \leftarrow \text{Com}_{\text{ck}}(e_i, r_i))) \wedge \phi \text{ is an injective map } [k] \rightarrow [n]\}. \quad (4)$$

The components of our $\mathfrak{T}\text{etris}$ are the following: (i) a doubly-authentication-preventing tag DAPT; (ii) an extendable non-interactive proof system for $R_{D_{\text{tr}}}$ that we call EP; (iii) an extendable non-interactive proof system for R_{SH} that we call SH; (iv) an IND-CPA public key encryption scheme PKE in which every public key has a unique secret key. Additionally, PKE must be homomorphic w.r.t. EP.AuxUpd and SH.AuxUpd. Notice that the algorithm PKE.Eval_1 (PKE.Eval_2) homomorphically evaluates EP.AuxUpd (SH.AuxUpd). We require EP and SH to work over the same commitment scheme Com_{ck} .

Our $\mathfrak{T}\text{etris}$ is shown in Fig. 11 and in Fig. 12. We refer to Sec. 1.2 for an intuitive description of our $\mathfrak{T}\text{etris}$. $\mathfrak{T}\text{etris.Setup}$ just runs the setup algorithm of all the used building blocks. Additionally, it samples a random instance x_{trap} such that $x_{\text{trap}} \notin \mathcal{L}_{\text{trap}}$ ¹². All the public parameters are generally omitted from the input of the algorithms. In our construction, it is crucial that EP and SH share the same lists of commitments $C = \{c_i\}_{i \in [n]}$ and $\tilde{C} = \{\tilde{c}_i\}_{i \in [n]}$.

¹² This can be done efficiently by sampling an instance in the complement of $\mathcal{L}_{\text{trap}}$.

$\text{Exp}_{\mathcal{A}, \text{DAPT}}^{\text{TagNF}}(\lambda)$	$\text{OTag}(\text{sk}, \tau, m)$
1 : $\text{L}_{\text{tag}} \leftarrow \emptyset, \text{gk} \leftarrow \mathcal{G}(1^\lambda)$	1 : $e \leftarrow \text{Tag}(\text{sk}, \tau, m)$
2 : $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{gk})$	2 : $\text{L}_{\text{tag}} \leftarrow \text{L}_{\text{tag}} \cup \{(\tau, m)\}$
3 : $(e_0, e_1, m_0, m_1) \leftarrow \mathcal{A}^{\text{OTag}(\text{sk}, \cdot, \cdot)}(\text{pk})$	3 : return e
4 : if $m_0 = m_1$:	
5 : return <i>lose</i>	
6 : $b' \leftarrow \text{TagTrace}(e_0, e_1, m_0, m_1, \text{pk})$	
7 : $\forall (\tau', m'), (\tau, m) \in \text{L}_{\text{tag}}$:	
8 : if $\tau' = \tau \wedge m' \neq m$:	
9 : return <i>lose</i>	
10 : if $b' = 1$: return <i>win</i> else : return <i>lose</i>	

Fig. 10. Tag non-frameability game of DAPT.

Moreover, the addition and the extension operations on EP and SH must preserve such connection. Therefore, every time we perform an addition or an extension with EP, we re-use the same transformation randomness for SH. In this way, we end up with two updated proofs that are still talking about two updated statements sharing the same common part (i.e., C' and $\tilde{C}'$). We use the square brackets notation to denote an element of a list, using 1 as the index of the first element. We treat sets as ordered lists, meaning that the elements of a set can be accessed, via the square bracket operator, in the same order they were added to the set. The relation $R_{\text{key}}^{\mathfrak{T}\text{ctris}}$ is defined as $R_{\text{key}}^{\mathfrak{T}\text{ctris}} = \{\text{pk} = (\text{pk}_T, \text{pk}_e), \text{sk} = (\text{sk}_T, \text{sk}_e) \mid (\text{pk}_T, \text{sk}_T) \in R_{\text{key}} \wedge (\text{pk}_e, \text{sk}_e) \in R_{\text{key}}^{\text{PKE}}\}$, where R_{key} and $R_{\text{key}}^{\text{PKE}}$ are the poly-time relation verifying well-formed DAPT and PKE keys respectively.

Instantiating our $\mathfrak{T}\text{ctris}$. We use the ElGamal encryption in $\hat{\mathbb{G}}$ and $\check{\mathbb{H}}$ as a perfectly binding commitment Com_{ck} , in Sec. 8 we build an extendable shuffle for such commitments, in App. C we present a EP for relation R_{Dtr} that is a simple adaptation of [8] and we set $\mathcal{L}_{\text{trap}}$ to be the language of Diffie-Hellman tuples. We point out that the property of perfect Δ -extraction is not defined in [8], therefore we prove it in App. C. We set

$$\Delta(w_1 \dots, w_k) = \begin{cases} \perp & \text{if } \exists i \in [k] : (x_{\text{trap}}, w_i) \in R_{\mathcal{L}_{\text{trap}}} \\ (e_1, \dots, e_k) & \text{otherwise} \end{cases} \quad (5)$$

and the property holds thanks to the perfect F-extraction of GS (see [28, Def. 3]).

Finally, in Sec. 7 we propose a DAPT. All the components are instantiated from the same bilinear groups and thus work well with each other. Additionally, since in both EP and SH, updating the auxiliary values (AuxUpd) simply consists of applying the group operation between two elements of $\hat{\mathbb{G}}$ or $\check{\mathbb{H}}$, we can use ElGamal instantiated in $\hat{\mathbb{G}}$ and $\check{\mathbb{H}}$ as PKE. We point out that since the setup of our SH bounds the length of the commitments list, our $\mathfrak{T}\text{ctris}$ supports rings of bounded size. Getting a linear-size $\mathfrak{T}\text{ctris}$ without this bound is an open problem. We rely on the SXDH (Ass. 4), Co-CDH (Ass. 5), subset permutation pairing (Ass. 6), and subset simultaneous pairing (Ass. 7) assumptions, along with the random oracle model (ROM).

6.1 Security of Our $\mathfrak{T}\text{ctris}$

In this section, we prove the security of our $\mathfrak{T}\text{ctris}$ via the following theorem.

Theorem 2. *Let DAPT be defined as in Sec. 5. Let EP be defined as in Sec. 4 for relation R_{Dtr} with WAI (Def. 17), FPWI (Def. 18), EWI (Def. 19), perfect Δ -extraction (Def. 16) where Δ is defined as in Eq. 5. Let SH be defined as in Sec. 4 for relation R_{SH} with EZK (Def. 20). Let PKE be an IND-CPA public key encryption scheme which is homomorphic w.r.t. EP.AuxUpd and SH.AuxUpd. Then, the scheme in Fig. 11,12 is a $\mathfrak{T}\text{ctris}$.*

Sign($\tau, m, \{\text{pk}_i\}_{i \in \mathcal{R}}, \text{sk}$)	KeyGen()
1 : $\text{A}_{\text{EP}} \leftarrow \emptyset, \text{A}_{\text{SH}} \leftarrow \emptyset, \text{E} \leftarrow \emptyset$ 2 : parse $\{\text{pk}_i\}_{i \in \mathcal{R}} = (\text{pk}_1, \dots, \text{pk}_n)$ 3 : $\forall i \in [n] : \text{parse } \text{pk}_i = (\text{pk}_e^i, \text{pk}_T^i)$ 4 : parse $\text{sk} = (\text{sk}_e, \text{sk}_T)$ 5 : if $\nexists \text{pk}_T^j, j \in [n]$ s.t. $(\text{pk}_T^j, \text{sk}_T) \in R_{\text{key}} :$ 6 : return $\perp$ 7 : $\text{E} \leftarrow \text{E} \cup (e \leftarrow \text{Tag}(\text{sk}_T, \tau, m))$ 8 : $r_j, \tilde{r}_j \leftarrow \mathbb{S}\{0, 1\}^\lambda$ 9 : $c_j \leftarrow \text{Com}_{\text{ck}}(e; r_j), \tilde{c}_j \leftarrow \text{Com}_{\text{ck}}(1; \tilde{r}_j)$ 10 : for $i \neq j \wedge i \in [n] :$ 11 : $r_i, \tilde{r}_i \leftarrow \mathbb{S}\{0, 1\}^\lambda$ 12 : $c_i \leftarrow \text{Com}_{\text{ck}}(0; r_i), \tilde{c}_i \leftarrow \text{Com}_{\text{ck}}(0; \tilde{r}_i)$ 13 : $\forall i \in [n] : x_i \leftarrow (c_i, \text{pk}_T^i, \tau, m, x_{\text{trap}})$ 14 : $w' \leftarrow (e, r_j, \text{sk}_T)$ 15 : $C \leftarrow (c_i)_{i \in [n]}, \tilde{C} \leftarrow (\tilde{c}_i)_{i \in [n]}$ 16 : $R \leftarrow (r_i)_{i \in [n]}, \tilde{R} \leftarrow (\tilde{r}_i)_{i \in [n]}$ 17 : $x_{\text{EP}} \leftarrow (1, x_1, \dots, x_n, \tilde{C})$ 18 : $x_{\text{SH}} \leftarrow (1, C, \tilde{C}, \text{E})$ 19 : $\phi(1) \leftarrow j$ 20 : $(\Pi_1, \text{AUX}_{\text{EP}}) \leftarrow \text{EP.Priv}(x_{\text{EP}}, (\phi, w', \tilde{R}))$ 21 : $(\Pi_2, \text{AUX}_{\text{SH}}) \leftarrow$ $\text{SH.Priv}(x_{\text{SH}}, (\phi, R, \tilde{R}))$ 22 : for $i \in [n] :$ 23 : if $i = j :$ 24 : $\text{A}_{\text{EP}}[i] \leftarrow \text{PKE.Enc}(\perp, \text{pk}_e^i)$ 25 : $\text{A}_{\text{SH}}[i] \leftarrow \text{PKE.Enc}(\perp, \text{pk}_e^i)$ 26 : else : 27 : $\text{A}_{\text{EP}}[i] \leftarrow \text{PKE.Enc}(\text{AUX}_{\text{EP}}[i], \text{pk}_e^i)$ 28 : $\text{A}_{\text{SH}}[i] \leftarrow \text{PKE.Enc}(\text{AUX}_{\text{SH}}[i], \text{pk}_e^i)$ 29 : $\sigma \leftarrow (1, \Pi_{\text{EP}}, \Pi_{\text{SH}}, \text{A}_{\text{EP}}, \text{A}_{\text{SH}}, C, \tilde{C}, \text{E})$ 30 : return σ	1 : $(\text{pk}_e, \text{sk}_e) \leftarrow \text{PKE.KeyGen}()$ 2 : $(\text{pk}_T, \text{sk}_T) \leftarrow \text{DAPT.KeyGen}()$ 3 : $(\text{pk} \leftarrow (\text{pk}_e, \text{pk}_T), \text{sk} \leftarrow (\text{sk}_e, \text{sk}_T))$ 4 : return (pk, sk) Join($\tau, m, \{\text{pk}_i\}_{i \in \mathcal{R}}, \text{sk}, \sigma$) 1 : parse $\{\text{pk}_i\}_{i \in \mathcal{R}} = (\text{pk}_1, \dots, \text{pk}_n)$ 2 : $\forall i \in [n] : \text{parse } \text{pk}_i = (\text{pk}_e^i, \text{pk}_T^i)$ 3 : parse $\text{sk} = (\text{sk}_e, \text{sk}_T)$ 4 : if $\nexists \text{pk}_T^j, j \in [n]$ s.t. $(\text{pk}_T^j, \text{sk}_T) \in R_{\text{key}} :$ 5 : return $\perp$ 6 : parse $\sigma = (k, \Pi_{\text{EP}}, \Pi_{\text{SH}}, \text{A}_{\text{EP}}, \text{A}_{\text{SH}}, C, \tilde{C}, \text{E})$ 7 : $\text{E} \leftarrow \text{E} \cup (e \leftarrow \text{Tag}(\text{sk}_T, \tau, m))$ 8 : $r_j, \tilde{r}_j \leftarrow \mathbb{S}\{0, 1\}^\lambda$ 9 : $c_j \leftarrow \text{Com}_{\text{ck}}(e; r_j), \tilde{c}_j \leftarrow \text{Com}_{\text{ck}}(1; \tilde{r}_j)$ 10 : $\text{aux}_{\text{EP}} \leftarrow \text{PKE.Dec}(\text{A}_{\text{EP}}[j], \text{sk}_e)$ 11 : $\text{aux}_{\text{SH}} \leftarrow \text{PKE.Dec}(\text{A}_{\text{SH}}[j], \text{sk}_e)$ 12 : $x_i \leftarrow (C[i], \text{pk}_T^i, \tau, m, x_{\text{trap}}) \forall i \in [n]$ 13 : $w' \leftarrow (e, r_j, \text{sk}_T^j)$ 14 : $x_{\text{EP}} \leftarrow (k, x_1, \dots, x_n, \tilde{C})$ 15 : $x_{\text{SH}} \leftarrow (k, C, \tilde{C}, \text{E})$ 16 : Sample transformation randomness R_{Add} 17 : $(\Pi'_{\text{EP}}, x'_{\text{EP}}, \text{aux}'_j, R_{\text{EP}}) \leftarrow$ $\text{EP.PAdd}(x_{\text{EP}}, w, j, \cdot, \tilde{r}_j, \text{aux}_{\text{EP}}, \Pi_{\text{EP}}, R_{\text{Add}})$ 18 : $(\Pi'_{\text{SH}}, (k+1, C', \tilde{C}', \text{E}), \text{aux}'_j, R_{\text{SH}}) \leftarrow$ $\text{SH.PAdd}(x_{\text{SH}}, r_j, j, e, \tilde{r}_j, \text{aux}_{\text{SH}}, \Pi_{\text{SH}}, R_{\text{Add}})$ 19 : $\text{A}_{\text{EP}}[j] \leftarrow \text{PKE.Enc}(\perp, \text{pk}_e^j)$ 20 : $\text{A}_{\text{SH}}[j] \leftarrow \text{PKE.Enc}(\perp, \text{pk}_e^j)$ 21 : for $i \in [n] :$ 22 : $\text{A}_{\text{EP}}[i] \leftarrow \text{PKE.Eval}_1(\Pi_{\text{EP}}, \text{A}_{\text{EP}}[i], R_{\text{EP}}, \text{pk}_e^i)$ 23 : $\text{A}_{\text{SH}}[i] \leftarrow \text{PKE.Eval}_2(\Pi_{\text{SH}}, \text{A}_{\text{SH}}[i], R_{\text{SH}}, \text{pk}_e^i)$ 24 : $\sigma \leftarrow (k+1, \Pi'_{\text{EP}}, \Pi'_{\text{SH}}, \text{A}_{\text{EP}}, \text{A}_{\text{SH}}, C', \tilde{C}', \text{E})$ 25 : return σ

Fig. 11. KeyGen, Sign, and Join algorithms of our $\mathfrak{T}$ etris.

Extend($\tau, m, \sigma, \{\text{pk}_i\}_{i \in \mathcal{R}}, \text{pk}^*$)	Verify($t, \tau, m, \{\text{pk}_i\}_{i \in \mathcal{R}}, \sigma$)
<pre> 1 : if $\text{pk}^* \in \{\text{pk}_i\}_{i \in \mathcal{R}}$: return $\perp$ 2 : parse $\{\text{pk}_i\}_{i \in \mathcal{R}} = (\text{pk}_1, \dots, \text{pk}_n)$ 3 : $\forall i \in [n]$: parse $\text{pk}_i = (\text{pk}_e^i, \text{pk}_T^i)$ 4 : parse $\text{pk}^* = (\text{pk}_e^{n+1}, \text{pk}_T^{n+1})$ 5 : parse $\sigma = (k, \Pi_{\text{EP}}, \Pi_{\text{SH}}, \text{A}_{\text{EP}}, \text{A}_{\text{SH}}, C, \tilde{C}, E)$ 6 : $\forall i \in [n]$: $x_i \leftarrow (C[i], \text{pk}_T^i, \tau, m, x_{\text{trap}})$ 7 : $x_{\text{EP}} \leftarrow (k, x_1, \dots, x_n, \tilde{C})$ 8 : $r_{n+1}, \widetilde{r_{n+1}} \leftarrow \\$_\{0, 1\}^\lambda$ 9 : $c_{n+1} \leftarrow \text{Com}_{\text{ck}}(0; r_{n+1})$ 10 : $\widetilde{c_{n+1}} \leftarrow \text{Com}_{\text{ck}}(0; \widetilde{r_{n+1}})$ 11 : $x_{\text{SH}} \leftarrow (k, C, \tilde{C}, E)$ 12 : Sample transformation randomness R_{Ext} 13 : $(\Pi'_{\text{EP}}, x'_{\text{EP}}, \text{aux}_{\text{EP}}, R_{\text{EP}}) \leftarrow$ EP.PExt($x_{\text{EP}}, x_{n+1}, \widetilde{r_{n+1}}, \Pi_{\text{EP}}, R_{\text{Ext}}$) 14 : $(\Pi'_{\text{SH}}, (k, C', \tilde{C}', E), \text{aux}_{\text{SH}}, R_2) \leftarrow$ SH.PExt($x_{\text{SH}}, c_{n+1}, \widetilde{r_{n+1}}, \Pi_{\text{SH}}, R_{\text{Ext}}$) 15 : $\text{A}_{\text{EP}}[n+1] \leftarrow \text{PKE.Enc}(\text{aux}_{\text{EP}}, \text{pk}_e^{n+1})$ 16 : $\text{A}_{\text{SH}}[n+1] \leftarrow \text{PKE.Enc}(\text{aux}_{\text{SH}}, \text{pk}_e^{n+1})$ 17 : for $i \in [n+1]$: 18 : $\text{A}_{\text{EP}}[i] \leftarrow \text{PKE.Eval}_1(\Pi_1, \text{A}_{\text{EP}}[i], R_1, \text{pk}_e^i)$ 19 : $\text{A}_{\text{SH}}[i] \leftarrow \text{PKE.Eval}_2(\Pi_2, \text{A}_{\text{SH}}[i], R_2, \text{pk}_e^i)$ 20 : $\sigma \leftarrow (k, \Pi'_{\text{EP}}, \Pi'_{\text{SH}}, \text{A}_{\text{EP}}, \text{A}_{\text{SH}}, C', \tilde{C}', E)$ 21 : return σ </pre>	<pre> 1 : parse $\{\text{pk}_i\}_{i \in \mathcal{R}} = (\text{pk}_1, \dots, \text{pk}_n)$ 2 : $\forall i \in [n]$: parse $\text{pk}_i = (\text{pk}_e^i, \text{pk}_T^i)$ 3 : parse $\sigma = (k, \Pi_{\text{EP}}, \Pi_{\text{SH}}, \text{A}_{\text{EP}}, \text{A}_{\text{SH}}, C, \tilde{C}, E)$ 4 : if $k < t$: 5 : return 0 6 : else : 7 : $\forall i \in [n]$: $x_i \leftarrow (C[i], \text{pk}_T^i, \tau, m, x_{\text{trap}})$ 8 : $x_{\text{EP}} \leftarrow (k, x_1, \dots, x_n, \tilde{C})$ 9 : $x_{\text{SH}} \leftarrow (k, C, \tilde{C}, E)$ 10 : return $(\text{EP.PVfy}(x_{\text{EP}}, \Pi_{\text{EP}}) \wedge$ SH.PVfy($x_{\text{SH}}, \Pi_{\text{SH}}$)) </pre>
	<pre> Trace($\tau, m_1, m_2, \sigma_1, \sigma_2, \{\text{pk}_i\}_{i \in \mathcal{R}_1}, \{\text{pk}_i\}_{i \in \mathcal{R}_2}$) 1 : $\text{PK}_1 \leftarrow \{\text{pk}_i\}_{i \in \mathcal{R}_1} \cap \{\text{pk}_i\}_{i \in \mathcal{R}_2}$ 2 : $\text{PK}_2 \leftarrow \emptyset, n = 0$ 3 : parse $\sigma_1 = (k_1, \Pi_{\text{EP}}^1, \Pi_{\text{SH}}^1, \text{A}_{\text{EP}}^1, \text{A}_{\text{SH}}^1, C_1, \tilde{C}_1, E_1)$ 4 : parse $\sigma_2 = (k_2, \Pi_{\text{EP}}^2, \Pi_{\text{SH}}^2, \text{A}_{\text{EP}}^2, \text{A}_{\text{SH}}^2, C_2, \tilde{C}_2, E_2)$ 5 : $\forall \text{pk} \in \text{PK}_1, e_1 \in E_1, e_2 \in E_2$: 6 : parse $\text{pk} = (\text{pk}_e, \text{pk}_T)$ 7 : if $m_1 = m_2$: 8 : if $e_1 = e_2$: 9 : $n \leftarrow n + 1$ 10 : else if TagTrace($e_1, e_2, m_1, m_2, \text{pk}_T$) = 1 : 11 : $\text{PK}_2 \leftarrow \text{PK}_2 \cup \text{pk}$ 12 : if $\text{PK}_2 > 0$: 13 : return PK_2 14 : if $n > 0$: 15 : return (linked, n) 16 : return indep </pre>

Fig. 12. Extend, Verify, and Trace algorithms of our $\mathfrak{T}\mathfrak{e}\mathfrak{t}\mathfrak{t}\mathfrak{i}\mathfrak{s}$.

We prove Thm. 2 via the following lemmas. Verifiability of keys (Def. 2) follows from the verifiability of keys of underlying schemes (Def. 21 and Def. 27).

Lemma 3. *Let EP be defined as in Sec. 4 for relation R_{Dtr} , and let SH be defined as in Sec. 4 for relation R_{SH} . Then the scheme in Fig. 11,12 is correct (Def. 1).*

Proof. Let us assume that lad is not well-formed, then $Proc$ (Fig. 3) would return $\perp$ as required by the definition. On the other end, if the ladder is well-formed, $Proc$ will return a triple $(\Sigma, Th, \mathcal{R})$, where $\Sigma = \{\sigma_1, \dots, \sigma_\ell\}$, $Th = \{t_1, \dots, t_\ell\}$. Recall that for each element in $t_i \in Th$ with $i \in [\ell]$, we have that $t_1 = 1$ and $t_i = t_{i-1}$ if $action_i = \text{Extend}$, and $t_i = t_{i-1} + 1$ if $action_i = \text{Join}$. We now argue that $\bigwedge_{j=1}^{\ell} \text{Verify}(t_j, \tau, m, \{pk_i\}_{i \in \mathcal{R}}, \sigma_j) = 1$ (Fig. 12). Let us now consider all the possible actions contained in the ladder:

1. $action_1$ of lad is always a **Sign** action, which produces a signature of the format $\sigma_1 \leftarrow (k = 1, \Pi'_{EP}, \Pi'_{SH}, A_{EP}, A_{SH}, C', \tilde{C}', E)$. Therefore, the check of line 4 (Fig. 12) of $\text{Verify}(t_1, \tau, m, \{pk_i\}_{i \in \mathcal{R}}, \sigma_1)$ will not return 0.
2. If $action_i$ of lad is an **Extend** action, it produces a signature of the format $\sigma_i \leftarrow (k = t_{i-1}, \Pi'_{EP}, \Pi'_{SH}, A_{EP}, A_{SH}, C', \tilde{C}', E)$. Therefore, the check of line 4 (Fig. 12) of $\text{Verify}(t_i, \tau, m, \{pk_i\}_{i \in \mathcal{R}}, \sigma_i)$ will not return 0, since $t_i = t_{i-1}$.
3. If $action_i$ of lad is a **Join** action, it produces a signature of the format $\sigma_i \leftarrow (k = t_{i-1} + 1, \Pi'_{EP}, \Pi'_{SH}, A_{EP}, A_{SH}, C', \tilde{C}', E)$. Therefore, the check of line 4 (Fig. 12) of $\text{Verify}(t_i, \tau, m, \{pk_i\}_{i \in \mathcal{R}}, \sigma_i)$ will not return 0, since $t_i = t_{i-1} + 1$.

It remains to notice that in all of the above cases, the proof generation (in $Proc$) and verification (in Verify) algorithms of EP and SH are used with the intended inputs. Therefore, the completeness of EP and SH guarantees that the check of line 10 of Verify (Fig. 12) returns 1.

Lemma 4. *Let DAPT be defined as in Sec. 5. Then, the scheme in Fig. 11,12 has tracing correctness (Def. 3).*

Proof. We have to consider the following cases:

1. if the set of overlapping signers is empty (i.e., $lad_0.S \cap lad_1.S = \emptyset$), then Trace must return **indep** with overwhelming probability;
2. if $m_0 \neq m_1$, then Trace must return the set of overlapping signers $lad_0.S \cap lad_1.S$ with overwhelming probability;
3. if $m_0 = m_1$, then Trace must return (linked, n) where n is the number of overlapping signers (i.e., $n = |lad_0.S \cap lad_1.S|$) with overwhelming probability.

We now argue that the above requirements hold:

- 1 : Let us assume that there exists a PPT adversary $\mathcal{A}$ that returns $(\tau, m_0, m_1, lad_0, lad_1, L_{keys})$ such that $lad_0.S \cap lad_1.S = \emptyset$, but Trace does not return **indep**. Trace returns a value different from **indep** if and only if $\text{TagTrace}(e_0, e_1, m_0, m_1, pk_T) = 1$ for some pk_T (line 10 of Trace Fig. 12). Let pk_T, e_0, e_1, m_0, m_1 be the values in the loop at line 5 of Trace (Fig. 12) in the algorithm Trace for which $\text{TagTrace}(e_0, e_1, m_0, m_1, pk_T) = 1$. Notice that, due to the check at line 6, of $\text{Exp}_{\mathcal{A}, \Sigma_{ctris}}^{\text{TCorr}}(\lambda)$ (Fig. 5) if $lad_0.S \cap lad_1.S = \emptyset$ then there is only one signer that has public key containing pk_T . Since all pk_T are different, we look for all $sk_T^i, sk_T^j \in L_{keys}$ such that $e_0 \leftarrow \text{Tag}(sk_T^i, \tau, m_0)$ and $e_1 \leftarrow \text{Tag}(sk_T^j, \tau, m_1)$. We use $\mathcal{A}$ to define an adversary $\mathcal{B}$ that breaks tag traceability of DAPT with the same probability of $\mathcal{A}$ winning in the tracing correctness game. $\mathcal{B}$ runs $\mathcal{A}$ and outputs $(\tau, m_0, m_1, pk_T^i, pk_T^j, sk_T^i, sk_T^j)$, where pk_T^i, pk_T^j are the public keys associated with sk_T^i, sk_T^j . Notice that $\mathcal{B}$ wins the tag traceability game, with the same advantage of $\mathcal{A}$, since the secret and public keys used by $\mathcal{A}$ belong to $R_{key}^{\Sigma_{ctris}}$, otherwise the check at line 4 of $\text{Exp}_{\mathcal{A}, \Sigma_{ctris}}^{\text{TCorr}}(\lambda)$ (Fig. 5) fails.

- 2** : Let us assume that there exists a PPT adversary $\mathcal{A}$ that returns, with non-negligible probability, $(\tau, m_0, m_1, \text{lad}_0, \text{lad}_1, \text{L}_{\text{keys}})$ such that $m_0 \neq m_1$ and $\text{lad}_0.\mathcal{S} \cap \text{lad}_1.\mathcal{S} \neq \emptyset$, but Trace returns $\text{PK} \neq \text{lad}_0.\mathcal{S} \cap \text{lad}_1.\mathcal{S}$. Let tinfo_b , for $b \in \{0, 1\}$, be the tracing information produced by $\text{Proc}(\tau, m_b, \text{L}_{\text{keys}}, \text{lad}_b)$. For each public key $\text{pk}_T \in \text{lad}_0.\mathcal{S} \cap \text{lad}_1.\mathcal{S}$ such that $\text{pk}_T \notin \text{PK}$, let $e^b \leftarrow \text{Tag}(\text{sk}_T, \tau, m_b)$, where sk_T is the secret key associated with pk_T taken from L_{keys} , it must be that $e^b \in \text{tinfo}_b$. Therefore, $\text{TagTrace}(e_0, e_1, m_0, m_1, \text{pk}_T) = 0$ only with negligible probability due to tag traceability (Def. 22), implying that the check of line 10 of Trace (Fig. 12) can fail on input $(e_0, e_1, m_0, m_1, \text{pk}_T)$ only with negligible probability. Thus, $\mathcal{A}$ can win only with the same probability, reaching a contradiction.
- 3** : The analysis is identical to the previous case.

Lemma 5. *Let DAPT be defined as in Sec. 5. Let EP be defined as in Sec. 4 for relation R_{Dtr} with perfect Δ -extraction (Def. 16) where Δ is defined as in Eq. 5. Let SH be defined as in Sec. 4 for relation R_{SH} . Then, the scheme in Fig. 11,12 has signers extraction (Def. 4).*

Proof. Let $(\text{pp} = (x_{\text{trap}}, \text{crs}_{\text{SH}}, \text{crs}_{\text{EP}}), \text{td} = (\text{td}_{\text{SH}}, \text{td}_{\text{EP}}))$ be the public parameters and trapdoor of our $\mathfrak{T}\mathfrak{e}\mathfrak{t}\mathfrak{r}\mathfrak{i}\mathfrak{s}$. The extractor $\text{Ext}(\text{td}, \tau, m, t, \{\text{pk}_i\}_{i \in \mathcal{R}}, \sigma)$ is defined as follows:

- Parse $\sigma = (\mu = (t, \Pi_{\text{EP}}, \Pi_{\text{SH}}, \text{A}_{\text{EP}}, \text{A}_{\text{SH}}, C, \tilde{C}), \text{tinfo} = \text{E})$.
- Let $x_{\text{EP}} = (t, x_1, \dots, x_n, \tilde{C})$ where $x_i = (C[i], \text{pk}_T^i, \tau, m, x_{\text{trap}})$ be the statement of EP.
- $(\phi, \text{E}' = (e'_1, \dots, e'_k)) \leftarrow \text{EP.Ext}(\text{crs}_{\text{EP}}, \text{td}_{\text{EP}}, \text{ck}, \Pi_{\text{EP}}, x)$.
- If E' is a permutation of E do the following:
 - Initialize $\text{PK}, \text{Ind} \leftarrow \emptyset$; for all $i \in [t]$ $\text{Ind} \leftarrow \text{Ind} \cup \phi(i)$ and $\text{PK} \leftarrow \text{PK} \cup \text{pk}_{\phi(i)}$.
 - Return (PK, Ind) .
- Else return $\perp$.

First, let us analyse the case in which Ext returns (PK, Ind) . According to relations R_{D} (see (2)) and R_{Dtr} (see (3)) Π_{EP} proves that:

1. for all $i \in [n]$, $\tilde{C}[i]$ is either a commitment to 1 (if i is a signer), or to 0 (if i is not a signer);
2. the sum of the values committed in $\tilde{C}$ is t ;
3. if i is a signer, $C[i]$ is a commitment of a tag e with $e \leftarrow \text{Tag}(\text{sk}_T^i, \tau, m)$, where sk_T^i is the DAPT secret key of the signer i .

This follows from the fact that $x_{\text{trap}} \notin \mathcal{L}_{\text{trap}}$, thus the disjunctive relation (2) can only be satisfied by correctly generated tags. Recall that EP has perfect Δ -extraction and Com_{ck} is perfectly binding. Therefore, the range of the extracted ϕ is uniquely determined and corresponds to the indices of the active signers. Observe that each element in E is produced by the *deterministic* algorithm Tag and every DAPT public key has only one corresponding secret key, therefore each element e in E (i.e., tinfo) is uniquely determined by $(\text{pk}, \phi, \tau, m)$ for some $\text{pk} \in \text{PK}$. We can therefore conclude that every accepting signature $\sigma = (\mu, \text{tinfo})$ has a *unique* tinfo determined by t, τ, m , and the set of signers. Thus, there must exist a ladder lad producing a final signature $\sigma' = (\mu', \text{tinfo}')$ with $\text{tinfo}' = \text{E} = \text{tinfo}$. This ladder contains sign and join operations which are done using, in an appropriate order, the signers with indices in Ind .

Let us now consider the case in which Ext returns $\perp$ (i.e., Π_{EP} and Π_{SH} verifies, but E' is not a permutation of E that happens if the elements in the SH statements are not the one committed in EP). Assume that there exists a PPT $\mathcal{A}$ producing an accepting signature that makes Ext return $\perp$ with non-negligible probability. According to relation R_{SH} (see relation (4)), Π_{SH} proves that the following tuple $x_{\text{SH}} = (k, C, \tilde{C}, \text{E})$ has the following properties:

1. for all $i \in [n]$, $\tilde{C}[i]$ is either a commitment to 1 (if i is a signer), or to 0 (if i is not a signer);
2. the sum of the values committed in $\tilde{C}$ is t ;
3. $\phi : [t] \rightarrow [n]$ is injective;
4. for all $i \in [t]$, $C[\phi(i)]$ is a commitment to $\text{E}[i]$.

Observe that Ext returns $\perp$ if and only if the point 4 above is not satisfied. However, this means that $x_{\text{SH}} \notin \mathcal{L}_{\text{SH}}$ but Π_{SH} is accepting, contradicting the (computational) soundness of SH.

Lemma 6. Let DAPT be defined as in Sec. 5. Let EP be defined as in Sec. 4 for relation R_{Dtr} with WAI (Def. 17), FPWI (Def. 18), EWI (Def. 19). Let SH be defined as in Sec. 4 for relation R_{SH} with EZK (Def. 20). Let PKE be an IND-CPA scheme which is homomorphic w.r.t. EP.AuxUpd and SH.AuxUpd. Then the scheme in Fig. 11,12 is anonymous (Def. 6).

Proof. Let $\mathcal{B}$ be the reduction that internally runs $\mathcal{A}$. Every $\mathcal{H}_i$ is identical to $\mathcal{H}_{i-1}$ except the changes that are explicitly reported in the description of $\mathcal{H}_i$.

- $\mathcal{H}_0$: This is exactly the anonymity game of Fig. 6 with $b = 0$.
- $\mathcal{H}_1$: Instead of sampling x_{trap} such that $x_{\text{trap}} \notin \mathcal{L}_{\text{trap}}$, $\mathcal{B}$ samples $(x_{\text{trap}}, w_{\text{trap}}) \in R_{\mathcal{L}_{\text{trap}}}$. This hybrid is computationally indistinguishable from the previous one thanks to the hardness of $\mathcal{L}_{\text{trap}}$.
- $\mathcal{H}_2$: $\mathcal{B}$ switches the common reference string crs_{SH} to $(\text{crs}_{\text{SH}}, \text{td}_{\text{crs}_{\text{SH}}}) \leftarrow \text{SH.Sim}_0(\text{gk})$. This hybrid is computationally indistinguishable from the previous one because of the computational indistinguishability of honest and simulated crs_{SH} imposed by EZK (Def. 20) of SH. $\mathcal{B}$ includes the crs_{SH} to be distinguished in the public parameters given to $\mathcal{A}$. Then, $\mathcal{B}$ replies to all queries of $\mathcal{A}$ by simply running SH.Priv. Notice that $\mathcal{B}$ perfectly simulates the view of $\mathcal{A}$ in both hybrids, thus $\mathcal{B}$ can use the output of a successful distinguisher to win in the EZK game with the same success probability.
- $\mathcal{H}_3$: $\mathcal{B}$ replies to $\text{OSign}(\tau, m, i, \mathcal{R})$ queries using w_{trap} instead of sk_T^i as a witness to compute the proof Π_{EP} . This hybrid is computationally indistinguishable from the previous one thanks to the FPWI property (Def. 18) of EP. Let $\mathcal{C}$ be the challenger of Def. 18. Upon a sign query, $\mathcal{B}$ creates $c_i \leftarrow \text{Com}_{\text{ck}}(e_i; r_i)$ with $e_i \leftarrow \text{Tag}(\text{sk}_T^i, \tau, m)$, $\tilde{c}_i \leftarrow \text{Com}_{\text{ck}}(1; \tilde{r}_i)$, $\tilde{c}_j \leftarrow \text{Com}_{\text{ck}}(0; \tilde{r}_j)$ and $c_j \leftarrow \text{Com}_{\text{ck}}(0; r_j)$ for $j \in [n] \setminus i$. $\mathcal{B}$ then sets $x_j = (c_j, \text{pk}_T^j, \tau, m, x_{\text{trap}})$ with $j \in [n]$. Since each OSig queries are performed only on not corrupted indices i , it holds that $\mathcal{B}$ can choose the secret keys for the signer with index i . $\mathcal{B}$ sends $(1, x_1, \dots, x_n, \tilde{c}_1, \dots, \tilde{c}_n, w_0, w_1)$ to $\mathcal{C}$ with $w_0 = (\phi, (e_i, r_i, \text{sk}_T^i), \tilde{r}_1, \dots, \tilde{r}_n)$, $w_1 = (\phi, w_{\text{trap}}, \tilde{r}_1, \dots, \tilde{r}_n)$, and $\phi = \{1 \rightarrow i\}$. $\mathcal{C}$ replies with a proof Π_{EP} , and a list of auxiliary values AUX. $\mathcal{B}$ can now construct a reply to the sign query $\sigma = (1, \Pi_{\text{EP}}, \Pi_{\text{SH}}, \text{A}_{\text{EP}}, \text{A}_{\text{SH}}, C, \tilde{C}, \{e_i\})$ where Π_{EP} is the one received by $\mathcal{C}$, the auxiliary values related to Π_{EP} received from $\mathcal{C}$ are encrypted by $\mathcal{B}$ in A_{EP} , C and $\tilde{C}$ contain the commitments computed as above, Π_{SH} with its encrypted auxiliary values are regularly computed by $\mathcal{B}$, while $e_i \leftarrow \text{Tag}(\text{sk}_T^i, \tau, m)$. $\mathcal{B}$ perfectly simulates the view of $\mathcal{A}$ in both hybrids and thus $\mathcal{B}$ can use the output of a successful distinguisher to win in the FPWI game with the same success probability.
- $\mathcal{H}_4$: $\mathcal{B}$ replies to $\text{OJoin}(\tau, m, \mathcal{R}, i, \sigma)$ queries using w_{trap} instead of sk_T^i as a witness to compute the add operation over the proof Π_{EP} contained in σ . This hybrid is computationally indistinguishable from the previous one thanks to the WAI (Def. 17) of EP. Let $\mathcal{C}$ be the challenger of the WAI game. Upon a join query, $\mathcal{B}$ parses the queried $\sigma = (k, \Pi_{\text{EP}}, \Pi_{\text{SH}}, \text{A}_{\text{EP}}, \text{A}_{\text{SH}}, C, \tilde{C}, E)$ and decrypts the auxiliary value aux_i in $\text{A}_{\text{EP}}[i]$ (recall that signer with index i is not corrupted, therefore $\mathcal{B}$ knows his secret keys). Then $\mathcal{B}$ defines $x = (k, x_1, \dots, x_n, \tilde{C})$ with $x_j = (c_j, \text{pk}_T^j, \tau, m, x_{\text{trap}})$ where c_j is the j -th element of C , for all $j \in [n]$. $\mathcal{B}$ samples a randomness $\tilde{r}'_i$ and transformation randomness R_{Add} and sends $(x, \Pi_{\text{EP}}, i, \text{aux}_i, e_i, w_0, w_1, \tilde{r}'_i, R_{\text{Add}})$ to $\mathcal{C}$, where $w_0 = (e_i, r_i, \text{sk}_T^i)$ and $w_1 = w_{\text{trap}}$. Then, $\mathcal{B}$ gets $x', \Pi'_{\text{EP}}, \text{aux}'_i, (r_1, \dots, r_n)$ from $\mathcal{C}$. $\mathcal{B}$ now computes the reply to the query $\sigma' = (k+1, \Pi'_{\text{EP}}, \Pi'_{\text{SH}}, \text{A}'_{\text{EP}}, \text{A}'_{\text{SH}}, C', \tilde{C}', E')$ in the following way. It sets Π'_{EP} as the one received from $\mathcal{C}$, it updates the auxiliary values in A_{EP} using $(r_1, \dots, r_n)$ and PKE.Eval_1 , it sets C' and $\tilde{C}'$ as the one contained in x' . It computes all the elements related to the shuffle proofs as in the previous hybrid, using commitment randomness $\tilde{r}'_i$ and transformation randomness R_{Add} . $\mathcal{B}$ perfectly simulates the view of $\mathcal{A}$ in both hybrids and thus $\mathcal{B}$ can use the output of a successful distinguisher to win in the WAI game with the same success probability.
- $\mathcal{H}_5$: When processing lad_0 , $\mathcal{B}$ processes the sign action in the ladder with the same modification of $\mathcal{H}_3$. Indistinguishability from the previous hybrid can be argued as above.
- $\mathcal{H}_6$: When processing the ladder, $\mathcal{B}$ processes the join actions in the ladder with the same modification of $\mathcal{H}_4$. Indistinguishability from the previous hybrid can be argued as above.
- $\mathcal{H}_7$: Let s be the index of the first signer in lad_0 . After that $\mathcal{A}$ returns $(\tau^*, m^*, \text{lad}_0^*, \text{lad}_1^*)$ (line 4 of $\text{Exp}_{\mathcal{A}, \mathcal{F}_{\text{ettis}}}^{\text{Anon}}(\lambda)$), the first action of lad_0 that is $\text{Sign}(\tau^*, m, s, \mathcal{R})$ is processed by $\mathcal{B}$ setting e as $e \leftarrow \text{sk}_{\mathcal{E}}$ instead of $e \leftarrow \text{Tag}(\text{sk}_T^s, \tau^*, m)$. This hybrid is computationally indistinguishable from the previous one

thanks to the pseudo-randomness property of the DAPT (Def. 23). Let $\mathcal{C}$ be the challenger of Def. 23. Every time $\mathcal{A}$ makes a **KeyGen** query with index i , $\mathcal{B}$ generates the pk_T^i starting a new interaction with a pseudo-randomness tag game. Every time $\mathcal{A}$ makes a **Sign** or a **Join** query on an index i with public key pk_T^i for a topic τ at line 4 of $\text{Exp}_{\mathcal{A}, \mathcal{T}_{\text{etris}}}^{\text{Anon}}(\lambda)$, $\mathcal{B}$ queries **OTag** (Fig. 9) to obtain the tag for (sk_T^i, τ, m) to compute the signature. Notice that for each different index i , $\mathcal{B}$ is interacting with a different **OTag** in order to obtain the output of **OTag**($\text{sk}_T^i, \tau, m, \text{abrt} = 0, \text{honest} = 1$). Once $\mathcal{A}$ returns $(\tau^*, m^*, \text{lad}_0^*, \text{lad}_1^*)$, $\mathcal{B}$ sets s as the index of the first signer in lad_0 . $\mathcal{B}$ generates Σ in line 5 of $\text{Exp}_{\mathcal{A}, \mathcal{T}_{\text{etris}}}^{\text{Anon}}(\lambda)$ interacting with **OTag** as follows: for each index $i \neq s$, $\mathcal{A}$ obtains the tag calling **OTag**($\text{sk}_T^i, \tau^*, m^*, \text{abrt} = 0, \text{honest} = 1$); while for each action of type **Join**($\tau^*, m^*, \mathcal{R}, s, \sigma$) and **Sign**($\tau^*, m^*, s, \mathcal{R}$), $\mathcal{A}$ obtains the tag calling **OTag**($\text{sk}_T^s, \tau^*, m^*, \text{abrt} = 0, \text{honest} = 0$). $\mathcal{B}$ returns Σ to $\mathcal{A}$ and replies all queries in line 6 of $\text{Exp}_{\mathcal{A}, \mathcal{T}_{\text{etris}}}^{\text{Anon}}(\lambda)$ from $\mathcal{A}$ as follows. For **Sign** and **Join** queries on all positions $i \neq s$, $\mathcal{B}$ uses a tag obtained by **OTag**($\text{sk}_T^i, \tau, m, \text{abrt} = 0, \text{honest} = 1$). For sign and join operations on position s , $\mathcal{B}$ uses a tag obtained by **OTag**($\text{sk}_T^s, \tau, m, \text{abrt} = 0, \text{honest} = 1$). For each corruption query on $i \neq s$, $\mathcal{B}$ calls **OTag**($\perp, \perp, \perp, \text{abrt} = 1, \perp$) and returns sk_T^i to $\mathcal{A}$. Notice that by construction, an admissible $\mathcal{A}$ makes $\mathcal{B}$ ask exclusively admissible queries in the tag pseudo-randomness game w.r.t. pk_T^s . $\mathcal{B}$ perfectly simulates $\mathcal{H}_6$ if $\mathcal{C}$ always honestly generated tags (i.e., $b = 0$ in $\text{Exp}_{\mathcal{A}, \text{DAPT}}^{\text{TagPR}}(\lambda)$ w.r.t. pk_T^s Fig. 9) and perfectly simulates $\mathcal{H}_7$ if $\mathcal{C}$ always returns (when $\text{honest} = 0$) random tags (i.e., $b = 1$ in $\text{Exp}_{\mathcal{A}, \text{DAPT}}^{\text{TagPR}}(\lambda)$ w.r.t. pk_T^s Fig. 9). Thus, a successful distinguisher between the two hybrids can be used to break the tag pseudo-randomness of DAPT with the same success probability.

- $\mathcal{H}_8$: $\mathcal{B}$ modifies the tag for the second signer in lad_0 to be $e \leftarrow \mathcal{E}$ for the **Join** action done while computing the signatures in Σ associated to lad_0 . Moreover, we implicitly add another set of hybrids, one for each of the additional signers in lad_0 . For each of these hybrids, $\mathcal{B}$ sets $e \leftarrow \mathcal{E}$ for the current signer. All these hybrids are indistinguishable for the same argument of indistinguishability between $\mathcal{H}_7$ and $\mathcal{H}_6$.
- $\mathcal{H}_9$: When processing the ladder, for every join action on an index i , $\mathcal{B}$ uses $(\text{EP.AddT}_x, \text{EP.AddT}_w, \text{EP.Prv})$ instead of EP.PAdd . Additionally, instead of performing **AuxUpd**, each element of $\mathbf{A}_{\text{EP}}$ is replaced with a fresh encryption of the auxiliary values output by the **Prv** algorithm, except in the signer's ciphertexts where $\mathcal{B}$ keeps encrypting $\perp$. This hybrid is computationally indistinguishable from the previous one thanks to the addition privacy (Def. 14) of EP. Whenever $\mathcal{B}$ has to perform a **Join** action on a signature $\sigma = (k, \Pi_{\text{EP}}, \Pi_{\text{SH}}, \mathbf{A}_{\text{EP}}, \mathbf{A}_{\text{SH}}, C, \tilde{C}, E)$, $\mathcal{B}$ sets $x = (k, x_1, \dots, x_n, \tilde{C})$ with $x_j = (c_j, \text{pk}_T^j, \tau, m, x_{\text{trap}})$ where c_j is the j -th element of C . $\mathcal{B}$ sets $w = (\phi, \{w_{\text{trap}}\}^k, \tilde{r}_1, \dots, \tilde{r}_n)$ that is the witness that was used to compute Π_{EP} . $\mathcal{B}$ sets $w^* = w_{\text{trap}}$ and sends $(x, w, w^*, i, \tilde{r}_i', \Pi_{\text{EP}}, \text{AUX}_{\text{EP}}, R_{\text{Add}})$ where AUX_{EP} is known to $\mathcal{B}$ since it processed all the ladders by itself, while $\tilde{r}_i'$ and R_{Add} are uniformly sampled commitment randomness and transformation randomness. $\mathcal{B}$ now computes the next signature in the ladder $\sigma' = (k + 1, \Pi'_{\text{EP}}, \Pi'_{\text{SH}}, \mathbf{A}'_{\text{EP}}, \mathbf{A}'_{\text{SH}}, C', \tilde{C}', E')$ in the following way. It sets Π'_{EP} as the one received from $\mathcal{C}$, it updates the auxiliary values in $\mathbf{A}'_{\text{EP}}$ by encrypting the auxiliary values received by $\mathcal{C}$ in the non-signers' positions (and $\perp$ in the signers' ones), sets C' and $\tilde{C}'$ as the one contained in x' . It computes all the elements related to the shuffle proofs as in the previous hybrid, using commitment randomness $\tilde{r}_i'$ and transformation randomness R_{Add} . $\mathcal{B}$ perfectly simulates the view of $\mathcal{A}$ in both hybrids and thus $\mathcal{B}$ can use the output of a successful distinguisher to win in the addition privacy game with the same success probability.
- $\mathcal{H}_{10}$: When processing the ladder, $\mathcal{B}$ uses $(\text{EP.ExtT}_x, \text{EP.ExtT}_w, \text{EP.Prv})$ instead of EP.PExt . Additionally, instead of performing **AuxUpd**, each element of $\mathbf{A}_{\text{EP}}$ is replaced with a fresh encryption of the auxiliary values in output of **Prv** algorithm. This hybrid is computationally indistinguishable from the previous one thanks to the extension privacy of EP. Indistinguishability can be argued in a similar way to as was done in $\mathcal{H}_9$.
- $\mathcal{H}_{11}$: $\mathcal{B}$ uses $(\text{SH.AddT}_x, \text{SH.AddT}_w, \text{SH.Prv})$ instead of SH.PAdd . Additionally, instead of performing **AuxUpd**, each element of $\mathbf{A}_{\text{SH}}$ is replaced with a fresh encryption of the auxiliary values in output of **SH.Prv** algorithm. This hybrid is computationally indistinguishable from the previous one thanks to the addition privacy of SH. Indistinguishability can be argued in a similar way to as was done in $\mathcal{H}_9$.

- $\mathcal{H}_{12}$: $\mathcal{B}$ uses $(\text{SH.ExtT}_x, \text{SH.ExtT}_w, \text{SH.Priv})$ instead of SH.PExt . Additionally, instead of performing AuxUpd , each element of $\mathbf{A}_{\text{SH}}$ is replaced with a fresh encryption of the auxiliary values in output of Priv algorithm. This hybrid is computationally indistinguishable from the previous one thanks to the extension privacy of SH . Indistinguishability can be argued in a similar way to as was done in $\mathcal{H}_9$.
- $\mathcal{H}_{13}$: When processing the ladders, $\mathcal{B}$ encrypts $\perp$ in all the signers' ciphertexts of $\mathbf{A}_{\text{EP}}$ and $\mathbf{A}_{\text{SH}}$. Notice that, at this point, $\mathcal{B}$ is already not using any auxiliary value. Recall that an admissible $\mathcal{A}$ cannot corrupt any of the signers in lad_0 or lad_1 . This hybrid is computationally indistinguishable from the previous one thanks to the IND-CPA property of the encryption scheme.
- $\mathcal{H}_{14}$: When processing the ladder, $\mathcal{B}$ uses SH.Sim_1 instead of SH.Priv when producing Π_{SH} and associated auxiliary values. This hybrid is computationally indistinguishable from the previous one thanks to the EZK property (Def. 20) of SH . Let $\mathcal{C}$ be the challenger of the EZK game. Whenever $\mathcal{B}$ has to compute a shuffle proof to process the ladder, it simply forwards $(x_{\text{SH}}, w_{\text{SH}})$ to $\mathcal{C}$ and puts in the next signature in the ladder Π_{SH} received from $\mathcal{C}$, $\mathbf{A}_{\text{SH}}$ is computed by encrypting the *non*-signers' auxiliary values AUX received from $\mathcal{C}$ and encrypting $\perp$ in the signers' auxiliary values. The remaining parts of the signatures are computed as in the previous hybrid. $\mathcal{B}$ perfectly simulates the view of $\mathcal{A}$ in both hybrids and thus $\mathcal{B}$ can use the output of a successful distinguisher to win in the EZK game with the same success probability.
- $\mathcal{H}_{15}$: Let C_i be the list of commitments to tags contained in σ_i (i.e. the i -th signature in Σ). For all $i \in [\ell]$, where ℓ is the length of the ladder, $\mathcal{B}$ now creates C_i by committing to 0 in all positions. This hybrid is computationally indistinguishable from the previous one thanks to the computational hiding of Com_{ck} . Let $\mathcal{C}$ be the challenger of the computational hiding game. For every Sign or Join action in the ladder that adds an additional signer with index j , $\mathcal{B}$, instead of directly creating the j -th commitment within C_i , for $i \in \ell$, by directly committing to e_j , it queries $\mathcal{C}$ with $(e_j, 0)$. Then, $\mathcal{B}$ puts the received commitment in $C_i[j]$. Notice that at this point $\mathcal{B}$ does not need the opening of any of the commitments to compute any of the proofs. $\mathcal{B}$ perfectly simulates the view of $\mathcal{A}$ in both hybrids and thus $\mathcal{B}$ can use the output of a successful distinguisher to win in the computational hiding game with the same success probability.
- $\mathcal{H}_{16}$: Let C_i be the list of commitments to tags contained in σ_i . For all $i \in [\ell]$, $\mathcal{B}$ now creates the C_i by committing to the tags implicitly specified by lad_1 . Indistinguishability can be shown as done in $\mathcal{H}_{15}$.
- $\mathcal{H}_{17}$: Let $\mathbf{E}_i$ be the list of clear-text tags contained in σ_i . $\mathcal{B}$ now computes the clear-text tag to be (if needed) added to each $\mathbf{E}_i$ as the corresponding tags from lad_1 . This hybrid is indistinguishable from the previous one thanks to the tag pseudo-randomness of DAPT . Indistinguishability can be argued in the same way as $\mathcal{H}_7$ and $\mathcal{H}_8$.
- $\mathcal{H}_{18}$: $\mathcal{B}$ uses the DAPT secret keys related to lad_1 to compute the EP proofs. In particular, at the i -th step of the ladder with $i \in [\ell]$, $\mathcal{B}$ creates the commitments in $\tilde{C}$ are according to the signers of the i -th step in lad_1 . This hybrid is computationally indistinguishable from the previous one thanks to the EWI of EP (Def. 19). Let $\mathcal{C}$ be the challenger of the EWI game. Let us consider the first step y in which the signers are different in lad_0 and lad_1 . $\mathcal{B}$ sets $x = (k, x_1, \dots, x_n, \{\perp\}^k)$, with $x_j = (c_j, \text{pk}_{\text{T}}^j, \tau, m, x_{\text{trap}})$ where c_j is the j -th element of C computed as in the previous hybrid. $\mathcal{B}$ sets $w_0 = (\phi_0, \{w_{\text{trap}}\}^k)$, and $w_1 = (\phi_1, \{(e_j, \text{sk}_{\text{T}}^{\phi_1(j)}, r_{\phi_1(j)})\}_{j \in [k]})$ where ϕ_0 and ϕ_1 are the mappings induced by lad_0 and lad_1 respectively. $\mathcal{B}$ sends (x, w_0, w_1) to $\mathcal{C}$ and gets back $\Pi_{\text{EP}}, \text{AUX}_{\text{EP}}, \tilde{C}$. $\mathcal{B}$ then computes the signature $\sigma_y = (k, \Pi_{\text{EP}}, \Pi_{\text{SH}}, \mathbf{A}_{\text{EP}}, \mathbf{A}_{\text{SH}}, C, \tilde{C}, \mathbf{E})$ where Π_{EP} and $\tilde{C}$ are the ones received from $\mathcal{C}$, AUX_{EP} includes all the auxiliary values of non-signers' positions which can be encrypted by $\mathcal{B}$ and included in $\mathbf{A}_{\text{EP}}$, while the other positions are still encrypting $\perp$. The shuffle proofs can be computed as before as they are simulated and $\mathcal{B}$ does not need the openings of $\tilde{C}$. $\mathcal{B}$ perfectly simulates the view of $\mathcal{A}$ in both hybrids and thus $\mathcal{B}$ can use the output of a successful distinguisher to win in the extended witness indistinguishability game with the same success probability.
- $\mathcal{H}_{19}$: We start a sequence of hybrids that progressively removes the use of SH.Sim , w_{trap} and that switches back to using the extension and addition algorithms for Extend and Join queries/operations. To do that, we undo the changes from $\mathcal{H}_{14}$ to $\mathcal{H}_9$ and from $\mathcal{H}_6$ to $\mathcal{H}_1$. All of these hybrids are indistinguishable for the same reasons argued above. The final hybrid is exactly $\text{Exp}_{\mathcal{A}, \Sigma_{\text{ctris}}}^{\text{Anon}}(\lambda)$ with $b = 1$.

Lemma 7. *Let DAPT be defined as in Sec. 5. Let EP be defined as in Sec. 4 for relation R_{Dtr} with WAI (Def. 17) and FPWI (Def. 18). Then, the scheme in Fig. 11,12 is exculpable (Def. 7).*

Proof. Let us call $\mathcal{B}$ the reduction that runs $\mathcal{A}$ internally. The proof uses the following indistinguishable hybrids.

- $\mathcal{H}_0$: This is exactly the experiment of Fig. 7.
- $\mathcal{H}_1$: Instead of sampling a x_{trap} such that $x_{\text{trap}} \notin \mathcal{L}_{\text{trap}}$, $\mathcal{B}$ samples $(x_{\text{trap}}, w_{\text{trap}}) \in R_{\mathcal{L}_{\text{trap}}}$. This hybrid is computationally indistinguishable from the previous one thanks to the hardness of $\mathcal{L}_{\text{trap}}$.
- $\mathcal{H}_2$: This is equivalent to $\mathcal{H}_1$ except that $\mathcal{B}$ guesses an index i^* such that $\text{pk}_{i^*} \in \text{PK}$ at the end of exculpability experiment (see Fig. 7). The index i^* must exist, otherwise $\mathcal{A}$ would not win the exculpability game. From now on, every corruption query on i^* results in an abort of the experiment. Indeed, under the condition that i^* is correctly guessed, $\mathcal{A}$ cannot corrupt pk_{i^*} to win the exculpability game. Thus, the probability that $\mathcal{B}$ does not abort in this hybrid is at least $\frac{1}{q_{KG}+1}$, where q_{KG} is a polynomial bound on the number of key generation queries $\mathcal{A}$ can do.
- $\mathcal{H}_3$: $\mathcal{B}$ replies to $\text{Sign}(\tau, m, i^*, \mathcal{R})$ queries using w_{trap} instead of $\text{sk}_T^{i^*}$ as a witness to compute the proof Π_{EP} . This hybrid is indistinguishable from the previous thanks to the FPWI of EP (Def. 18). The proof is identical to the one of $\mathcal{H}_3$ of the proof of anonymity.
- $\mathcal{H}_4$: $\mathcal{B}$ replies to $\text{Join}(\tau, m, \mathcal{R}, i^*, \sigma)$ queries using w_{trap} instead of $\text{sk}_T^{i^*}$ as a witness to compute the add operation over the proof Π_{EP} contained in σ . This hybrid is indistinguishable from the previous thanks to the WAI of EP (Def. 17). The proof is identical to the one of $\mathcal{H}_4$ of the proof of anonymity (Lem. 6).
- $\mathcal{H}_5$: Let $\mathcal{C}$ be the challenger of the tag non-frameability game $\text{Exp}_{\mathcal{A}, \text{DAPT}}^{\text{TagNF}}(\lambda)$ (Fig. 10). Whenever asked to generate a key for index i^* , $\mathcal{B}$ forwards such KeyGen query to $\mathcal{C}$ and uses its reply to answer $\mathcal{A}$. Whenever $\mathcal{A}$ asks for a Sign or Join query over index i^* , $\mathcal{B}$ queries $\mathcal{C}$ to get the corresponding tag needed to answer the queries. Notice that at this point, $\mathcal{B}$ does not need $\text{sk}_T^{i^*}$ to answer any query involving i^* . Whenever $\mathcal{A}$ outputs $(\tau, t_0, t_1, \mathcal{R}_0, \mathcal{R}_1, m_0, m_1, \sigma_0, \sigma_1)$, $\mathcal{B}$ looks for a pair e_0, e_1 (respectively contained in tinfo_0 of σ_0 and tinfo_1 of σ_1) such that $\text{TagTrace}(e_0, e_1, m_0, m_1, \text{pk}_T^{i^*}) = 1$ and sends (e_0, e_1, m_0, m_1) to $\mathcal{C}$. Conditioned on correctly guessing i^* , $\mathcal{B}$ wins the tag non-frameability game with the same probability with which $\mathcal{A}$ wins the exculpability game. Notice that all the queries forwarded to $\mathcal{C}$ are admissible for a $\mathcal{B}$ playing the tag non-frameability game.

Lemma 8. *Let DAPT be defined as in Sec. 5. Let EP be defined as in Sec. 4 for relation R_{Dtr} with perfect Δ -extraction (Def. 16) where Δ is defined as Eq. 5. Let SH be defined as in Sec. 4 for relation R_{SH} . Then, the scheme in Fig. 11,12 is traceable (Def. 5).*

Proof. It follows from Lem. 4, Lem. 5, and Lem. 2.

7 Our DAPT

Let $\text{gk} = (p, \hat{\mathbb{G}}, \hat{\mathbb{H}}, \mathbb{T}, e, \hat{g}, \hat{h})$ where $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$. The topic space is $\{0, 1\}^*$, the message space is $\mathbb{Z}_p$, and the tag space is $\hat{\mathbb{G}}$. The algorithms work as follows:

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{gk})$: Sample random $x, y \leftarrow \mathbb{Z}_p$, compute $\text{pk} = (\hat{\text{pk}}_1 = \hat{g}x, \hat{\text{pk}}_2 = y\hat{g}, \check{\text{pk}}_2 = \check{h}y)$ and set $\text{sk} = (x, y)$.
- $\hat{e} \leftarrow \text{Tag}(\text{sk}, \tau, m)$: Let $\hat{\tau} \leftarrow H(\tau)$ output $\hat{e} = \hat{\tau}x + \hat{\text{pk}}_2mx$.
- $0/1 \leftarrow \text{TagTrace}(\hat{e}_0, \hat{e}_1, m_0, m_1, \text{pk})$: If $(\hat{e}_0 - \hat{e}_1) \cdot \check{h} = \hat{\text{pk}}_1 \cdot (m_0 - m_1)\check{\text{pk}}_2$ return 1, else return 0.
- $R_{\text{key}} = \{((\hat{\text{pk}}_1, \hat{\text{pk}}_2, \check{\text{pk}}_2), (x, y)) \mid \hat{\text{pk}}_1 = x\hat{g}, \hat{\text{pk}}_2 = \hat{g}y, \check{\text{pk}}_2 = \check{h}y\}$.

Theorem 3. *If the DDH and the Co-CDH assumptions hold relative to $\mathcal{G}$, then the scheme above is a DAPT in the ROM.*

Proof. We prove the above theorem via the following three lemmas.

Lemma 9. *The DAPT described above enjoys verifiability of keys (cfr, Def. 21).*

Proof. Given the poly-time relation R_{key} , one can check by inspection that the following three conditions hold.

1. For every $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{gk})$, it holds that $(\text{pk}, \text{sk}) \in R_{\text{key}}$.
2. Every $(\text{pk}, \text{sk}) \in R_{\text{key}}$ lies in the output space of $\text{KeyGen}(\text{gk})$.
3. For a given pk , there exists a unique sk such that $(\text{pk}, \text{sk}) \in R_{\text{key}}$. This is because the exponentiation is a unique map.

Lemma 10. *The DAPT described above enjoys tag traceability (cfr, Def. 22).*

Proof. First, let us consider two honestly generated tags $\hat{e}_0, \hat{e}_1$ w.r.t. the same key (pk, sk) , the same topic τ , and messages m_0, m_1 . We have that $\hat{e}_0 = \hat{\tau}x + \hat{\text{pk}}_1 m_0 x$ and $\hat{e}_1 = \hat{\tau}x + \hat{\text{pk}}_1 m_1 x$. We have that $(\hat{e}_0 - \hat{e}_1) \cdot \check{h} = \hat{\text{pk}}_1 x(m_0 - m_1) \cdot \check{h} = xy\hat{g}(m_0 - m_1) \cdot \check{h}$. On the other hand, $\hat{\text{pk}}_0 \cdot (m_0 - m_1)\check{\text{pk}}_1 = \hat{g}x \cdot (m_0 - m_1)\check{h}y$.

Second, let us consider an $\mathcal{A}$ winning the game of Def. 22 with non-negligible probability. $\mathcal{A}$ outputs $(\tau, m_0, m_1, \text{pk}_0, \text{pk}_1, \text{sk}_0, \text{sk}_1)$ such that $\text{sk}_0 = (x_0, y_0)$, $\text{sk}_1 = (x_1, y_1)$, $\text{pk}_0 = (x_0\hat{g}, y_0\hat{g}, \check{h}y_0)$ and $\text{pk}_1 = (x_1\hat{g}, y_1\hat{g}, \check{h}y_1)$. Additionally, given $\hat{e}_0 = \hat{\tau}x_0 + \hat{g}y_0 m_0 x$ and $\hat{e}_1 = \hat{\tau}x_1 + \hat{g}y_1 m_1 x$, where $\hat{\tau} = H(\tau)$, it holds that either $\text{TagTrace}(\hat{e}_0, \hat{e}_1, m_0, m_1, \text{pk}_0) = 1$ or $\text{TagTrace}(\hat{e}_0, \hat{e}_1, m_0, m_1, \text{pk}_1) = 1$. Let us assume, without loss of generality, that $\text{TagTrace}(\hat{e}_0, \hat{e}_1, m_0, m_1, \text{pk}_0) = 1$. This means that:

$$\begin{aligned} (\hat{\tau}x_0 + \hat{g}y_0 m_0 x_0 - \hat{\tau}x_1 - \hat{g}y_1 m_1 x_1) \cdot \check{h} &= \hat{g}x_0 y_0 (m_0 - m_1) \cdot \check{h} \\ \hat{\tau}(x_0 - x_1) \cdot \check{h} &= \hat{g}m_1(x_0 y_0 - x_1 y_1) \cdot \check{h} \\ u\hat{g}(x_0 - x_1) \cdot \check{h} &= \hat{g}m_1(x_0 y_0 - x_1 y_1) \cdot \check{h} \\ u &= m_1(x_0 y_0 - x_1 y_1)(x_0 - x_1)^{-1} \end{aligned}$$

Recall that we know that $x_0 \neq x_1 \wedge y_0 \neq y_1$. Basically, given an $\mathcal{A}$ winning with non-negligible probability, it is possible to find the discrete logarithm (DL) of $H(\tau)$ with the same probability. It is straightforward to build a reduction breaking the DL problem by making it program oracle queries with DL challenges.

Lemma 11. *If the EDDH assumption holds in $\mathbb{G}$ (Ass. 2), the DAPT described above enjoys pseudo-randomness (Def. 23) in the random oracle model.*

Proof. To prove this lemma we will transition, through a series of indistinguishable hybrids, from the experiment run with $b = 0$ to the experiment run with $b = 1$.

$\mathcal{H}_0$: This is exactly the pseudo-randomness experiment of Fig. 9 with $b = 0$.

$\mathcal{H}_1$: This is equivalent to $\mathcal{H}_0$ except that, when $\mathcal{B}$ is queried to produce a tag on the pair (τ, m) , instead of computing it as $\hat{e} = H(\tau)x + \hat{\text{pk}}_1 m y$, the reduction outputs $\hat{e} = \hat{r}_\tau + \hat{\text{pk}}_1 m y$ with $\hat{r}_\tau \leftarrow \mathbb{G}$. The reduction uses the same $\hat{r}_\tau$ in case the same τ is queried again.

$\mathcal{H}_2$: This is equivalent to $\mathcal{H}_1$ except that, when $\mathcal{B}$ is queried to produce a tag on the pair (τ, m) , the reduction samples a random $\hat{e} \leftarrow \mathbb{G}$. This is exactly the pseudo-randomness experiment of Fig. 9 with $b = 1$.

We now show that all the consecutive hybrids are indistinguishable:

- $\mathcal{H}_0$ and $\mathcal{H}_1$ are indistinguishable under the Type 2 EDDH assumption (which is implied by the EDDH assumption). Indeed, we can use a distinguisher $\mathcal{D}$ that, on input the view of $\mathcal{A}$, distinguishes between $\mathcal{H}_0$ and $\mathcal{H}_1$ with probability ϵ , to build an adversary $\mathcal{B}$ that breaks the EDDH assumption with the same probability. $\mathcal{B}$ creates $\text{pk} = (\text{pk}_1, \text{pk}_2, \check{\text{pk}}_2)$ as follows. It sets $\hat{\text{pk}}_1 = \hat{a}$, with $\hat{a}$ that it gets from the EDDH game, while it samples $y \leftarrow \mathbb{Z}_p$ to create $\hat{\text{pk}}_2 = y\hat{g}$ and $\check{\text{pk}}_2 = \check{h}y$. Whenever $\mathcal{A}$ calls OTag with input a not previously queried (τ_i, m_i) and $\text{honest} = 1$, $\mathcal{B}$ calls the ODDH with $\text{honest} = 1$ and gets $(\hat{t}_i, \hat{z}_i)$. Then, it programs the random oracle on input τ_i to give $\hat{t}_i$ as output. Finally, it replies to the query computing $\hat{e}_i = \hat{z}_i + \hat{\text{pk}}_1 m_i y$. Similarly, whenever $\mathcal{A}$ calls OTag with input a not previously queried (τ_i, m_i) and $\text{honest} = 0$, $\mathcal{B}$ calls the ODDH oracle of the EDDH game and gets $(\hat{t}_i, \hat{z}_i)$. Then, it programs the random oracle on input τ_i to give $\hat{t}_i$ as output. Finally, it replies to the query computing $\hat{e}_i = \hat{z}_i + \hat{\text{pk}}_1 m_i y$. $\mathcal{B}$ outputs whatever $\mathcal{D}$ outputs. Notice that $\mathcal{B}$ perfectly simulates $\mathcal{H}_0$ whenever the type 2 EDDH experiment is run with $b = 0$, and it perfectly simulates $\mathcal{H}_1$ otherwise.

- $\mathcal{H}_1$ and $\mathcal{H}_2$ are perfectly indistinguishable if $\mathcal{A}$ never queries (τ, m_1) and (τ, m_2) with $m_1 \neq m_2$. Indeed, in this case, $\hat{r}_\tau$ acts as a random mask that is only used once and perfectly hides the group element that is added to it.

Lemma 12. *If the EDDH (Assumption 2) and the Co-CDH (Assumption 5) assumptions hold relative to $\mathcal{G}$, the DAPT described above enjoys tag non-frameability (Def. 24) in the random oracle model.*

Proof. To prove this lemma we will transition through a series of indistinguishable hybrids. Then, in the final hybrid we will use the output of the adversary $\mathcal{A}$ of the tag non-frameability experiment to break the Co-CDH assumption.

$\mathcal{H}_0$: This is exactly the tag non-frameability experiment of Fig. 10.

$\mathcal{H}_1$: This is equivalent to $\mathcal{H}_0$ except that, when queried to produce a tag on the pair (τ, m) , instead of computing it as $\hat{e} = H(\tau)x + \mathbf{pk}_1my$, the reduction outputs $\hat{e} = \hat{r}_\tau + \mathbf{pk}_1my$ with $\hat{r}_\tau \leftarrow \mathbb{G}$. The reduction uses the same $\hat{r}_\tau$ in case the same τ is queried again.

$\mathcal{H}_2$: This is equivalent to $\mathcal{H}_1$ except that the reduction computes the tags simply as random elements of $\mathbb{G}$. The reduction keeps track of these values and returns the same value if the same pair (τ, m) is queried again.

The indistinguishability of all the hybrids can be argued as it was shown while proving the tag indistinguishability property. The key point now is that in $\mathcal{H}_2$, $\mathcal{B}$ can reply to the tag generation queries without knowing any part of the secret key. Therefore, $\mathcal{B}$ will set the public key to give to $\mathcal{A}$ as the Co-CDH challenge. Then whenever $\mathcal{A}$ outputs $(\hat{e}_0, \hat{e}_1, m_0, m_1)$ s.t. $(\hat{e}_0 - \hat{e}_1) \cdot \hat{h} = \mathbf{pk}_1 \cdot (m_0 - m_1)\mathbf{pk}_2$, $\mathcal{B}$ will output $\frac{(\hat{e}_0 - \hat{e}_1)}{m_0 - m_1}$. It follows that $\mathcal{B}$ wins the Co-CDH game with the same probability with which $\mathcal{A}$ wins the tag non-frameability game.

Remark 1. As shown in the above security proof, given two tags $\hat{e}_0$ and $\hat{e}_1$ w.r.t. the same topic τ and $m_0 \neq m_1$ it is possible to compute $\hat{t} = \frac{(\hat{e}_0 - \hat{e}_1)}{m_0 - m_1}$. It is also possible to compute $\hat{t}x = \hat{e}_0 - \hat{t}m_0$. Given this information, it is easy to issue tags for the topic τ and any message m by computing $\hat{t}m + \hat{t}x$.

8 Our Extendable Shuffle Argument

Our extendable shuffle argument (Sec. 1.2) is an EP with EZK for relation R_{SH} :

$$R_{\text{SH}} = \{(\text{ck}, x = (k, \hat{\mathbf{z}}_1, \dots, \hat{\mathbf{z}}_n, \tilde{\mathbf{m}}_1, \dots, \tilde{\mathbf{m}}_n, \hat{e}_1, \dots, \hat{e}_k), w = (\phi, r_1, \dots, r_k, r_{\tilde{m}_1}, \dots, r_{\tilde{m}_n})) \mid \forall i \in [n] \ \tilde{\mathbf{m}}_i \leftarrow \text{Com}_{\text{ck}}(\text{bit}_i; r_{\tilde{m}_i}) \wedge \text{bit}_i \in \{0, 1\} \wedge \sum_{i=1}^n \text{bit}_i = k \wedge \forall_{i=1}^k (\text{bit}_{\phi(i)} = 1 \wedge (\hat{\mathbf{z}}_{\phi(i)} \leftarrow \text{Com}_{\text{ck}}(\hat{e}_i; r_i))) \wedge \phi \text{ is an injective map } [k] \rightarrow [n]\}.$$

As commitment scheme, we use ElGamal with commitment keys $\hat{\text{ck}}$ and $\check{\text{ck}}$:

- To commit to a group element $\hat{E}$ using randomness $r_{\hat{\mathbf{z}}}$, compute $\hat{\mathbf{z}} = (\hat{\mathbf{z}}^1, \hat{\mathbf{z}}^2) = (r_{\hat{\mathbf{z}}}\hat{g}, \hat{E} + r_{\hat{\mathbf{z}}}\hat{\text{ck}})$.
- To commit to a bit with randomness $r_{\tilde{m}}$, compute $\tilde{\mathbf{m}} = (\tilde{\mathbf{m}}^1, \tilde{\mathbf{m}}^2) = (r_{\tilde{m}}\hat{h}, \text{bit}\hat{h} + r_{\tilde{m}}\check{\text{ck}})$.

We construct Π_{SH} for relation R_{SH} by defining a set of equations and proving their satisfiability with the GS proof system. Let ψ be a committed variable that is 1 on a regular crs_{SH} and can be equivocated to 0 with the simulation trapdoor; such a variable is implicitly available in the crs of GS [37]. The variable ψ is only needed to achieve EZK, so it can be ignored in the following overview in which we describe all the equations we prove and why they give a shuffle argument.

In order to prove the correctness of the switch variables $\{\text{bit}_i : i \in [n]\}$, i.e., that $\forall i \in [n] : \text{bit}_i \in \{0, 1\} \wedge \sum_{i=1}^n \text{bit}_i = k$, we define the equations $\mathcal{B}_i : \text{bit}_i(1 - \text{bit}_i) = 0$ and $\mathcal{K} : \sum_{i=1}^n \text{bit}_i = k\psi$.

The main challenge is to devise a set of equations whose satisfiability, proven with GS, implies the statement in the third line of R_{SH} . Towards this goal, we first introduce new auxiliary variables $\hat{E}_i, f_i, r_{\hat{\mathbf{z}}_i}$ and define the following equations (with public constants $\hat{\text{ck}}, \check{\text{ck}}, \hat{g}, \hat{h}, \hat{\mathbf{z}}_i, \tilde{\mathbf{m}}_i$):

$$(\forall i) \ \mathcal{D}_{i,1}^1 : f_i \hat{\mathbf{z}}_i^1 = r_{\hat{\mathbf{z}}_i} \hat{g}, \ \mathcal{D}_{i,2}^1 : f_i \hat{\mathbf{z}}_i^2 = f_i \hat{E}_i + r_{\hat{\mathbf{z}}_i} \hat{\text{ck}}, \ \mathcal{F}_i : f_i = \text{bit}_i \psi$$

$$(\forall i) \mathcal{D}_{i,1}^2 : \psi \tilde{\mathbf{m}}_i^1 = r_{\tilde{m}_i} \tilde{h}, \quad \mathcal{D}_{i,2}^2 : \psi \tilde{\mathbf{m}}_i^2 = \text{bit}_i \tilde{h} + r_{\tilde{m}_i} \tilde{c}k.$$

The equations in the first line guarantee that the prover can open k commitments to tags, while the equations in the second line guarantee that the i -th switch variable bit_i is correctly committed in $\tilde{\mathbf{m}}_i$.

In order to prove the shuffle w.r.t. the k public values $(\hat{e}_1, \dots, \hat{e}_k)$, we extend the techniques of Groth and Lu [37] as follows. We introduce new variables $\{\tilde{a}_i, \tilde{a}'_i, \tilde{b}_i, \tilde{b}'_i, \tilde{c}_i : i \in [n]\}$ and define the following equations (with public constants $\hat{g}, \hat{h}, \hat{h}_i, \hat{\delta}_i$), where all $\hat{h}_i, \hat{\delta}_i$ with $i \in [n]$ are part of crs_{SH} and are generated as stated in Ass. 6.

$$\begin{aligned} \mathcal{Q}_i^1 : \tilde{a}_i &= \text{bit}_i \tilde{a}'_i \quad (\forall i) \quad \mathcal{Q}_i^2 : \tilde{b}_i = \text{bit}_i \tilde{b}'_i \\ \mathcal{S}^1 : \sum_{i=1}^n \tilde{a}_i &= \sum_{i=1}^k \psi \tilde{h}_i \quad \mathcal{S}^2 : \sum_{i=1}^n \tilde{b}_i = \sum_{i=1}^k \psi \tilde{\delta}_i \\ (\forall i) \mathcal{V}_i^1 : \hat{g} \cdot \tilde{a}_i &= \tilde{c}_i \cdot \hat{h} \quad (\forall i) \mathcal{V}_i^2 : \hat{g} \cdot \tilde{b}_i = \tilde{c}_i \cdot \tilde{a}_i \end{aligned}$$

The above equations formulate a subset permutation pairing problem over the variables $\{\tilde{a}_i, \tilde{b}_i\}$ as stated in Assumption 6. In particular, for all $i \in [n]$ the equations $\mathcal{Q}_i^1, \mathcal{Q}_i^2$, together with equations $\mathcal{B}_i$ and $\mathcal{K}$ guarantee that at least k of the variables $\tilde{a}_i$ and $\tilde{b}_i$ are set to 0. Additionally, if the equations $\mathcal{S}^1, \mathcal{S}^2, \mathcal{V}^1, \mathcal{V}^2$ are satisfied then, thanks to the subset permutation assumption, there exists except with negligible probability a set of indices $\mathcal{J} = \{\alpha_1, \dots, \alpha_k\}$ with $1 \leq \alpha_i \leq n$ s.t. $\{(\tilde{a}_{\alpha_i}, \tilde{b}_{\alpha_i})\}_{i \in [k]}$ is a permutation of $\{(\tilde{h}_i, \tilde{\delta}_i)\}_{i \in [k]}$.

Namely, the switch variables bit_i modify the equations in such a way that setting $\text{bit}_i = 1$ implies that $i \in \mathcal{J}$. Finally, we define the following equations with variables $\hat{o}, \hat{o}', \psi, \hat{E}_i, \tilde{a}_i, \tilde{b}_i$ and public constants $\hat{e}_i, \hat{h}, \hat{\delta}_i, \hat{h}_i$.

$$\begin{aligned} \mathcal{E}^1 : \hat{o} \cdot \hat{h} + \sum_{i=1}^n \hat{E}_i \cdot \tilde{a}_i &= \sum_{i=1}^k \hat{e}_i \cdot \hat{h}_i, \quad \mathcal{E}^2 : \hat{o}' \cdot \hat{h} + \sum_{i=1}^n \hat{E}_i \cdot \tilde{b}_i = \sum_{i=1}^k \hat{e}_i \cdot \hat{\delta}_i \\ \mathcal{P}^2 : \psi \hat{o} &= \hat{o} \quad \mathcal{P}^3 : \psi \hat{o}' = \hat{o} \end{aligned}$$

As for ψ , the introduction of the variables $\hat{o}$ and $\hat{o}'$ is only useful to the simulator. Let us look at the equations in the first line, we have that $\sum_{i=1}^n \hat{E}_i \cdot \tilde{a}_i = \sum_{i=1}^k \hat{e}_i \cdot \hat{h}_i$ and $\sum_{i=1}^n \hat{E}_i \cdot \tilde{b}_i = \sum_{i=1}^k \hat{e}_i \cdot \hat{\delta}_i$. Given that $\{(\tilde{a}_{\alpha_i}, \tilde{b}_{\alpha_i})\}_{i \in [k]}$ is a permutation of $\{(\tilde{h}_i, \tilde{\delta}_i)\}_{i \in [k]}$, there must exist a permutation π such that $\sum_{i=1}^k (\hat{E}_{\alpha_{\pi(i)}} - \hat{e}_i) \cdot \hat{h}_i = \hat{o}$ and $\sum_{i=1}^k (\hat{E}_{\alpha_{\pi(i)}} - \hat{e}_i) \cdot \hat{\delta}_i = \hat{o}$. This constitutes a subset simultaneous pairing problem (see Assumption 7), and assuming its computational hardness, we can infer that $\hat{E}_{\alpha_{\pi(i)}} = \hat{e}_i$ for all $i \in [k]$. To prove all the above equations the prover assigns the value to the variables in the following way:

- For all $i \in [n]$ set $r_{\tilde{m}_i}$ as the value given in the witness.
- Set $\mathcal{I} \leftarrow \emptyset$. For all $j \in [k]$ let $\phi(j) = i$, set $\tilde{a}_i = \tilde{a}'_i = \tilde{h}_j, \tilde{b}_i = \tilde{b}'_i = \tilde{\delta}_j, \tilde{c}_i = \hat{g}_j, \hat{E}_i = \hat{e}_j, \text{bit}_i = 1, f_1 = 1, r_{\tilde{z}_i} = r_j, \mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$.
- For all $i \in [n] \setminus \mathcal{I}$, set $\tilde{a}_i = \tilde{a}'_i = 0, \tilde{b}_i = \tilde{b}'_i = 0, \tilde{c}_i = \hat{o}, \hat{E}_i = \hat{o}, \text{bit}_i = 0, f_i = 0, r_{\tilde{z}_i} = 0$.

The simulation trapdoor consists of the trapdoor of the GS proof system, that allows equivocating ψ to 0, and the discrete logs $\{p_i\}_{i \in [n]}$ of the elements $\{\hat{g}_i\}_{i \in [n]}$. Indeed, to simulate a proof without knowledge of the witness it suffices to set variables $\hat{o} = \sum_{i=1}^k p_i \hat{e}_i, \hat{o}' = \sum_{i=1}^k 2p_i \hat{e}_i, (\forall i) \tilde{a}_i = \tilde{a}'_i = 0, \tilde{b}_i = \tilde{b}'_i = 0, \tilde{c}_i = \hat{o}, \hat{E}_i = \hat{o}, \text{bit}_i = 0, f_i = 0, r_{\tilde{z}_i} = 0, r_{\tilde{m}_i} = 0$. The extended zero knowledge property follows from the witness indistinguishability of GS proofs and from the fact that the variables corresponding to the inactive positions are assigned in the same way both for real and simulated proofs.

Extend and add operations can be implemented with the same techniques of [8] since, as for their ENIWI, they only involve adding/removing the contribution of certain variables within a set of equations (i.e., $\mathcal{K}, \mathcal{S}^1, \mathcal{S}^2, \mathcal{E}^1, \mathcal{E}^2$). Whenever an extend/add operation is performed, the lists of commitments $\{\hat{\mathbf{z}}_i\}_{i \in [n]}$ to group elements and commitments $\{\tilde{\mathbf{m}}_i\}_{i \in [n]}$ to switch variables are also re-randomized (ElGamal is re-randomizable). Applying this re-randomization and updating the GS proofs consequently is straightforward given the malleability of the ElGamal encryption and of GS proofs. We defer to App. B for a detailed description of the construction and its security proofs.

References

1. Abe, M., Ohkubo, M., Suzuki, K.: 1-out-of-n signatures from a variety of keys. In: Zheng, Y. (ed.) ASIACRYPT 2002. LNCS, vol. 2501, pp. 415–432. Springer, Heidelberg (Dec 2002). https://doi.org/10.1007/3-540-36178-2_26
2. Aguilar Melchor, C., Cayrel, P.L., Gaborit, P.: A new efficient threshold ring signature scheme based on coding theory. In: Buchmann, J., Ding, J. (eds.) Post-quantum cryptography, second international workshop, PQCRYPTO 2008. pp. 1–16. Springer, Heidelberg (Oct 2008). https://doi.org/10.1007/978-3-540-88403-3_1
3. Aranha, D.F., Baum, C., Gjøsteen, K., Silde, T., Tunge, T.: Lattice-based proof of shuffle and applications to electronic voting. In: Paterson, K.G. (ed.) CT-RSA 2021. LNCS, vol. 12704, pp. 227–251. Springer, Heidelberg (May 2021). https://doi.org/10.1007/978-3-030-75539-3_10
4. Aranha, D.F., Hall-Andersen, M., Nitulescu, A., Pagnin, E., Yakoubov, S.: Count me in! extendability for threshold ring signatures. Cryptology ePrint Archive, Paper 2021/1240 (2021), <https://eprint.iacr.org/2021/1240>, <https://eprint.iacr.org/2021/1240>
5. Aranha, D.F., Hall-Andersen, M., Nitulescu, A., Pagnin, E., Yakoubov, S.: Count me in! Extendability for threshold ring signatures. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) PKC 2022, Part II. LNCS, vol. 13178, pp. 379–406. Springer, Heidelberg (Mar 2022). https://doi.org/10.1007/978-3-030-97131-1_13
6. Attema, T., Cramer, R., Rambaud, M.: Compressed Σ -protocols for bilinear group arithmetic circuits and application to logarithmic transparent threshold signatures. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part IV. LNCS, vol. 13093, pp. 526–556. Springer, Heidelberg (Dec 2021). https://doi.org/10.1007/978-3-030-92068-5_18
7. Au, M.H., Liu, J.K., Susilo, W., Yuen, T.H.: Secure id-based linkable and revocable-iff-linked ring signature with constant-size construction. Theor. Comput. Sci. **469**, 1–14 (2013)
8. Avitabile, G., Botta, V., Fiore, D.: Extendable threshold ring signatures with enhanced anonymity. In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023, Part I. LNCS, vol. 13940, pp. 281–311. Springer, Heidelberg (May 2023). https://doi.org/10.1007/978-3-031-31368-4_11
9. Avitabile, G., Botta, V., Friolo, D., Visconti, I.: Efficient proofs of knowledge for threshold relations. In: Atluri, V., Di Pietro, R., Jensen, C.D., Meng, W. (eds.) ESORICS 2022, Part III. LNCS, vol. 13556, pp. 42–62. Springer, Heidelberg (Sep 2022). https://doi.org/10.1007/978-3-031-17143-7_3
10. Backes, M., Döttling, N., Hanzlik, L., Kluczniak, K., Schneider, J.: Ring signatures: Logarithmic-size, no setup - from standard assumptions. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part III. LNCS, vol. 11478, pp. 281–311. Springer, Heidelberg (May 2019). https://doi.org/10.1007/978-3-030-17659-4_10
11. Belenkiy, M., Camenisch, J., Chase, M., Kohlweiss, M., Lysyanskaya, A., Shacham, H.: Randomizable proofs and delegatable anonymous credentials. In: Halevi, S. (ed.) CRYPTO 2009. LNCS, vol. 5677, pp. 108–125. Springer, Heidelberg (Aug 2009). https://doi.org/10.1007/978-3-642-03356-8_7
12. Bellare, M., Micciancio, D., Warinschi, B.: Foundations of group signatures: Formal definitions, simplified requirements, and a construction based on general assumptions. In: Biham, E. (ed.) EUROCRYPT 2003. LNCS, vol. 2656, pp. 614–629. Springer, Heidelberg (May 2003). https://doi.org/10.1007/3-540-39200-9_38
13. Bettaieb, S., Schrek, J.: Improved lattice-based threshold ring signature scheme. In: Gaborit, P. (ed.) Post-Quantum Cryptography - 5th International Workshop, PQCrypto 2013. pp. 34–51. Springer, Heidelberg (Jun 2013). https://doi.org/10.1007/978-3-642-38616-9_3
14. Beullens, W., Katsumata, S., Pintore, F.: Calamari and Falafi: Logarithmic (linkable) ring signatures from isogenies and lattices. In: Moriai, S., Wang, H. (eds.) ASIACRYPT 2020, Part II. LNCS, vol. 12492, pp. 464–492. Springer, Heidelberg (Dec 2020). https://doi.org/10.1007/978-3-030-64834-3_16
15. Boneh, D., Gentry, C., Lynn, B., Shacham, H.: Aggregate and verifiably encrypted signatures from bilinear maps. In: Biham, E. (ed.) EUROCRYPT 2003. LNCS, vol. 2656, pp. 416–432. Springer, Heidelberg (May 2003). https://doi.org/10.1007/3-540-39200-9_26
16. Bootle, J., Cerulli, A., Chaidos, P., Ghadafi, E., Groth, J.: Foundations of fully dynamic group signatures. In: Manulis, M., Sadeghi, A.R., Schneider, S. (eds.) ACNS 16. LNCS, vol. 9696, pp. 117–136. Springer, Heidelberg (Jun 2016). https://doi.org/10.1007/978-3-319-39555-5_7
17. Bootle, J., Cerulli, A., Chaidos, P., Ghadafi, E., Groth, J., Petit, C.: Short accountable ring signatures based on DDH. In: Pernul, G., Ryan, P.Y.A., Weippl, E.R. (eds.) ESORICS 2015, Part I. LNCS, vol. 9326, pp. 243–265. Springer, Heidelberg (Sep 2015). https://doi.org/10.1007/978-3-319-24174-6_13
18. Branco, P., Mateus, P.: A traceable ring signature scheme based on coding theory. In: Ding, J., Steinwandt, R. (eds.) Post-Quantum Cryptography. pp. 387–403. Springer International Publishing, Cham (2019)

19. Branco, P., Mateus, P.: A traceable ring signature scheme based on coding theory. In: Ding, J., Steinwandt, R. (eds.) *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019*. pp. 387–403. Springer, Heidelberg (2019). https://doi.org/10.1007/978-3-030-25510-7_21
20. Bresson, E., Stern, J., Szydło, M.: Threshold ring signatures and applications to ad-hoc groups. In: Yung, M. (ed.) *CRYPTO 2002*. LNCS, vol. 2442, pp. 465–480. Springer, Heidelberg (Aug 2002). https://doi.org/10.1007/3-540-45708-9_30
21. Catalano, D., Fuchsbaauer, G., Soleimanian, A.: Double-authentication-preventing signatures in the standard model. In: Galdi, C., Kolesnikov, V. (eds.) *SCN 20*. LNCS, vol. 12238, pp. 338–358. Springer, Heidelberg (Sep 2020). https://doi.org/10.1007/978-3-030-57990-6_17
22. Chakraborty, S., Hofheinz, D., Langrehr, R., Nielsen, J.B., Striecks, C., Venturi, D.: Malleable SNARKs and their applications. *Cryptology ePrint Archive*, Paper 2025/311 (2025), <https://eprint.iacr.org/2025/311>
23. Chase, M., Kohlweiss, M., Lysyanskaya, A., Meiklejohn, S.: Malleable proof systems and applications. In: Pointcheval, D., Johansson, T. (eds.) *EUROCRYPT 2012*. LNCS, vol. 7237, pp. 281–300. Springer, Heidelberg (Apr 2012). https://doi.org/10.1007/978-3-642-29011-4_18
24. Chase, M., Kohlweiss, M., Lysyanskaya, A., Meiklejohn, S.: Succinct malleable NIZKs and an application to compact shuffles. In: Sahai, A. (ed.) *TCC 2013*. LNCS, vol. 7785, pp. 100–119. Springer, Heidelberg (Mar 2013). https://doi.org/10.1007/978-3-642-36594-2_6
25. Chaum, D., van Heyst, E.: Group signatures. In: Davies, D.W. (ed.) *EUROCRYPT’91*. LNCS, vol. 547, pp. 257–265. Springer, Heidelberg (Apr 1991). https://doi.org/10.1007/3-540-46416-6_22
26. Derler, D., Ramacher, S., Slamanig, D.: Generic double-authentication preventing signatures and a post-quantum instantiation. In: Baek, J., Susilo, W., Kim, J. (eds.) *ProvSec 2018*. LNCS, vol. 11192, pp. 258–276. Springer, Heidelberg (Oct 2018). https://doi.org/10.1007/978-3-030-01446-9_15
27. Dodis, Y., Kiayias, A., Nicolosi, A., Shoup, V.: Anonymous identification in ad hoc groups. In: Cachin, C., Camenisch, J. (eds.) *EUROCRYPT 2004*. LNCS, vol. 3027, pp. 609–626. Springer, Heidelberg (May 2004). https://doi.org/10.1007/978-3-540-24676-3_36
28. Escala, A., Groth, J.: Fine-tuning Groth-Sahai proofs. In: Krawczyk, H. (ed.) *PKC 2014*. LNCS, vol. 8383, pp. 630–649. Springer, Heidelberg (Mar 2014). https://doi.org/10.1007/978-3-642-54631-0_36
29. Fujisaki, E.: Sub-linear size traceable ring signatures without random oracles. In: Kiayias, A. (ed.) *CT-RSA 2011*. LNCS, vol. 6558, pp. 393–415. Springer, Heidelberg (Feb 2011). https://doi.org/10.1007/978-3-642-19074-2_25
30. Fujisaki, E., Suzuki, K.: Traceable ring signature. In: Okamoto, T., Wang, X. (eds.) *PKC 2007*. LNCS, vol. 4450, pp. 181–200. Springer, Heidelberg (Apr 2007). https://doi.org/10.1007/978-3-540-71677-8_13
31. Fujisaki, E., Suzuki, K.: Traceable ring signature. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.* **91-A**(1), 83–93 (2008). <https://doi.org/10.1093/IETFEC/E91-A.1.83>, <https://doi.org/10.1093/ietfec/e91-a.1.83>
32. Ghadafi, E.: Efficient distributed tag-based encryption and its application to group signatures with efficient distributed traceability. In: Aranha, D.F., Menezes, A. (eds.) *LATINCRYPT 2014*. LNCS, vol. 8895, pp. 327–347. Springer, Heidelberg (Sep 2015). https://doi.org/10.1007/978-3-319-16295-9_18
33. Ghadafi, E.: Partially structure-preserving signatures: Lower bounds, constructions and more. In: Sako, K., Tappenhauer, N.O. (eds.) *ACNS 21, Part I*. LNCS, vol. 12726, pp. 284–312. Springer, Heidelberg (Jun 2021). https://doi.org/10.1007/978-3-030-78372-3_11
34. Ghadafi, E., Smart, N.P., Warinschi, B.: Groth-Sahai proofs revisited. In: Nguyen, P.Q., Pointcheval, D. (eds.) *PKC 2010*. LNCS, vol. 6056, pp. 177–192. Springer, Heidelberg (May 2010). https://doi.org/10.1007/978-3-642-13013-7_11
35. Goel, A., Green, M., Hall-Andersen, M., Kaptchuk, G.: Stacking sigmas: A framework to compose Σ -protocols for disjunctions. In: Dunkelman, O., Dziembowski, S. (eds.) *EUROCRYPT 2022, Part II*. LNCS, vol. 13276, pp. 458–487. Springer, Heidelberg (May / Jun 2022). https://doi.org/10.1007/978-3-031-07085-3_16
36. González, A.: Shorter ring signatures from standard assumptions. In: Lin, D., Sako, K. (eds.) *PKC 2019, Part I*. LNCS, vol. 11442, pp. 99–126. Springer, Heidelberg (Apr 2019). https://doi.org/10.1007/978-3-030-17253-4_4
37. Groth, J., Lu, S.: A non-interactive shuffle with pairing based verifiability. In: Kurosawa, K. (ed.) *ASIACRYPT 2007*. LNCS, vol. 4833, pp. 51–67. Springer, Heidelberg (Dec 2007). https://doi.org/10.1007/978-3-540-76900-2_4
38. Groth, J., Sahai, A.: Efficient non-interactive proof systems for bilinear groups. In: Smart, N.P. (ed.) *EUROCRYPT 2008*. LNCS, vol. 4965, pp. 415–432. Springer, Heidelberg (Apr 2008). https://doi.org/10.1007/978-3-540-78967-3_24
39. Haque, A., Scafuro, A.: Threshold ring signatures: New definitions and post-quantum security. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) *PKC 2020, Part II*. LNCS, vol. 12111, pp. 423–452. Springer, Heidelberg (May 2020). https://doi.org/10.1007/978-3-030-45388-6_15

40. Liu, J.K., Wei, V.K., Wong, D.S.: Linkable spontaneous anonymous group signature for ad hoc groups (extended abstract). In: Wang, H., Pieprzyk, J., Varadharajan, V. (eds.) ACISP 04. LNCS, vol. 3108, pp. 325–335. Springer, Heidelberg (Jul 2004). https://doi.org/10.1007/978-3-540-27800-9_28
41. Malavolta, G., Schröder, D.: Efficient ring signatures in the standard model. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017, Part II. LNCS, vol. 10625, pp. 128–157. Springer, Heidelberg (Dec 2017). https://doi.org/10.1007/978-3-319-70697-9_5
42. Munch-Hansen, A., Orlandi, C., Yakubov, S.: Stronger notions and a more efficient construction of threshold ring signatures. In: Longa, P., Ràfols, C. (eds.) LATINCRYPT 2021. LNCS, vol. 12912, pp. 363–381. Springer, Heidelberg (Oct 2021). https://doi.org/10.1007/978-3-030-88238-9_18
43. Naor, M.: Deniable ring authentication. In: Yung, M. (ed.) CRYPTO 2002. LNCS, vol. 2442, pp. 481–498. Springer, Heidelberg (Aug 2002). https://doi.org/10.1007/3-540-45708-9_31
44. Neff, C.A.: A verifiable secret shuffle and its application to e-voting. In: Reiter, M.K., Samarati, P. (eds.) ACM CCS 2001. pp. 116–125. ACM Press (Nov 2001). <https://doi.org/10.1145/501983.502000>
45. Petzoldt, A., Bulygin, S., Buchmann, J.: A multivariate based threshold ring signature scheme. *Appl. Algebra Eng. Commun. Comput.* **24**(3-4), 255–275 (2013)
46. Rivest, R.L., Shamir, A., Tauman, Y.: How to leak a secret. In: Boyd, C. (ed.) ASIACRYPT 2001. LNCS, vol. 2248, pp. 552–565. Springer, Heidelberg (Dec 2001). https://doi.org/10.1007/3-540-45682-1_32
47. Russo, A., Anta, A.F., Vasco, M.I.G., Romano, S.P.: Chirotonia: A Scalable and Secure e-Voting Framework based on Blockchains and Linkable Ring Signatures. In: 2021 IEEE International Conference on Blockchain (Blockchain). pp. 417–424 (2021)
48. Scafuro, A., Zhang, B.: One-time traceable ring signatures. In: Bertino, E., Shulman, H., Waidner, M. (eds.) ESORICS 2021, Part II. LNCS, vol. 12973, pp. 481–500. Springer, Heidelberg (Oct 2021). https://doi.org/10.1007/978-3-030-88428-4_24
49. Thyagarajan, S.A.K., Malavolta, G., Schmid, F., Schröder, D.: Verifiable timed linkable ring signatures for scalable payments for monero. In: Atluri, V., Di Pietro, R., Jensen, C.D., Meng, W. (eds.) ESORICS 2022, Part II. LNCS, vol. 13555, pp. 467–486. Springer, Heidelberg (Sep 2022). https://doi.org/10.1007/978-3-031-17146-8_23
50. Tsang, P.P., Wei, V.K.: Short linkable ring signatures for e-voting, e-cash and attestation. In: Deng, R.H., Bao, F., Pang, H., Zhou, J. (eds.) Information Security Practice and Experience, First International Conference, ISPEC 2005, Singapore, April 11-14, 2005, Proceedings. Lecture Notes in Computer Science, vol. 3439, pp. 48–60. Springer (2005), https://doi.org/10.1007/978-3-540-31979-5_5
51. Xu, S., Yung, M.: Accountable ring signatures: A smart card approach. In: Quisquater, J.J., Paradinas, P., Deswarte, Y., El Kalam, A.A. (eds.) Smart Card Research and Advanced Applications VI. pp. 271–286. Springer US, Boston, MA (2004)
52. Yuen, T.H., Esgin, M.F., Liu, J.K., Au, M.H., Ding, Z.: DualRing: Generic construction of ring signatures with efficient instantiations. In: Malkin, T., Peikert, C. (eds.) CRYPTO 2021, Part I. LNCS, vol. 12825, pp. 251–281. Springer, Heidelberg, Virtual Event (Aug 2021). https://doi.org/10.1007/978-3-030-84242-0_10
53. Yuen, T.H., Liu, J.K., Au, M.H., Susilo, W., Zhou, J.: Efficient Linkable and/or Threshold Ring Signature Without Random Oracles. *Comput. J.* **56**(4), 407–421 (2013)

Appendices

A Standard Tools and Definitions

A.1 Commitment Schemes

A non-interactive commitment is a pair of PPT algorithms $(\text{Setup}, \text{Com})$, where Setup takes as input 1^λ and returns a commitment key ck and Com takes as input a message m , and outputs a commitment $\text{Com} = \text{Com}_{\text{ck}}(m; r)$, where r is the randomness used to generate Com . The pair (m, r) is called the opening. Intuitively, a commitment satisfies two properties called binding and hiding. The first property says that it is hard to open a commitment in two different ways. The second property says that a commitment hides the underlying message.

Definition 25 (Perfect Binding). *We say that a non-interactive commitment is perfectly binding if $\nexists m_0 \neq m_1, r_0, r_1$ s.t. $\text{Com}_{\text{ck}}(m_0; r_0) = \text{Com}_{\text{ck}}(m_1; r_1)$, where $\text{ck} \leftarrow \text{Setup}(1^\lambda)$.*

Definition 26 (Computational Hiding). *We say that a non-interactive commitment is computationally hiding if for all PPT adversaries $\mathcal{A}$ the following quantity is negligible*

$$\left| \Pr \left[\mathcal{A}^{\text{O}(\text{ck}, 0, \cdot, \cdot)}(1^\lambda, \text{ck}) = 1 \right] - \Pr \left[\mathcal{A}^{\text{O}(\text{ck}, 1, \cdot, \cdot)}(1^\lambda, \text{ck}) = 1 \right] \right|,$$

where the oracle $\text{O}(\text{ck}, b, \cdot, \cdot)$ with hard-wired $b \in \{0, 1\}$ and $\text{ck} \leftarrow \text{Setup}(1^\lambda)$ takes as input a pair of messages m_0, m_1 , and outputs $\text{Com}_{\text{ck}}(m_b)$.

A.2 Public Key Encryption

A public key encryption scheme is a set of PPT algorithms $\text{PKE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: outputs public parameters pp .
- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$: generates a new public and secret key pair.
- $\text{a} \leftarrow \text{Enc}(m, \text{pk})$: on input a message m , and a public key pk , output a ciphertext a .
- $m \leftarrow \text{Dec}(\text{a}, \text{sk})$ on input a ciphertext a , and a secret key sk , output a message m .

Additionally, a public key encryption scheme is homomorphic w.r.t a function f , if there exists a PPT algorithm that works as follows.

- $\text{a}' \leftarrow \text{Eval}(\text{a}, x, \text{pk})$: on input a ciphertext a , a message x , and the public key pk . Let $y \leftarrow \text{Dec}(\text{a}, \text{sk})$, it returns a ciphertext a' s.t. $f(y, x) = \text{Dec}(\text{a}', \text{sk})$.

A public key encryption scheme is IND-CPA secure if the probability that a PPT adversary $\mathcal{A}$ wins the following game is negligibly close to $\frac{1}{2}$. The game involves the following steps: (i) $\mathcal{A}$ has access to the public key and outputs two messages m_0 and m_1 ; (ii) the challenger encrypts one of the two messages; (iii) $\mathcal{A}$ has to guess which message was encrypted.

ElGamal Encryption. The ElGamal encryption scheme is a public key encryption scheme with the following algorithms. The public parameters pp produced by Setup are implicitly available to all other algorithms:

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: on input the security parameter, sample a cyclic group $\hat{\mathbb{G}}$ of prime order p , a generator $\hat{g}$. Output $\text{pp} = (\hat{\mathbb{G}}, \hat{g})$.
- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$: sample an element $\zeta \leftarrow \mathbb{Z}_p^*$. Define public key as $\text{pk} = \hat{\mathbf{v}} = (\zeta \hat{g}, \hat{g})^\top \in \hat{\mathbb{G}}^{2 \times 1}$ and $\text{sk} = \zeta = (-\zeta^{-1}, 1)$. Output (pk, sk) .

- $\hat{\mathbf{a}} \leftarrow \text{Enc}(\hat{m}, \text{pk})$: with input the public key and a message $\hat{m} \in \hat{\mathbb{G}}$, sample $r \leftarrow \mathbb{Z}_p$ and output ciphertext $\hat{\mathbf{a}} = \mathbf{e}^\top \hat{m} + \hat{\mathbf{v}}r \in \hat{\mathbb{G}}^{2 \times 1}$, where $\mathbf{e} = (0, 1)$.
- $\hat{m} \leftarrow \text{Dec}(\hat{\mathbf{a}}, \text{sk})$: with input the secret key and a ciphertext $\mathbf{a} \in \hat{\mathbb{G}}^{2 \times 1}$, output $\hat{m} = \zeta \hat{\mathbf{a}}$.

The ElGamal encryption scheme is also homomorphic, with the function f being the group operation. In more detail:

- $\mathbf{a}' \leftarrow \text{Eval}(\mathbf{a}_1, \hat{m}_2, \text{pk})$: compute $\mathbf{a}_2 = \text{Enc}(\hat{m}_2, \text{pk})$, output $\mathbf{a}' = \mathbf{a}_1 + \mathbf{a}_2$. If the ciphertexts contained messages $\hat{m}_1$ and $\hat{m}_2$, the output ciphertext will contain message $\hat{m}_1 + \hat{m}_2$.

The ElGamal encryption is IND-CPA secure if the DDH assumption holds in $\hat{\mathbb{G}}$. Additionally, ciphertexts updated with Eval are identically distributed to freshly generated ciphertexts.

We also add the following definition.

Definition 27 (Verifiability of Keys). For all $\lambda \in \mathbb{N}$, $(\text{pp}, \text{td}) \leftarrow \text{Setup}(1^\lambda)$, it holds that $(\text{pk}, \text{sk}) \in \text{KeyGen}()$ iff $(\text{pk}, \text{sk}) \in R_{\text{key}}^{\text{PKE}}$. We also require that for all pk there exists a unique sk s.t. $(\text{pk}, \text{sk}) \in R_{\text{key}}^{\text{PKE}}$.

Note that the ElGamal encryption scheme satisfies Def. 27 for $R_{\text{key}}^{\text{PKE}} = \{(\text{pk}, \text{sk}) | \exists \zeta \in \mathbb{Z}_p^* \text{ s.t. } \text{pk} = (\zeta \hat{g}, \hat{g})^\top \wedge \text{sk} = (-\zeta^{-1}, 1)\}$. Indeed, it holds that:

1. For every $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$, it hold that $(\text{pk}, \text{sk}) \in R_{\text{key}}^{\text{PKE}}$.
2. Every $(\text{pk}, \text{sk}) \in R_{\text{key}}^{\text{PKE}}$ lies in the output space of $\text{KeyGen}()$.
3. For a given pk , there exists a unique sk such that $(\text{pk}, \text{sk}) \in R_{\text{key}}^{\text{PKE}}$. This is because the exponentiation is a unique map.

A.3 Non-Interactive Proof Systems

Let us consider an NP language $\mathcal{L}$ with associated poly-time relation $R_{\mathcal{L}}$. A non-interactive proof system for $R_{\mathcal{L}}$ consists of the following algorithms. The group key $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$ is considered as an implicit input to all algorithms.

- $(\text{crs}, \text{xk}) \leftarrow \text{CRSSetup}(\text{gk})$: on input the group key, output a common reference string $\text{crs} \in \{0, 1\}^\lambda$, and an extraction trapdoor xk .
- $\Pi \leftarrow \text{Prv}(\text{crs}, x, w)$: on input statement x and witness w s.t. $(x, w) \in R_{\mathcal{L}}$, output a proof Π .
- $0/1 \leftarrow \text{PVfy}(\text{crs}, x, \Pi)$: on input statement x and proof Π , output either 1 to accept or 0 to reject.
- $\Pi' \leftarrow \text{RandPr}(\text{crs}, x, \Pi)$: on input statement x and proof Π for $x \in \mathcal{L}$, output a randomized proof Π' .

A non-interactive proof system is said to be witness indistinguishable (NIWI) if all the properties below are satisfied.

Definition 28 (Completeness). A proof system for $R_{\mathcal{L}}$ is complete if $\forall \lambda \in \mathbb{N}$, $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, $\text{crs} \leftarrow \text{CRSSetup}(\text{gk})$, $(x, w) \in R_{\mathcal{L}}$, and $\Pi \leftarrow \text{Prv}(\text{crs}, x, w)$ it holds that $\Pr[\text{PVfy}(\text{crs}, x, \Pi) = 1] = 1$.

Definition 29 (Witness Indistinguishability). We say that the proof system is witness indistinguishable (WI) if the following holds. For all (x, w_1, w_2) such that $(x, w_1), (x, w_2) \in R_{\mathcal{L}}$, the tuples (crs, Π_1) and (crs, Π_2) , where $\text{crs} \leftarrow \text{CRSSetup}(\text{gk})$, $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$ and for $i \in [2]$, $\Pi_i \leftarrow \text{Prv}(\text{crs}, x, w_i)$, are computationally indistinguishable. If the two tuples are identically distributed, we say that the proof system is perfect WI.

Definition 30 (Soundness). For all PPT $\mathcal{A}$, and for $\text{crs} \leftarrow \text{CRSSetup}(\text{gk})$, $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, the probability that $\mathcal{A}(\text{crs})$ outputs (x, Π) such that $x \notin \mathcal{L}$ but $\text{PVfy}(\text{crs}, x, \Pi) = 1$, is negligible.

Additionally, a NIWI is said to be a NIWI proof of knowledge (PoK) if the property below is also satisfied.

Definition 31 (Adaptive Extractable Soundness). There exists a polynomial-time extractor $\text{Ext} = (\text{Ext}_0, \text{Ext}_1)$, for all $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$, with the following properties:

- $\text{Ext}_0(\text{gk})$ outputs $(\text{crs}_{\text{Ext}}, \text{xk})$ such that crs_{Ext} is indistinguishable from crs obtained running $\text{crs} \leftarrow \$\text{CRSSetup}(\text{gk})$.
- For all PPT $\mathcal{A}$, the probability that $\mathcal{A}(\text{crs}_{\text{Ext}}, \text{xk})$ outputs (x, Π) such that $\text{PVfy}(\text{crs}_{\text{Ext}}, x, \Pi) = 1$ and $(x, w) \notin R_{\mathcal{L}}$ where $w \leftarrow \text{Ext}_1(\text{crs}_{\text{Ext}}, \text{xk}, x, \Pi)$ is negligible.

We can also consider a stronger notion than witness indistinguishability called zero knowledge.

Definition 32 (Zero Knowledge). *There exists a polynomial-time simulator algorithm $\text{Sim} = (\text{Sim}_0, \text{Sim}_1)$ such that $\text{Sim}_0(\text{gk})$ outputs (crs, td) , and $\text{Sim}_1(\text{crs}, \text{td}, x)$ outputs a value π such that for all $(x, w) \in R_{\mathcal{L}}$ and PPT adversaries $\mathcal{A}$, the following two interactions are indistinguishable: in the first, we compute $\text{crs} \leftarrow \$\text{CRSSetup}(\text{gk})$ and give $\mathcal{A}$ both crs and oracle access to $\text{Prv}(\text{crs}, \cdot, \cdot)$ (where Prv will output $\perp$ on input $(x, w) \notin R_{\mathcal{L}}$); in the second, we compute crs as $(\text{crs}, \text{td}) \leftarrow \text{Sim}_0(\text{gk})$, and give $\mathcal{A}$ such crs and oracle access to $\text{Sim}_2(\text{crs}, \text{td}, \cdot, \cdot)$, with the exception that it outputs $\perp$ whenever $(x, w) \notin R_{\mathcal{L}}$.*

An extra property of a proof system is re-randomizability, which we state below. In our paper, when talking about a proof system we usually refer to a re-randomizable proof system.

Definition 33 (Re-Randomizable Proof System). *Consider the following experiment:*

- $\text{gk} \leftarrow \mathcal{G}(1^\lambda)$
- $\text{crs} \leftarrow \$\text{CRSSetup}(\text{gk})$
- $(x, w, \Pi) \leftarrow \mathcal{A}(\text{crs})$
- If either $\text{PVfy}(\text{crs}, x, \Pi) = 0$ or $(x, w) \notin R_{\mathcal{L}}$ output $\perp$ and abort. Otherwise, sample $b \leftarrow \$\{0, 1\}$
 - If $b = 0$ $\Pi' \leftarrow \text{Prv}(\text{crs}, x, w)$
 - If $b = 1$ $\Pi' \leftarrow \text{RandPr}(\text{crs}, x, \Pi)$
- $b' \leftarrow \mathcal{A}(\Pi')$.

We say that the proof system is re-randomizable if for every PPT $\mathcal{A}$, there exists a negligible function $\nu(\cdot)$, such that $\Pr[b = b'] \leq 1/2 + \nu(\lambda)$.

A.4 Groth-Sahai Proofs

The Groth-Sahai (GS) proof system [38] is a proof system for the language of satisfiable equations (of types listed below) over a bilinear group $\text{gk} = (p, \hat{\mathbb{G}}, \check{\mathbb{H}}, \mathbb{T}, e, \hat{g}, \check{h}) \leftarrow \mathcal{G}(1^\lambda)$. The prover wants to show that there exists an assignment of all the variables that satisfies the equation. Such equations can be of four types:

Pairing-product equations (PPE): For public constants $\hat{a}_j \in \hat{\mathbb{G}}$, $\check{b}_i \in \check{\mathbb{H}}$, $\gamma_{ij} \in \mathbb{Z}_p$, $t_{\mathbb{T}} \in \mathbb{T}$:

$$\sum_i \hat{x}_i \cdot \check{b}_i + \sum_j \hat{a}_j \cdot \check{y}_j + \sum_i \sum_j \gamma_{ij} \hat{x}_i \cdot \check{y}_j = t_{\mathbb{T}}.$$

Multi-scalar multiplication equation in $\hat{\mathbb{G}}$ ($\text{ME}_{\hat{\mathbb{G}}}$): For public constants $\hat{a}_j \in \hat{\mathbb{G}}$, $b_i \in \mathbb{Z}_p$, $\gamma_{ij} \in \mathbb{Z}_p$, $\hat{t} \in \hat{\mathbb{G}}$:

$$\sum_i \hat{x}_i b_i + \sum_j \hat{a}_j y_j + \sum_i \sum_j \gamma_{ij} \hat{x}_i y_j = \hat{t}.$$

Multi-scalar multiplication equation in $\check{\mathbb{H}}$ ($\text{ME}_{\check{\mathbb{H}}}$): For public constants $a_j \in \mathbb{Z}_p$, $\check{b}_i \in \check{\mathbb{H}}$, $\gamma_{ij} \in \mathbb{Z}_p$, $\check{t} \in \check{\mathbb{H}}$:

$$\sum_i x_i \check{b}_i + \sum_j a_j \check{y}_j + \sum_i \sum_j \gamma_{ij} x_i \check{y}_j = \check{t}.$$

Quadratic equation in $\mathbb{Z}_p$ (QE): For public constants $a_j \in \mathbb{Z}_p$, $b_i \in \mathbb{Z}_p$, $\gamma_{ij} \in \mathbb{Z}_p$, $t \in \mathbb{Z}_p$:

$$\sum_i x_i b_i + \sum_j a_j y_j + \sum_i \sum_j \gamma_{ij} x_i y_j = t.$$

We use the notation and formalization of the GS proof system proposed in [28]. The GS proof system is a commit-and-prove system. Each committed variable is also provided with a public label that specifies the type of input (i.e., scalar or group element). Accordingly, the prover algorithm takes as input a label L which indicates the type of equation to be proved (i.e., $L \in \{\text{PPE}, \text{ME}_{\hat{\mathbb{G}}}, \text{ME}_{\check{\mathbb{H}}}, \text{QE}\}$). GS features the following PPT algorithms, the common reference string crs and the group key gk are considered as implicit input of all the algorithms.

- $\text{crs} \leftarrow \text{CRSSetup}(\text{gk})$: on input the group key, output the common reference string. The common reference string defines the parameters of the commitment scheme.
- $(l, c) \leftarrow \text{Com}(l, w; r)$: return a commitment (l, c) to message w according to the label l and randomness r .
- $\pi \leftarrow \text{Prv}(L, x, (l_1, w_1, r_1), \dots, (l_n, w_n, r_n))$: consider statement x as an equation of type specified by L , and on input a list of commitment openings produce a proof π .
- $0/1 \leftarrow \text{PVfy}(x, (l_1, c_1), \dots, (l_n, c_n), \pi)$: given committed variables, statement x , and proof π , output 1 to accept and 0 to reject.
- $((l_1, c'_1), \dots, (l_n, c'_n), \pi') \leftarrow \text{RandPr}(L, (l_1, c_1), \dots, (l_n, c_n), \pi; r)$: on input equation type specified by L , a list of commitments, a proof π , and a randomness r , output a re-randomized proof along with the corresponding list of re-randomized commitments.

GS can also be used to prove that a set of equations S , with possibly shared variables across the equations, has a satisfying assignment. To do so, the prover reuses the same commitments for the shared variables while executing the Prv algorithm for each individual equation. Notice that a commit-and-prove system can always fit the interface of a regular proof system by including both the commitments and proof elements in the proof itself.

The GS proof system is a NIWI for all types of the above equations under the SXDH assumption. In addition, it is a PoK for all equations involving solely group elements. To be more specific, Escala and Groth formulated the notion of F -knowledge [28] for a commit-and-prove system (a variation of adaptive extractable soundness). In a nutshell, it requires the existence of an Ext_2 algorithm that, on input a valid commitment and the extraction key produced by an algorithm Ext_1 which is also producing the crs , outputs a function F of the committed value. They prove that GS enjoys F -knowledge. For commitments to group elements, F is the identity function. Regarding commitments to scalars, F is a one-way function (i.e., exponentiation) that uniquely determines the committed value. Additionally, GS proofs are zero knowledge for equations of type $\text{ME}_{\hat{\mathbb{G}}}$, $\text{ME}_{\hat{\mathbb{H}}}$, QE , and for pairing-product equations in which $t_{\mathbb{T}} = 0_{\mathbb{T}}$ and no public constants are paired with each other. However, zero knowledge can be achieved if a representation of $t_{\mathbb{T}}$ in terms of the source groups is known. If this representation is known in terms of $\hat{g}$ and $\hat{h}$, zero knowledge comes basically at no extra cost.

An important feature of GS is that it follows the dual-mode paradigm. Basically, the CRS can be set in two modes that are indistinguishable under the SXDH assumption. In perfect binding mode, the CRS can be sampled with a trapdoor that allows for extraction and thus defines a perfectly binding commitment scheme giving a perfect proof of knowledge with computational zero knowledge. On the contrary, the perfect hiding mode CRS comes with a trapdoor that allows simulation and thus defines a perfectly hiding commitment scheme giving perfect WI (or ZK) and computational extractable soundness.

Internals of GS proofs. In [28], the authors provide a very fine-grained description of GS proofs. In this description, we report only the aspects that are relevant to our constructions. It is possible to write the equations of Sec. A.4 in a more compact way. Consider $\hat{\mathbf{x}} = (\hat{x}_1, \dots, \hat{x}_m)$ and $\tilde{\mathbf{y}} = (\tilde{y}_1, \dots, \tilde{y}_n)$, which may be both public constants (i.e., written before as $\hat{a}_j, \tilde{b}_i$) or secret values. Let $\Gamma = \{\gamma_{ij}\}_{i=1, j=1}^{m, n} \in \mathbb{Z}_p^{m \times n}$. We can now write a PPE as $\hat{\mathbf{x}}\Gamma\tilde{\mathbf{y}} = t_{\mathbb{T}}$. Similarly, a $\text{ME}_{\hat{\mathbb{G}}}$, a $\text{ME}_{\hat{\mathbb{H}}}$, and a QE can be written as $\hat{\mathbf{x}}\Gamma\mathbf{y} = \hat{t}$, $\mathbf{x}\Gamma\tilde{\mathbf{y}} = \tilde{t}$, and $\mathbf{x}\Gamma\mathbf{y} = t$. This holds for $\hat{\mathbf{x}} \in \hat{\mathbb{G}}^{1 \times m}$, $\tilde{\mathbf{y}} \in \hat{\mathbb{H}}^{n \times 1}$, $\mathbf{x} \in \mathbb{Z}_p^{1 \times m}$, $\mathbf{y} \in \mathbb{Z}_p^{n \times 1}$. The structure of the crs is clear from Fig. 13, where the generation of perfectly binding crs (i.e., via Ext_0) and of a perfectly hiding crs (i.e., via Sim_0) are shown.

In Fig. 14, we report the commitment labels and corresponding commit algorithm that are of interest for this work. Commitments of type $\text{unit}_{\hat{\mathbb{G}}}$, $\text{base}_{\hat{\mathbb{G}}}$, $\text{unit}_{\hat{\mathbb{H}}}$, and $\text{base}_{\hat{\mathbb{H}}}$ have a distinctive feature that is crucial for simulation. The simulation trapdoor specifies ρ such that $\hat{\mathbf{u}} = \rho\hat{\mathbf{v}}$ and $\mathbf{e}^{\top}\hat{g} = \rho\hat{\mathbf{v}} - \hat{\mathbf{w}}$. This means that commitments of types $\text{base}_{\hat{\mathbb{G}}}$ and $\text{unit}_{\hat{\mathbb{G}}}$ can be equivocated as either commitments to $\hat{g}$ and 1 or as commitments to $\hat{0}$ and 0. The same holds for commitments of type $\text{unit}_{\hat{\mathbb{H}}}$ and $\text{base}_{\hat{\mathbb{H}}}$.

In Fig. 15 and in Fig. 16, we report the prover and verifier algorithm respectively. In Fig. 17 we report the proof re-randomization algorithm.

$(\text{crs}, \text{xk}) \leftarrow \text{Ext}_0(\text{gk})$	$(\text{crs}, \text{td}) \leftarrow \text{Sim}_0(\text{gk})$
1 : Parse $\text{gk} = (p, \hat{\mathbb{G}}, \hat{\mathbb{H}}, \mathbb{T}, e, \hat{g}, \hat{h})$	1 : Parse $\text{gk} = (p, \hat{\mathbb{G}}, \hat{\mathbb{H}}, \mathbb{T}, e, \hat{g}, \hat{h})$
2 : $\rho \leftarrow \mathbb{Z}_p, \xi \leftarrow \mathbb{Z}_p^*$ and $\sigma \leftarrow \mathbb{Z}_p, \psi \leftarrow \mathbb{Z}_p^*$	2 : $\rho \leftarrow \mathbb{Z}_p, \xi \leftarrow \mathbb{Z}_p^*$ and $\sigma \leftarrow \mathbb{Z}_p, \psi \leftarrow \mathbb{Z}_p^*$
3 : $\hat{\mathbf{v}} = (\xi \hat{g}, \hat{g})^\top$ and $\hat{\mathbf{v}} = (\psi \hat{h}, \hat{h})$	3 : $\hat{\mathbf{v}} = (\xi \hat{g}, \hat{g})^\top$ and $\hat{\mathbf{v}} = (\psi \hat{h}, \hat{h})$
4 : $\hat{\mathbf{w}} = \rho \hat{\mathbf{v}}$ and $\hat{\mathbf{w}} = \sigma \hat{\mathbf{v}}$	4 : $\hat{\mathbf{w}} = \rho \hat{\mathbf{v}} - (\hat{0}, \hat{g})^\top$ and $\hat{\mathbf{w}} = \sigma \hat{\mathbf{v}} - +(\check{0}, \hat{g})$
5 : $\hat{\mathbf{u}} = \hat{\mathbf{w}} + (\hat{0}, \hat{g})^\top$ and $\hat{\mathbf{u}} = \hat{\mathbf{w}} + (\check{0}, \hat{g})$	5 : $\hat{\mathbf{u}} = \hat{\mathbf{w}} + (\hat{0}, \hat{g})^\top$ and $\hat{\mathbf{u}} = \hat{\mathbf{w}} + (\check{0}, \hat{g})$
6 : $\xi = (-\xi^{-1} \bmod p, 1)$ and	6 : $\text{crs} = (\hat{\mathbf{u}}, \hat{\mathbf{v}}, \hat{\mathbf{w}}, \check{\mathbf{u}}, \check{\mathbf{v}}, \check{\mathbf{w}})$
7 : $\psi = (-\psi^{-1} \bmod p, 1)^\top$	7 : $\text{td} = (\rho, \sigma)$
8 : $\text{crs} = (\hat{\mathbf{u}}, \hat{\mathbf{v}}, \hat{\mathbf{w}}, \check{\mathbf{u}}, \check{\mathbf{v}}, \check{\mathbf{w}})$	8 : return (crs, td)
9 : $\text{xk} = (\xi, \psi)$	
10 : return (crs, xk)	

Fig. 13. Generation of the CRS along with either the extraction key or the trapdoor key in the GS proof system.

Input	Randomness	Output	Input	Randomness	Output
$\text{pub}_{\hat{\mathbb{G}}}, \hat{x}$	$r = 0, s = 0$	$\hat{\mathbf{c}} = \mathbf{e}^\top \hat{x}$	$\text{pub}_{\hat{\mathbb{H}}}, \check{y}$	$r = 0, s = 0$	$\check{\mathbf{d}} = \check{y} \mathbf{e}$
$\text{com}_{\hat{\mathbb{G}}}, \hat{x}$	$r, s \leftarrow \mathbb{Z}_p$	$\hat{\mathbf{c}} = \mathbf{e}^\top \hat{x} + \hat{\mathbf{v}} r + \hat{\mathbf{w}} s$	$\text{com}_{\hat{\mathbb{H}}}, \check{x}$	$r, s \leftarrow \mathbb{Z}_p$	$\check{\mathbf{d}} = \check{y} \mathbf{e} + r \check{\mathbf{v}} + s \check{\mathbf{w}}$
$\text{base}_{\hat{\mathbb{G}}}, \hat{g}$	$r = 0, s = 0$	$\hat{\mathbf{c}} = \mathbf{e}^\top \hat{g}$	$\text{base}_{\hat{\mathbb{H}}}, \check{h}$	$r = 0, s = 0$	$\check{\mathbf{d}} = \check{h} \mathbf{e}$
$\text{sca}_{\hat{\mathbb{G}}}, x$	$r \leftarrow \mathbb{Z}_p, s = 0$	$\hat{\mathbf{c}} = \hat{\mathbf{u}} x + \hat{\mathbf{v}} r$	$\text{sca}_{\hat{\mathbb{H}}}, y$	$r \leftarrow \mathbb{Z}_p, s = 0$	$\check{\mathbf{d}} = y \check{\mathbf{u}} + r \check{\mathbf{v}}$
$\text{unit}_{\hat{\mathbb{G}}}, 1$	$r = 0, s = 0$	$\hat{\mathbf{c}} = \hat{\mathbf{u}}$	$\text{unit}_{\hat{\mathbb{H}}}, 1$	$r = 0, s = 0$	$\check{\mathbf{d}} = \check{\mathbf{u}}$

Fig. 14. GS commit labels and corresponding commit algorithm, $\mathbf{e} = (0, 1)$.

For the sake of simplicity, we omit the explicit description of Sim_1 which can be found in [28]. In a nutshell, the simulator sets all of the committed variables to the corresponding neutral element with the goal of using a trivial assignment to satisfy the equation. Having the trapdoor, the simulator can equivocate the commitments of type $\text{base}_{\hat{\mathbb{G}}}, \text{unit}_{\hat{\mathbb{G}}}, \text{base}_{\hat{\mathbb{H}}}, \text{unit}_{\hat{\mathbb{H}}}$ to the neutral element. This gives the simulator a satisfying witness, and using this simulated witness it can now create the proof as an honest prover would do.

$$\begin{array}{l}
\text{Prv}(L, \Gamma, \{(l_{x_i}, x_i, (r_{x_i}, s_{x_i}))\}_{i=1}^m, \{(l_{y_j}, y_j, (r_{y_j}, s_{y_j}))\}_{j=1}^n) \\
\hline
\text{if } \mathbf{x} \in \hat{\mathbb{G}}^m \text{ define } \hat{\mathbf{C}} = \mathbf{e}^\top \mathbf{x} + \hat{\mathbf{v}} \mathbf{r}_x + \hat{\mathbf{w}} \mathbf{s}_x \text{ else if } \mathbf{x} \in \mathbb{Z}_p^m \text{ define } \hat{\mathbf{C}} = \hat{\mathbf{u}} \mathbf{x} + \hat{\mathbf{v}} \mathbf{r}_x \\
\text{if } \mathbf{y} \in \hat{\mathbb{H}}^n \text{ define } \check{\mathbf{D}} = \mathbf{e}^\top \mathbf{y} + \mathbf{r}_y \check{\mathbf{v}} + \mathbf{s}_y \check{\mathbf{w}} \text{ else if } \mathbf{y} \in \mathbb{Z}_p^n \text{ define } \check{\mathbf{D}} = \check{\mathbf{u}} \mathbf{y} + \mathbf{r}_y \check{\mathbf{v}} \\
\text{Set } \alpha = \beta = \gamma = \delta = 0 \\
\text{if } L = \text{PPE } \alpha, \beta, \gamma, \delta \leftarrow \mathbb{Z}_p \\
\text{if } L = \text{ME}_{\hat{\mathbb{G}}} \alpha, \beta \leftarrow \mathbb{Z}_p \\
\text{if } L = \text{ME}_{\hat{\mathbb{H}}} \alpha, \gamma \leftarrow \mathbb{Z}_p \\
\text{if } L = \text{QE } \alpha \leftarrow \mathbb{Z}_p \\
\hat{\pi}_{\hat{\mathbf{v}}} = \mathbf{r}_x \Gamma \check{\mathbf{D}} + \alpha \check{\mathbf{v}} + \beta \check{\mathbf{w}} \quad \hat{\pi}_{\hat{\mathbf{v}}} = (\hat{\mathbf{C}} - \hat{\mathbf{v}} \mathbf{r}_x - \hat{\mathbf{w}} \mathbf{s}_x) \Gamma \mathbf{r}_y - \hat{\mathbf{v}} \alpha - \hat{\mathbf{w}} \gamma \\
\hat{\pi}_{\hat{\mathbf{w}}} = \mathbf{s}_x \Gamma \check{\mathbf{D}} + \gamma \check{\mathbf{v}} + \delta \check{\mathbf{w}} \quad \hat{\pi}_{\hat{\mathbf{w}}} = (\hat{\mathbf{C}} - \hat{\mathbf{v}} \mathbf{r}_x - \hat{\mathbf{w}} \mathbf{s}_x) \Gamma \mathbf{s}_y - \hat{\mathbf{v}} \beta - \hat{\mathbf{w}} \delta \\
\text{return } \boldsymbol{\pi} = (\hat{\pi}_{\hat{\mathbf{v}}}, \hat{\pi}_{\hat{\mathbf{v}}}, \hat{\pi}_{\hat{\mathbf{w}}}, \hat{\pi}_{\hat{\mathbf{w}}})
\end{array}$$

Fig. 15. Prover algorithm of the GS proof system.

$\text{PVfy}(L, \Gamma, \{(l_{x_i}, \hat{c}_i)\}_{i=1}^m, \{(l_{y_j}, \check{d}_j)\}_{j=1}^n, \pi, t)$

Check that the equation has a valid format.
 Check $\hat{C} = (\hat{c}_1 \dots \hat{c}_m) \in \hat{\mathbb{G}}^{2 \times m}$ and $\check{D} = (\check{d}_1 \dots \check{d}_n)^\top \in \check{\mathbb{H}}^{n \times 2}$
 Check $\pi = (\tilde{\pi}_{\hat{v}}, \tilde{\pi}_{\hat{w}}, \hat{\pi}_{\check{v}}, \hat{\pi}_{\check{w}}) \in \check{\mathbb{H}}^{2 \times 1} \times \check{\mathbb{H}}^{2 \times 1} \times \hat{\mathbb{G}}^{1 \times 2} \times \hat{\mathbb{G}}^{1 \times 2}$
if $L = \text{PPE}$ Check that $t \in \mathbb{T}$ and compute $\mathbf{l}_{\mathbb{T}} = \mathbf{e}^\top t \mathbf{e}$
if $L = \text{ME}_{\hat{\mathbb{G}}}$ Check that $t \in \hat{\mathbb{G}}$ and compute $\mathbf{l}_{\mathbb{T}} = \mathbf{e}^\top t \tilde{\mathbf{u}}$
if $L = \text{ME}_{\check{\mathbb{H}}}$ Check that $t \in \check{\mathbb{H}}$ and compute $\mathbf{l}_{\mathbb{T}} = \hat{\mathbf{u}} t \mathbf{e}$
if $L = \text{QE}$ Check that $t \in \mathbb{Z}_p$ and compute $\mathbf{l}_{\mathbb{T}} = \hat{\mathbf{u}} t \tilde{\mathbf{u}}$
 Check $\hat{C} \Gamma \check{D} = \hat{\mathbf{v}} \tilde{\pi}_{\hat{v}} + \hat{\mathbf{w}} \tilde{\pi}_{\hat{w}} + \hat{\pi}_{\check{v}} \check{\mathbf{v}} + \hat{\pi}_{\check{w}} \check{\mathbf{w}} + \mathbf{l}_{\mathbb{T}}$
return 1 if and only if all checks pass and 0 otherwise.

Fig. 16. Verifier algorithm of the GS proof system.

$(\hat{C}', \check{D}', \pi') \leftarrow \text{RandPr}(L, \Gamma, \{(l_{x_i}, \hat{c}_i)\}_{i=1}^m, \{(l_{y_j}, \check{d}_j)\}_{j=1}^n, \pi; \mathbf{r})$

Parse $\mathbf{r} = (\mathbf{r}_x, \mathbf{s}_x, \mathbf{r}_y, \mathbf{s}_y)$
 Define $\hat{C} = (\hat{c}_1 \dots \hat{c}_m) \in \hat{\mathbb{G}}^{2 \times m}$, $\check{D} = (\check{d}_1 \dots \check{d}_n)^\top \in \check{\mathbb{H}}^{n \times 2}$ and parse $\pi = (\tilde{\pi}_{\hat{v}}, \tilde{\pi}_{\hat{w}}, \hat{\pi}_{\check{v}}, \hat{\pi}_{\check{w}})$
if $\mathbf{x} \in \hat{\mathbb{G}}^m$ define $\hat{C}' = \hat{C} + \hat{\mathbf{v}} \mathbf{r}_x + \hat{\mathbf{w}} \mathbf{s}_x$ **else if** $\mathbf{x} \in \mathbb{Z}_p^m$ define $\hat{C}' = \hat{C} + \hat{\mathbf{v}} \mathbf{r}_x$
if $\mathbf{y} \in \check{\mathbb{H}}^n$ define $\check{D}' = \check{D} + \mathbf{r}_y \check{\mathbf{v}} + \mathbf{s}_y \check{\mathbf{w}}$ **else if** $\mathbf{y} \in \mathbb{Z}_p^n$ define $\check{D}' = \check{D} + \mathbf{r}_y \check{\mathbf{v}}$
if $L = \text{PPE}$ $\alpha, \beta, \gamma, \delta \leftarrow \mathbb{Z}_p$
if $L = \text{ME}_{\hat{\mathbb{G}}}$ $\alpha, \beta \leftarrow \mathbb{Z}_p$
if $L = \text{ME}_{\check{\mathbb{H}}}$ $\alpha, \gamma \leftarrow \mathbb{Z}_p$
if $L = \text{QE}$ $\alpha \leftarrow \mathbb{Z}_p$
 $\tilde{\pi}_{\hat{v}}' = \tilde{\pi}_{\hat{v}} + \mathbf{r}_x \Gamma \check{D}' + \alpha \check{\mathbf{v}} + \beta \check{\mathbf{w}} \quad \hat{\pi}_{\check{v}}' = \hat{\pi}_{\check{v}} + \hat{C} \Gamma \mathbf{r}_y - \hat{\mathbf{v}} \alpha - \hat{\mathbf{w}} \gamma$
 $\tilde{\pi}_{\hat{w}}' = \tilde{\pi}_{\hat{w}} + \mathbf{s}_x \Gamma \check{D}' + \gamma \check{\mathbf{v}} + \delta \check{\mathbf{w}} \quad \hat{\pi}_{\check{w}}' = \hat{\pi}_{\check{w}} + \hat{C} \Gamma \mathbf{s}_y - \hat{\mathbf{v}} \beta - \hat{\mathbf{w}} \delta$
return $(\hat{C}', \check{D}', \pi' = (\tilde{\pi}_{\hat{v}}', \tilde{\pi}_{\hat{w}}', \hat{\pi}_{\check{v}}', \hat{\pi}_{\check{w}}'))$

Fig. 17. Proof re-randomization algorithm of the GS proof system.

B Details on Our Extendable Shuffle

We build our extendable shuffle argument building upon the shuffle argument of [37], which is based on Groth-Sahai proofs. In particular, we commit to elements $\hat{e}_i$ and to the bits bit_i using as a commitment scheme the ElGamal encryption in $\hat{\mathbb{G}}$ and $\check{\mathbb{H}}$ respectively. In the following, given a vector $\mathbf{v}$ we indicate its i -th component as $\mathbf{v}^i$. Additionally, we use the following notation to refer to the GS commitment to a variable v . If the GS commitment to v is in group $\hat{\mathbb{G}}$ we write $\hat{c}_v$. If the GS commitment to v is in group $\check{\mathbb{H}}$ we write $\check{d}_v$. We use the same convention for the randomness r and s used to generate the GS commitment. The algorithms of our extendable shuffle argument are the following:

- $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk}, N)$: run $(\text{crs}_1, \text{xk}_1) \leftarrow \text{GS.Ext}_0(\text{gk})$. Sample $p_1, \dots, p_N \leftarrow \mathbb{Z}_p$, compute sets $\{\hat{g}_i\} = \{\hat{g}p_i\}$, $\{\hat{\gamma}_i\} = \{\hat{g}p_i^2\}$, $\{\hat{h}_i\} = \{p_i\hat{h}\}$, $\{\hat{\delta}_i\} = \{p_i^2\hat{h}\}$. Set $\text{crs}_2 = (\{\hat{g}_i\}, \{\hat{\gamma}_i\}, \{\hat{h}_i\}, \{\hat{\delta}_i\})$. Return $(\text{crs} = (\text{crs}_1, \text{crs}_2), \text{td} = \text{xk}_1)$.
- $\text{ck} = \text{ComSetup}(\text{gk})$: sample $\hat{\text{ck}} \leftarrow \mathbb{G}$ and $\check{\text{ck}} \leftarrow \mathbb{H}$. Return $\text{ck} = (\hat{\text{ck}}, \check{\text{ck}})$.
- $\hat{\mathbf{z}} \leftarrow \text{Com}_{\hat{\text{ck}}}(\hat{e}; r)$: return $\hat{\mathbf{z}} = (\hat{g}r, \hat{e} + \hat{\text{ck}}r)$.
- $\check{\mathbf{m}} \leftarrow \text{Com}_{\check{\text{ck}}}(\text{bit}; r)$: Return $\check{\mathbf{m}} = (r\check{h}, \text{bit}\check{h} + r\check{\text{ck}})$.
- $(\Pi, \text{aux}_1, \dots, \text{aux}_n) \leftarrow \text{ShPriv}(\text{crs}, \text{ck}, x, w)$: See Fig. 18 for a detailed description of the prover algorithm.
- $0/1 \leftarrow \text{ShVerify}(\text{crs}, \text{ck}, x, \Pi)$: reconstruct equations as shown in Fig. 18, appropriately parse Π , and for every equation run GS.PVfy with the obvious inputs.
- $(\Pi', x', \text{aux}_{n+1}, \mathbf{r}) \leftarrow \text{ShExtend}(\text{crs}, \text{ck}, x, \hat{\mathbf{z}}_{n+1}, r_{\check{\mathbf{m}}_{n+1}}, \Pi)$:
 1. Parse Π in terms of its commitments and proof elements lists.
 2. Parse x as $(k, \hat{\mathbf{z}}_1, \dots, \hat{\mathbf{z}}_n, \check{\mathbf{m}}_1, \dots, \check{\mathbf{m}}_n, \hat{e}_1, \dots, \hat{e}_k)$.
 3. For each of the equation types $\mathcal{D}^1, \mathcal{D}^2, \mathcal{F}, \mathcal{B}, \mathcal{Q}^1, \mathcal{Q}^2, \mathcal{V}^1, \mathcal{V}^2$, add a new equation by defining the corresponding new independent variables, $r_{\hat{\mathbf{z}}_{n+1}} = 0, f_{n+1} = 0, \text{bit}_{n+1} = 0, \check{a}'_{n+1} = \check{a}_{n+1} = \check{0}, \check{b}'_{n+1} = \check{b}_{n+1} = \check{0}, \hat{e}_{n+1} = \hat{0}, \hat{E}_{n+1} = \hat{0}$. The value of the variable $r_{\check{\mathbf{m}}_{n+1}}$ is set as the one given in input.
 4. Compute $\check{\mathbf{m}}_{n+1} = (r_{\check{\mathbf{m}}_{n+1}}\check{h}, \text{bit}_{n+1}\check{h} + r_{\check{\mathbf{m}}_{n+1}}\check{\text{ck}})$.
 5. Compute GS commitments to new variables and appropriately add them to the corresponding commitment lists.
 6. Compute the related new GS proofs and add them to the corresponding proof element lists.
 7. Compute auxiliary value $\text{aux}_{n+1} = (\text{aux}_{\mathcal{K}}^{n+1}, \text{aux}_{\mathcal{S}^1}^{n+1}, \text{aux}_{\mathcal{S}^2}^{n+1}, \text{aux}_{\mathcal{E}^1}^{n+1}, \text{aux}_{\mathcal{E}^2}^{n+1})$
 - $\text{aux}_{\mathcal{K}}^{n+1} = (\text{aux}_{\hat{\pi}_{\hat{e}}})$ with $\text{aux}_{\hat{\pi}_{\hat{e}}} = (r_{\text{bit}_{n+1}}\hat{\mathbf{u}}^1, r_{\text{bit}_{n+1}}\hat{\mathbf{u}}^2)$.
 - $\text{aux}_{\mathcal{S}^1}^{n+1} = (\text{aux}_{\hat{\pi}_{\hat{e}}}, \text{aux}_{\hat{\pi}_{\hat{w}}})$ with $\text{aux}_{\hat{\pi}_{\hat{e}}} = (r_{\hat{a}_{n+1}}\hat{\mathbf{u}}^1, r_{\hat{a}_{n+1}}\hat{\mathbf{u}}^2), \text{aux}_{\hat{\pi}_{\hat{w}}} = (s_{\hat{a}_{n+1}}\hat{\mathbf{u}}^1, s_{\hat{a}_{n+1}}\hat{\mathbf{u}}^2)$.
 - $\text{aux}_{\mathcal{S}^2}^{n+1} = (\text{aux}_{\hat{\pi}_{\hat{e}}}, \text{aux}_{\hat{\pi}_{\hat{w}}})$ with $\text{aux}_{\hat{\pi}_{\hat{e}}} = (r_{\hat{b}_{n+1}}\hat{\mathbf{u}}^1, r_{\hat{b}_{n+1}}\hat{\mathbf{u}}^2), \text{aux}_{\hat{\pi}_{\hat{w}}} = (s_{\hat{b}_{n+1}}\hat{\mathbf{u}}^1, s_{\hat{b}_{n+1}}\hat{\mathbf{u}}^2)$.
 - $\text{aux}_{\mathcal{E}^1}^{n+1} = (\text{aux}_{\hat{\pi}_{\hat{e}}}, \text{aux}_{\hat{\pi}_{\hat{v}}}, \text{aux}_{\hat{\pi}_{\hat{w}}}, \text{aux}_{\hat{\pi}_{\hat{w}}})$ with $\text{aux}_{\hat{\pi}_{\hat{e}}} = (r_{\hat{E}_{n+1}}\check{\mathbf{d}}_{\hat{a}_{n+1}}^1, r_{\hat{E}_{n+1}}\check{\mathbf{d}}_{\hat{a}_{n+1}}^2), \text{aux}_{\hat{\pi}_{\hat{v}}} = (\hat{0}, \hat{0}), \text{aux}_{\hat{\pi}_{\hat{w}}} = (\hat{0}, \hat{0}), \text{aux}_{\hat{\pi}_{\hat{w}}} = (s_{\hat{E}_{n+1}}\check{\mathbf{d}}_{\hat{a}_{n+1}}^1, s_{\hat{E}_{n+1}}\check{\mathbf{d}}_{\hat{a}_{n+1}}^2)$.
 - $\text{aux}_{\mathcal{E}^2}^{n+1} = (\text{aux}_{\hat{\pi}_{\hat{e}}}, \text{aux}_{\hat{\pi}_{\hat{v}}}, \text{aux}_{\hat{\pi}_{\hat{w}}}, \text{aux}_{\hat{\pi}_{\hat{w}}})$ with $\text{aux}_{\hat{\pi}_{\hat{e}}} = (r_{\hat{E}_{n+1}}\check{\mathbf{d}}_{\hat{b}_{n+1}}^1, r_{\hat{E}_{n+1}}\check{\mathbf{d}}_{\hat{b}_{n+1}}^2), \text{aux}_{\hat{\pi}_{\hat{v}}} = (\hat{0}, \hat{0}), \text{aux}_{\hat{\pi}_{\hat{w}}} = (\hat{0}, \hat{0}), \text{aux}_{\hat{\pi}_{\hat{w}}} = (s_{\hat{E}_{n+1}}\check{\mathbf{d}}_{\hat{b}_{n+1}}^1, s_{\hat{E}_{n+1}}\check{\mathbf{d}}_{\hat{b}_{n+1}}^2)$.
 8. Parse $\pi_{\mathcal{K}}$ as $(\hat{\pi}_{\hat{e}}, \hat{\pi}_{\hat{v}}, \hat{\pi}_{\hat{w}}, \hat{\pi}_{\hat{w}})$ and update $\hat{\pi}_{\hat{e}}$ as $\hat{\pi}_{\hat{e}} = \hat{\pi}_{\hat{e}} + \text{aux}_{\hat{\pi}_{\hat{e}}}$, with $\text{aux}_{\hat{\pi}_{\hat{e}}}$ taken from $\text{aux}_{\mathcal{K}}^{n+1}$.
 9. Parse $\pi_{\mathcal{S}^1}$ as $(\hat{\pi}_{\hat{e}}, \hat{\pi}_{\hat{v}}, \hat{\pi}_{\hat{w}}, \hat{\pi}_{\hat{w}})$ and update $\hat{\pi}_{\hat{v}} = \hat{\pi}_{\hat{v}} + \text{aux}_{\hat{\pi}_{\hat{v}}}, \hat{\pi}_{\hat{w}} = \hat{\pi}_{\hat{w}} + \text{aux}_{\hat{\pi}_{\hat{w}}}$ π with $\text{aux}_{\hat{\pi}_{\hat{v}}}, \text{aux}_{\hat{\pi}_{\hat{w}}}$ taken from $\text{aux}_{\mathcal{S}^1}^{n+1}$.
 10. Parse $\pi_{\mathcal{S}^2}$ as $(\hat{\pi}_{\hat{e}}, \hat{\pi}_{\hat{v}}, \hat{\pi}_{\hat{w}}, \hat{\pi}_{\hat{w}})$ and update $\hat{\pi}_{\hat{v}} = \hat{\pi}_{\hat{v}} + \text{aux}_{\hat{\pi}_{\hat{v}}}, \hat{\pi}_{\hat{w}} = \hat{\pi}_{\hat{w}} + \text{aux}_{\hat{\pi}_{\hat{w}}}$ π with $\text{aux}_{\hat{\pi}_{\hat{v}}}, \text{aux}_{\hat{\pi}_{\hat{w}}}$ taken from $\text{aux}_{\mathcal{S}^2}^{n+1}$.
 11. Parse $\pi_{\mathcal{E}^1}$ as $(\hat{\pi}_{\hat{e}}, \hat{\pi}_{\hat{v}}, \hat{\pi}_{\hat{w}}, \hat{\pi}_{\hat{w}})$ and update $\hat{\pi}_{\hat{e}} = \hat{\pi}_{\hat{e}} + \text{aux}_{\hat{\pi}_{\hat{e}}}, \hat{\pi}_{\hat{v}} = \hat{\pi}_{\hat{v}} + \text{aux}_{\hat{\pi}_{\hat{v}}}, \hat{\pi}_{\hat{w}} = \hat{\pi}_{\hat{w}} + \text{aux}_{\hat{\pi}_{\hat{w}}}, \hat{\pi}_{\hat{w}} = \hat{\pi}_{\hat{w}} + \text{aux}_{\hat{\pi}_{\hat{w}}}$ with $\text{aux}_{\hat{\pi}_{\hat{e}}}, \text{aux}_{\hat{\pi}_{\hat{v}}}, \text{aux}_{\hat{\pi}_{\hat{w}}}, \text{aux}_{\hat{\pi}_{\hat{w}}}$ taken from $\text{aux}_{\mathcal{E}^1}^{n+1}$.
 12. Parse $\pi_{\mathcal{E}^2}$ as $(\hat{\pi}_{\hat{e}}, \hat{\pi}_{\hat{v}}, \hat{\pi}_{\hat{w}}, \hat{\pi}_{\hat{w}})$ and update $\hat{\pi}_{\hat{e}} = \hat{\pi}_{\hat{e}} + \text{aux}_{\hat{\pi}_{\hat{e}}}, \hat{\pi}_{\hat{v}} = \hat{\pi}_{\hat{v}} + \text{aux}_{\hat{\pi}_{\hat{v}}}, \hat{\pi}_{\hat{w}} = \hat{\pi}_{\hat{w}} + \text{aux}_{\hat{\pi}_{\hat{w}}}, \hat{\pi}_{\hat{w}} = \hat{\pi}_{\hat{w}} + \text{aux}_{\hat{\pi}_{\hat{w}}}$ with $\text{aux}_{\hat{\pi}_{\hat{e}}}, \text{aux}_{\hat{\pi}_{\hat{v}}}, \text{aux}_{\hat{\pi}_{\hat{w}}}, \text{aux}_{\hat{\pi}_{\hat{w}}}$ taken from $\text{aux}_{\mathcal{E}^2}^{n+1}$.
 13. Randomize the statement and adjust the related GS commitments as follows for all $i \in [n+1]$:
 - Compute $\hat{\mathbf{z}}'_i = \hat{\mathbf{z}}_i + (\hat{g}r'_{\hat{\mathbf{z}}_i}, \hat{\text{ck}}r'_{\hat{\mathbf{z}}_i})$ with uniformly sampled $r'_{\hat{\mathbf{z}}_i}$ (i.e., obtain a commitment to the same value with randomness $r'_{\hat{\mathbf{z}}_i} + r'_{\hat{\mathbf{z}}_i}$).
 - Compute $\check{\mathbf{m}}'_i = \check{\mathbf{m}}_i + (r'_{\check{\mathbf{m}}_i}\check{h}, r'_{\check{\mathbf{m}}_i}\check{\text{ck}})$ with uniformly sampled $r'_{\check{\mathbf{m}}_i}$.
 - Adjust GS commitments $\check{\mathbf{d}}_{r_{\hat{\mathbf{z}}_i}}$ to variables $r_{\hat{\mathbf{z}}_i}$ as $\check{\mathbf{d}}_{r_{\hat{\mathbf{z}}_i}} = \check{\mathbf{d}}_{r_{\hat{\mathbf{z}}_i}} + \hat{\mathbf{u}}r'_{\hat{\mathbf{z}}_i}$ (i.e., obtain a GS commitment with the same randomness but to value $r_{\hat{\mathbf{z}}_i} + r'_{\hat{\mathbf{z}}_i}$).
 - Adjust GS commitments $\check{\mathbf{d}}_{r_{\check{\mathbf{m}}_i}}$ to variables $r_{\check{\mathbf{m}}_i}$ as $\check{\mathbf{d}}_{r_{\check{\mathbf{m}}_i}} = \check{\mathbf{d}}_{r_{\check{\mathbf{m}}_i}} + \hat{\mathbf{u}}r'_{\check{\mathbf{m}}_i}$.
- Set $x' = (k, \hat{\mathbf{z}}'_1, \dots, \hat{\mathbf{z}}'_{n+1}, \check{\mathbf{m}}'_1, \dots, \check{\mathbf{m}}'_{n+1}, \hat{e}_1, \dots, \hat{e}_k)$.
- 14. Run GS.RandPr on each of the proofs, appropriately fixing the random coins when re-randomizing proofs related to equations involving shared variables (i.e., s.t. we end up again with shared variables having the exact same commitments). Let $\mathbf{r} = (r_1, \dots, r_{n+1})$ with $r_i = (r_{\hat{a}_i}, s_{\hat{a}_i}, r_{\hat{b}_i}, s_{\hat{b}_i}, r_{\hat{E}_i}, s_{\hat{E}_i}, r_{\text{bit}_i})$. Namely, r_i contains all the randomnesses used to re-randomize the GS commitments to the corresponding i -th variables. Let randomized proof elements and commitments be contained in Π' .

15. Output $(\Pi', x', \text{aux}_{n+1}, \mathbf{r})$.
- $(\Pi', x', \text{aux}'_\alpha, \mathbf{r}) \leftarrow \text{ShAdd}(\text{crs}, \text{ck}, x, r_{\hat{z}_\alpha}, \alpha, \hat{e}_{k+1}, \text{aux}_\alpha, \Pi)$:
 1. Parse Π in terms of its commitments and proof elements lists.
 2. Parse x as $(k, \hat{z}_1, \dots, \hat{z}_n, \tilde{\mathbf{m}}_1, \dots, \tilde{\mathbf{m}}_n, \hat{e}_1, \dots, \hat{e}_k)$.
 3. Replace $\tilde{\mathbf{m}}_\alpha$ with a freshly computed $\tilde{\mathbf{m}}_\alpha = (r_{\tilde{\mathbf{m}}_\alpha} \tilde{h}, \text{bit}_\alpha \tilde{h} + r_{\tilde{\mathbf{m}}_\alpha} \check{\mathbf{c}}\mathbf{k})$ with $\text{bit}_\alpha = 1$ and uniformly sampled $r_{\tilde{\mathbf{m}}_\alpha}$.
 4. Replace $\hat{z}_\alpha$ with a freshly computed $\hat{z}_\alpha = (\hat{g}r_{\hat{z}_\alpha}, \hat{e} + \hat{\mathbf{c}}\mathbf{k}r_{\hat{z}_\alpha})$.
 5. For each of the equation types $\mathcal{D}^1, \mathcal{D}^2, \mathcal{F}, \mathcal{B}, \mathcal{Q}^1, \mathcal{Q}^2, \mathcal{V}^1, \mathcal{V}^2$, replace the variables in equations related to position α (i.e. $\mathcal{D}^1_{\cdot, \alpha}, \mathcal{D}^2_{\cdot, \alpha}, \mathcal{F}_\alpha, \mathcal{B}_\alpha, \mathcal{Q}^1_\alpha, \mathcal{Q}^2_\alpha, \mathcal{V}^1_\alpha, \mathcal{V}^2_\alpha$) as follows: $f_\alpha = 1, \text{bit}_\alpha = 1, \tilde{a}_\alpha = \tilde{a}'_\alpha = \tilde{h}_{k+1}, \tilde{b}_\alpha = \tilde{b}'_\alpha = \tilde{d}_{k+1}, \hat{c}_\alpha = \hat{g}_{k+1}, \hat{E}_\alpha = \hat{e}_{k+1}$. The value of the variable $r_{\hat{z}_\alpha}$ is set as the one taken in input. The value of the variable $r_{\tilde{\mathbf{m}}_\alpha}$ is set as the one sampled at step 3.
 6. Replace the GS commitments related to equations $\mathcal{D}^1_{\cdot, \alpha}, \mathcal{D}^2_{\cdot, \alpha}, \mathcal{F}_\alpha, \mathcal{B}_\alpha, \mathcal{Q}^1_\alpha, \mathcal{Q}^2_\alpha, \mathcal{V}^1_\alpha, \mathcal{V}^2_\alpha$ with freshly generated ones updating the corresponding commitment lists accordingly.
 7. Replace the GS proof elements related to equations $\mathcal{D}^1_{\cdot, \alpha}, \mathcal{D}^2_{\cdot, \alpha}, \mathcal{F}_\alpha, \mathcal{B}_\alpha, \mathcal{V}^1_\alpha, \mathcal{V}^2_\alpha$ with freshly generated ones updating the corresponding proof element lists accordingly.
 8. Parse $\text{aux}_\alpha = (\text{aux}_\mathcal{K}^\alpha, \text{aux}_{\mathcal{S}^1}^\alpha, \text{aux}_{\mathcal{S}^2}^\alpha, \text{aux}_{\mathcal{E}^1}^\alpha, \text{aux}_{\mathcal{E}^2}^\alpha)$
 9. Compute $\text{aux}'_\alpha = (\text{aux}'_\mathcal{K}, \text{aux}'_{\mathcal{S}^1}, \text{aux}'_{\mathcal{S}^2}, \text{aux}'_{\mathcal{E}^1}, \text{aux}'_{\mathcal{E}^2})$
 - $\text{aux}'_\mathcal{K} = (\text{aux}'_{\tilde{\pi}_\emptyset})$ with $\text{aux}'_{\tilde{\pi}_\emptyset} = (r_{\text{bit}_i} \tilde{\mathbf{u}}^1, r_{\text{bit}_i} \tilde{\mathbf{u}}^2)$.
 - $\text{aux}'_{\mathcal{S}^1} = (\text{aux}'_{\tilde{\pi}_\emptyset}, \text{aux}'_{\tilde{\pi}_w})$ with $\text{aux}'_{\tilde{\pi}_\emptyset} = (r_{\tilde{a}_i} \hat{\mathbf{u}}^1, r_{\tilde{a}_i} \hat{\mathbf{u}}^2), \text{aux}'_{\tilde{\pi}_w} = (s_{\tilde{a}_i} \hat{\mathbf{u}}^1, s_{\tilde{a}_i} \hat{\mathbf{u}}^2)$.
 - $\text{aux}'_{\mathcal{S}^2} = (\text{aux}'_{\tilde{\pi}_\emptyset}, \text{aux}'_{\tilde{\pi}_w})$ with $\text{aux}'_{\tilde{\pi}_\emptyset} = (r_{\tilde{b}_i} \hat{\mathbf{u}}^1, r_{\tilde{b}_i} \hat{\mathbf{u}}^2), \text{aux}'_{\tilde{\pi}_w} = (s_{\tilde{b}_i} \hat{\mathbf{u}}^1, s_{\tilde{b}_i} \hat{\mathbf{u}}^2)$.
 - $\text{aux}'_{\mathcal{E}^1} = (\text{aux}'_{\tilde{\pi}_\emptyset}, \text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\tilde{\pi}_w})$ with $\text{aux}'_{\tilde{\pi}_\emptyset} = (r_{\hat{E}_i} \tilde{\mathbf{d}}^1_{\tilde{a}_i}, r_{\hat{E}_i} \tilde{\mathbf{d}}^2_{\tilde{a}_i}), \text{aux}'_{\tilde{\pi}_w} = (0, r_{\tilde{a}_i} \hat{E}_i), \text{aux}'_{\tilde{\pi}_w} = (0, s_{\tilde{a}_i} \hat{E}_i), \text{aux}'_{\tilde{\pi}_w} = (s_{\hat{E}_i} \tilde{\mathbf{d}}^1_{\tilde{a}_i}, s_{\hat{E}_i} \tilde{\mathbf{d}}^2_{\tilde{a}_i})$.
 - $\text{aux}'_{\mathcal{E}^2} = (\text{aux}'_{\tilde{\pi}_\emptyset}, \text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\tilde{\pi}_w})$ with $\text{aux}'_{\tilde{\pi}_\emptyset} = (r_{\hat{E}_i} \tilde{\mathbf{d}}^1_{\tilde{b}_i}, r_{\hat{E}_i} \tilde{\mathbf{d}}^2_{\tilde{b}_i}), \text{aux}'_{\tilde{\pi}_w} = (0, r_{\tilde{b}_i} \hat{E}_i), \text{aux}'_{\tilde{\pi}_w} = (0, s_{\tilde{b}_i} \hat{E}_i), \text{aux}'_{\tilde{\pi}_w} = (s_{\hat{E}_i} \tilde{\mathbf{d}}^1_{\tilde{b}_i}, s_{\hat{E}_i} \tilde{\mathbf{d}}^2_{\tilde{b}_i})$.
 10. Parse $\pi_\mathcal{K}$ as $(\tilde{\pi}_\emptyset, \tilde{\pi}_w, \tilde{\pi}_w, \tilde{\pi}_w)$ and update $\tilde{\pi}_\emptyset$ as $\tilde{\pi}_\emptyset = \tilde{\pi}_\emptyset - \text{aux}_{\tilde{\pi}_\emptyset} + \text{aux}'_{\tilde{\pi}_\emptyset}$, with $\text{aux}_{\tilde{\pi}_\emptyset}$ taken from $\text{aux}_\mathcal{K}^i$ and $\text{aux}'_{\tilde{\pi}_\emptyset}$ taken from $\text{aux}'_\mathcal{K}$.
 11. Parse $\pi_{\mathcal{S}^1}$ as $(\tilde{\pi}_\emptyset, \tilde{\pi}_w, \tilde{\pi}_w, \tilde{\pi}_w)$ and update $\tilde{\pi}_w = \tilde{\pi}_w - \text{aux}_{\tilde{\pi}_w} + \text{aux}'_{\tilde{\pi}_w}$, $\hat{\pi}_w = \hat{\pi}_w - \text{aux}_{\hat{\pi}_w} + \text{aux}'_{\hat{\pi}_w}$, with $\text{aux}_{\tilde{\pi}_w}, \text{aux}_{\hat{\pi}_w}$ taken from $\text{aux}_{\mathcal{S}^1}^i$ and $\text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\hat{\pi}_w}$ taken from $\text{aux}'_{\mathcal{S}^1}$.
 12. Parse $\pi_{\mathcal{S}^2}$ as $(\tilde{\pi}_\emptyset, \tilde{\pi}_w, \tilde{\pi}_w, \tilde{\pi}_w)$ and update $\tilde{\pi}_w = \tilde{\pi}_w - \text{aux}_{\tilde{\pi}_w} + \text{aux}'_{\tilde{\pi}_w}$, $\hat{\pi}_w = \hat{\pi}_w - \text{aux}_{\hat{\pi}_w} + \text{aux}'_{\hat{\pi}_w}$, with $\text{aux}_{\tilde{\pi}_w}, \text{aux}_{\hat{\pi}_w}$ taken from $\text{aux}_{\mathcal{S}^2}^i$ and $\text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\hat{\pi}_w}$ taken from $\text{aux}'_{\mathcal{S}^2}$.
 13. Parse $\pi_{\mathcal{E}^1}$ as $(\tilde{\pi}_\emptyset, \tilde{\pi}_w, \tilde{\pi}_w, \tilde{\pi}_w)$ and update $\tilde{\pi}_w = \tilde{\pi}_w - \text{aux}_{\tilde{\pi}_w} + \text{aux}'_{\tilde{\pi}_w}$, $\hat{\pi}_w = \hat{\pi}_w - \text{aux}_{\hat{\pi}_w} + \text{aux}'_{\hat{\pi}_w}$, $\tilde{\pi}_w = \tilde{\pi}_w - \text{aux}_{\tilde{\pi}_w} + \text{aux}'_{\tilde{\pi}_w}$, $\hat{\pi}_w = \hat{\pi}_w - \text{aux}_{\hat{\pi}_w} + \text{aux}'_{\hat{\pi}_w}$ with $\text{aux}_{\tilde{\pi}_w}, \text{aux}_{\hat{\pi}_w}, \text{aux}_{\tilde{\pi}_w}, \text{aux}_{\hat{\pi}_w}$ taken from $\text{aux}_{\mathcal{E}^1}^i$ and $\text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\hat{\pi}_w}, \text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\hat{\pi}_w}$ taken from $\text{aux}'_{\mathcal{E}^1}$.
 14. Parse $\pi_{\mathcal{E}^2}$ as $(\tilde{\pi}_\emptyset, \tilde{\pi}_w, \tilde{\pi}_w, \tilde{\pi}_w)$ and update $\tilde{\pi}_w = \tilde{\pi}_w - \text{aux}_{\tilde{\pi}_w} + \text{aux}'_{\tilde{\pi}_w}$, $\hat{\pi}_w = \hat{\pi}_w - \text{aux}_{\hat{\pi}_w} + \text{aux}'_{\hat{\pi}_w}$, $\tilde{\pi}_w = \tilde{\pi}_w - \text{aux}_{\tilde{\pi}_w} + \text{aux}'_{\tilde{\pi}_w}$, $\hat{\pi}_w = \hat{\pi}_w - \text{aux}_{\hat{\pi}_w} + \text{aux}'_{\hat{\pi}_w}$ with $\text{aux}_{\tilde{\pi}_w}, \text{aux}_{\hat{\pi}_w}, \text{aux}_{\tilde{\pi}_w}, \text{aux}_{\hat{\pi}_w}$ taken from $\text{aux}_{\mathcal{E}^2}^i$ and $\text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\hat{\pi}_w}, \text{aux}'_{\tilde{\pi}_w}, \text{aux}'_{\hat{\pi}_w}$ taken from $\text{aux}'_{\mathcal{E}^2}$.
 15. Randomize the statement and adjust the related GS commitments as follows for all $i \in [n+1]$:
 - Compute $\hat{z}'_i = \hat{z}_i + (\hat{g}r'_{\hat{z}_i}, \hat{\mathbf{c}}\mathbf{k}r'_{\hat{z}_i})$ with uniformly sampled $r'_{\hat{z}_i}$ (i.e. obtain a commitment to the same value with randomness $r_{\hat{z}_i} + r'_{\hat{z}_i}$).
 - Compute $\tilde{\mathbf{m}}'_i = \tilde{\mathbf{m}}_i + (r'_{\tilde{\mathbf{m}}_i} \tilde{h}, r'_{\tilde{\mathbf{m}}_i} \check{\mathbf{c}}\mathbf{k})$ with uniformly sampled $r'_{\tilde{\mathbf{m}}_i}$.
 - Adjust GS commitments $\tilde{\mathbf{d}}_{r_{\hat{z}_i}}$ to variables $r_{\hat{z}_i}$ as $\tilde{\mathbf{d}}_{r_{\hat{z}_i}} = \tilde{\mathbf{d}}_{r_{\hat{z}_i}} + \hat{\mathbf{u}}r'_{\hat{z}_i}$ (i.e. obtain a GS commitment with the same randomness but to value $r_{\hat{z}_i} + r'_{\hat{z}_i}$).
 - Adjust GS commitments $\tilde{\mathbf{d}}_{r_{\tilde{\mathbf{m}}_i}}$ to variables $r_{\tilde{\mathbf{m}}_i}$ as $\tilde{\mathbf{d}}_{r_{\tilde{\mathbf{m}}_i}} = \tilde{\mathbf{d}}_{r_{\tilde{\mathbf{m}}_i}} + \hat{\mathbf{u}}r'_{\tilde{\mathbf{m}}_i}$.
- Set $x' = (k+1, \hat{z}'_1, \dots, \hat{z}'_n, \tilde{\mathbf{m}}'_1, \dots, \tilde{\mathbf{m}}'_n, \hat{e}_1, \dots, \hat{e}_{k+1})$.
16. Run GS.RandPr on each of the proofs, appropriately fixing the random coins when re-randomizing proofs related to equations involving shared variables (i.e., s.t. we end up again with shared variables having the exact same commitments). Let $\mathbf{r} = (r_1, \dots, r_n)$ with $r_i = (r_{\tilde{a}_i}, s_{\tilde{a}_i}, r_{\tilde{b}_i}, s_{\tilde{b}_i}, r_{\hat{E}_i}, s_{\hat{E}_i}, r_{\text{bit}_i})$. Namely, r_i contains all the randomnesses used to re-randomize the commitments to the corresponding i -th variables. Let randomized proof elements and commitments be contained in Π' .

17. Output $(\Pi', x', \text{aux}'_i, r)$.
 - $\text{aux}'_i \leftarrow \text{ShAuxUpd}(\text{crs}, x, \Pi, \text{aux}_i, r_i)$: Parse $r_i = (r_{\tilde{a}_i}, s_{\tilde{a}_i}, r_{\tilde{b}_i}, s_{\tilde{b}_i}, r_{\hat{E}_i}, s_{\hat{E}_i}, r_{\text{bit}_i})$. Parse Π to get commitments $\tilde{\mathbf{d}}_{\tilde{a}_i}, \tilde{\mathbf{d}}_{\tilde{b}_i}, \hat{\mathbf{c}}_{\hat{E}_i}, \hat{\mathbf{c}}_{\text{bit}_i}$. Set $\tilde{\mathbf{d}}'_{\tilde{a}_i} = \tilde{\mathbf{d}}_{\tilde{a}_i} + r_{\tilde{a}_i} \tilde{\mathbf{v}} + s_{\tilde{a}_i} \tilde{\mathbf{w}}, \tilde{\mathbf{d}}'_{\tilde{b}_i} = \tilde{\mathbf{d}}_{\tilde{b}_i} + r_{\tilde{b}_i} \tilde{\mathbf{v}} + s_{\tilde{b}_i} \tilde{\mathbf{w}}$. Then run the same procedure of GS.RandPr but component-wise.
 1. Parse $\text{aux}_i = (\text{aux}_{\mathcal{K}}^i, \text{aux}_{\mathcal{S}^1}^i, \text{aux}_{\mathcal{S}^2}^i, \text{aux}_{\mathcal{E}^1}^i, \text{aux}_{\mathcal{E}^2}^i)$.
 2. Parse $\text{aux}_{\mathcal{K}}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i)$ and compute $\text{aux}_{\mathcal{K}}^{i'} = (\text{aux}'_{\tilde{\pi}_{\tilde{v}}})$ with $\text{aux}'_{\tilde{\pi}_{\tilde{v}}} = \text{aux}_{\tilde{\pi}_{\tilde{v}}} + r_{\text{bit}_i} \tilde{\mathbf{u}}$.
 3. Parse $\text{aux}_{\mathcal{S}^1}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i)$ and compute $\text{aux}_{\mathcal{S}^1}^{i'} = (\text{aux}'_{\tilde{\pi}_{\tilde{v}}}, \text{aux}'_{\tilde{\pi}_{\tilde{w}}})$ with
 - $\text{aux}'_{\tilde{\pi}_{\tilde{v}}} = \text{aux}_{\tilde{\pi}_{\tilde{v}}} + \hat{\mathbf{u}} r_{\tilde{a}_i}$.
 - $\text{aux}'_{\tilde{\pi}_{\tilde{w}}} = \text{aux}_{\tilde{\pi}_{\tilde{w}}} + \hat{\mathbf{u}} s_{\tilde{a}_i}$.
 4. Parse $\text{aux}_{\mathcal{S}^2}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i)$ and compute $\text{aux}_{\mathcal{S}^2}^{i'} = (\text{aux}'_{\tilde{\pi}_{\tilde{v}}}, \text{aux}'_{\tilde{\pi}_{\tilde{w}}}, \text{aux}'_{\tilde{\pi}_{\tilde{w}}})$ with
 - $\text{aux}'_{\tilde{\pi}_{\tilde{v}}} = \text{aux}_{\tilde{\pi}_{\tilde{v}}} + \hat{\mathbf{u}} r_{\tilde{b}_i}$.
 - $\text{aux}'_{\tilde{\pi}_{\tilde{w}}} = \text{aux}_{\tilde{\pi}_{\tilde{w}}} + \hat{\mathbf{u}} s_{\tilde{b}_i}$.
 5. Parse $\text{aux}_{\mathcal{E}^1}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i)$ and compute $\text{aux}_{\mathcal{E}^1}^{i'} = (\text{aux}'_{\tilde{\pi}_{\tilde{v}}}, \text{aux}'_{\tilde{\pi}_{\tilde{w}}}, \text{aux}'_{\tilde{\pi}_{\tilde{w}}}, \text{aux}'_{\tilde{\pi}_{\tilde{w}}})$ with
 - $\text{aux}'_{\tilde{\pi}_{\tilde{v}}} = \text{aux}_{\tilde{\pi}_{\tilde{v}}} + r_{\hat{E}_i} \tilde{\mathbf{d}}'_{\tilde{a}_i}$.
 - $\text{aux}'_{\tilde{\pi}_{\tilde{w}}} = \text{aux}_{\tilde{\pi}_{\tilde{w}}} + s_{\hat{E}_i} \tilde{\mathbf{d}}'_{\tilde{a}_i}$.
 - $\text{aux}'_{\tilde{\pi}_{\tilde{w}}} = \text{aux}_{\tilde{\pi}_{\tilde{w}}} + \hat{\mathbf{c}}_{\hat{E}_i} r_{\tilde{a}_i}$.
 - $\text{aux}'_{\tilde{\pi}_{\tilde{w}}} = \text{aux}_{\tilde{\pi}_{\tilde{w}}} + \hat{\mathbf{c}}_{\hat{E}_i} s_{\tilde{a}_i}$.
 6. Parse $\text{aux}_{\mathcal{E}^2}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i)$ and compute $\text{aux}_{\mathcal{E}^2}^{i'} = (\text{aux}'_{\tilde{\pi}_{\tilde{v}}}, \text{aux}'_{\tilde{\pi}_{\tilde{w}}}, \text{aux}'_{\tilde{\pi}_{\tilde{w}}}, \text{aux}'_{\tilde{\pi}_{\tilde{w}}})$ with
 - $\text{aux}'_{\tilde{\pi}_{\tilde{v}}} = \text{aux}_{\tilde{\pi}_{\tilde{v}}} + r_{\hat{E}_i} \tilde{\mathbf{d}}'_{\tilde{b}_i}$.
 - $\text{aux}'_{\tilde{\pi}_{\tilde{w}}} = \text{aux}_{\tilde{\pi}_{\tilde{w}}} + s_{\hat{E}_i} \tilde{\mathbf{d}}'_{\tilde{b}_i}$.
 - $\text{aux}'_{\tilde{\pi}_{\tilde{w}}} = \text{aux}_{\tilde{\pi}_{\tilde{w}}} + \hat{\mathbf{c}}_{\hat{E}_i} r_{\tilde{b}_i}$.
 - $\text{aux}'_{\tilde{\pi}_{\tilde{w}}} = \text{aux}_{\tilde{\pi}_{\tilde{w}}} + \hat{\mathbf{c}}_{\hat{E}_i} s_{\tilde{b}_i}$.
 - Output $\text{aux}'_i = (\text{aux}_{\mathcal{K}}^{i'}, \text{aux}_{\mathcal{S}^1}^{i'}, \text{aux}_{\mathcal{S}^2}^{i'}, \text{aux}_{\mathcal{E}^1}^{i'}, \text{aux}_{\mathcal{E}^2}^{i'})$.
 - $0/1 \leftarrow \text{ShAuxVerify}(\text{crs}, x, w, \text{aux}_1, \dots, \text{aux}_n, \Pi)$: Parse the proof Π in terms of its commitments and proof elements lists and parse each auxiliary value $\text{aux}_i = (\text{aux}_{\mathcal{K}}^i, \text{aux}_{\mathcal{S}^1}^i, \text{aux}_{\mathcal{S}^2}^i, \text{aux}_{\mathcal{E}^1}^i, \text{aux}_{\mathcal{E}^2}^i)$. For all $i \in [n]$ set the variables $\text{bit}_i, \tilde{a}_i, \tilde{b}_i, \hat{c}_i, \hat{E}_i$ as follows:
 - Set $\mathcal{I} \leftarrow \emptyset$
 - For all $j \in [k]$ let $\phi(j) = i$, set $\text{bit}_i = 1, \tilde{a}_i = \tilde{h}_j, \tilde{b}_i = \tilde{\delta}_j, \hat{E}_i = \hat{e}_j, \mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$.
 - For all $i \in [n] \setminus \mathcal{I}$, set $\text{bit}_i = 0, \tilde{a}_i = \tilde{0}, \tilde{b}_i = \tilde{0}, \hat{E}_i = \hat{0}, \hat{c}_i = \hat{0}$.
- Do the following checks, where all the $\tilde{\mathbf{d}}_{\tilde{a}_i}, \tilde{\mathbf{d}}_{\tilde{b}_i}, \hat{\mathbf{c}}_{\text{bit}_i}$ are taken from the commitment lists in Π (i.e, run the verification of the GS proof system but for individual variables).
1. Equation $\mathcal{K}$: for all $i \in [n]$ parse $\text{aux}_{\mathcal{K}}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i)$ and check that $\hat{\mathbf{c}}_{\text{bit}_i} \tilde{\mathbf{u}} = \hat{\mathbf{v}} \text{aux}_{\tilde{\pi}_{\tilde{v}}} + \text{bit}_i \hat{\mathbf{u}} \tilde{\mathbf{u}}$.
 2. Equation $\mathcal{S}^1$: for all $i \in [n]$ parse $\text{aux}_{\mathcal{S}^1}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i)$ and check that $\hat{\mathbf{u}} \tilde{\mathbf{d}}_{\tilde{a}_i} = \text{aux}_{\tilde{\pi}_{\tilde{v}}} \tilde{\mathbf{v}} + \text{aux}_{\tilde{\pi}_{\tilde{w}}} \tilde{\mathbf{w}} + (0_{\mathbb{T}}, \hat{\mathbf{u}} \tilde{a}_i)$.
 3. Equation $\mathcal{S}^2$: for all $i \in [n]$ parse $\text{aux}_{\mathcal{S}^2}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i)$ and check that $\hat{\mathbf{u}} \tilde{\mathbf{d}}_{\tilde{b}_i} = \text{aux}_{\tilde{\pi}_{\tilde{v}}} \tilde{\mathbf{v}} + \text{aux}_{\tilde{\pi}_{\tilde{w}}} \tilde{\mathbf{w}} + (0_{\mathbb{T}}, \hat{\mathbf{u}} \tilde{b}_i)$.
 4. Equation $\mathcal{E}^1$: for all $i \in [n]$ parse $\text{aux}_{\mathcal{E}^1}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i)$ and check that $\hat{\mathbf{c}}_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{a}_i} = \text{aux}_{\tilde{\pi}_{\tilde{v}}} \tilde{\mathbf{v}} + \text{aux}_{\tilde{\pi}_{\tilde{w}}} \tilde{\mathbf{w}} + \hat{\mathbf{v}} \text{aux}_{\tilde{\pi}_{\tilde{v}}} + \hat{\mathbf{w}} \text{aux}_{\tilde{\pi}_{\tilde{w}}} + (0_{\mathbb{T}}, \hat{E}_i \cdot \tilde{a}_i)$.
 5. Equation $\mathcal{E}^2$: for all $i \in [n]$ parse $\text{aux}_{\mathcal{E}^2}^i = (\text{aux}_{\tilde{\pi}_{\tilde{v}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{w}}}^i)$ and check that $\hat{\mathbf{c}}_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{b}_i} = \text{aux}_{\tilde{\pi}_{\tilde{v}}} \tilde{\mathbf{v}} + \text{aux}_{\tilde{\pi}_{\tilde{w}}} \tilde{\mathbf{w}} + \hat{\mathbf{v}} \text{aux}_{\tilde{\pi}_{\tilde{v}}} + \hat{\mathbf{w}} \text{aux}_{\tilde{\pi}_{\tilde{w}}} + (0_{\mathbb{T}}, \hat{E}_i \cdot \tilde{b}_i)$.
- $(\text{crs}, \text{td}) \leftarrow \text{Sim}_0(\text{gk}, N)$: run $(\text{crs}_1, \text{td}_1) \leftarrow \text{GS.Sim}_0(\text{gk})$. Sample $p_1, \dots, p_N \leftarrow_{\$} \mathbb{Z}_p$, and set $\text{td}_2 = (p_1, \dots, p_N)$. Compute sets $\{\hat{g}_i\} = \{\hat{g} p_i\}, \{\hat{\gamma}_i\} = \{\hat{g} p_i^2\}, \{\hat{h}_i\} = \{p_i \hat{h}\}, \{\hat{\delta}_i\} = \{p_i^2 \hat{\delta}\}$. Set $\text{crs}_2 = (\{\hat{g}_i\}, \{\hat{\gamma}_i\}, \{\hat{h}_i\}, \{\hat{\delta}_i\})$. Return $(\text{crs} = (\text{crs}_1, \text{crs}_2), \text{td} = (\text{td}_1, \text{td}_2))$.
- $(\Pi, \text{aux}_1, \dots, \text{aux}_n) \leftarrow \text{Sim}_1(\text{crs}, \text{td}, x)$: The description of this algorithm is reported in Fig. 19.
- The transformation $\text{AddT} = (\text{AddT}_x, \text{AddT}_w)$ works as follows:
 - $x' \leftarrow \text{AddT}_x(\text{ck}, x, \hat{e}_{k+1}, \alpha, r_{\hat{z}_\alpha}, r_{\tilde{m}_\alpha}; \mathbf{r}')$:
 - * Parse x as $(k, \hat{\mathbf{z}}_1, \dots, \hat{\mathbf{z}}_n, \tilde{\mathbf{m}}_1, \dots, \tilde{\mathbf{m}}_n, \hat{e}_1, \dots, \hat{e}_k)$.
 - * Parse $\mathbf{r}'$ as $(r'_{\hat{z}_1}, \dots, r'_{\hat{z}_n}, r'_{\tilde{m}_1}, \dots, r'_{\tilde{m}_n})$.

- * Replace $\tilde{\mathbf{m}}_\alpha$ with a freshly computed $\tilde{\mathbf{m}}_\alpha = (r_{\tilde{\mathbf{m}}_\alpha} \tilde{h}, \text{bit}_\alpha \tilde{h} + r_{\tilde{\mathbf{m}}_\alpha} \check{\mathbf{c}}\mathbf{k})$. with $\text{bit}_\alpha = 1$.
- * Replace $\hat{\mathbf{z}}_\alpha$ with a freshly computed $\hat{\mathbf{z}}_\alpha = (\hat{g}r_{\hat{\mathbf{z}}_\alpha}, \hat{e}_{k+1} + \hat{\mathbf{c}}\mathbf{k}r_{\hat{\mathbf{z}}_\alpha})$.
- * For all $i \in [n]$, compute $\hat{\mathbf{z}}'_i = \hat{\mathbf{z}}_i + (\hat{g}r'_{\hat{\mathbf{z}}_i}, \hat{\mathbf{c}}\mathbf{k}r'_{\hat{\mathbf{z}}_i})$.
- * For all $i \in [n]$, compute $\tilde{\mathbf{m}}'_i = \tilde{\mathbf{m}}_i + (r'_{\tilde{\mathbf{m}}_i} \tilde{h}, r'_{\tilde{\mathbf{m}}_i} \check{\mathbf{c}}\mathbf{k})$.
- * Set $x' = (k+1, \hat{\mathbf{z}}'_1, \dots, \hat{\mathbf{z}}'_n, \tilde{\mathbf{m}}'_1, \dots, \tilde{\mathbf{m}}'_n, \hat{e}_1, \dots, \hat{e}_{k+1})$.
- $w' \leftarrow \text{AddT}_w(\text{ck}, w, r_{\hat{\mathbf{z}}_\alpha}, \alpha, r_{\tilde{\mathbf{m}}_\alpha}; \mathbf{r}')$
 - * Parse $w = (\phi, r_1, \dots, r_k, r_{\tilde{\mathbf{m}}_1}, \dots, r_{\tilde{\mathbf{m}}_n})$.
 - * Parse $\mathbf{r}'$ as $(r'_{\hat{\mathbf{z}}_1}, \dots, r'_{\hat{\mathbf{z}}_n}, r'_{\tilde{\mathbf{m}}_1}, \dots, r'_{\tilde{\mathbf{m}}_n})$.
 - * For all $i \in [n]$, compute $r_{\tilde{\mathbf{m}}_i}^* = r_{\tilde{\mathbf{m}}_i} + r'_{\tilde{\mathbf{m}}_i}$.
 - * For all $i \in [k]$, compute $r'_i = r_i + r'_{\hat{\mathbf{z}}_{\phi(i)}}$.
 - * Compute $r'_{k+1} = r_{\hat{\mathbf{z}}_\alpha} + r'_{\hat{\mathbf{z}}_\alpha}$.
 - * Compute $\phi' = \phi \cup \{k+1 \rightarrow \alpha\}$.
 - * Set $w' = (\phi', r'_1, \dots, r'_{k+1}, r_{\tilde{\mathbf{m}}_1}^*, \dots, r_{\tilde{\mathbf{m}}_n}^*)$.
- The transformation $\text{ExtT} = (\text{ExtT}_x, \text{ExtT}_w)$ works as follows:
 - $x' \leftarrow \text{ExtT}_x(\text{ck}, x, \hat{\mathbf{z}}_{n+1}, r_{\tilde{\mathbf{m}}_{n+1}}; \mathbf{r}')$
 - * Parse x as $(k, \hat{\mathbf{z}}_1, \dots, \hat{\mathbf{z}}_n, \tilde{\mathbf{m}}_1, \dots, \tilde{\mathbf{m}}_n, \hat{e}_1, \dots, \hat{e}_k)$.
 - * Parse $\mathbf{r}'$ as $(r'_{\hat{\mathbf{z}}_1}, \dots, r'_{\hat{\mathbf{z}}_n}, r'_{\tilde{\mathbf{m}}_1}, \dots, r'_{\tilde{\mathbf{m}}_{n+1}})$.
 - * Compute $\tilde{\mathbf{m}}_{n+1} = (r_{\tilde{\mathbf{m}}_{n+1}} \tilde{h}, r_{\tilde{\mathbf{m}}_{n+1}} \check{\mathbf{c}}\mathbf{k})$.
 - * Compute $\hat{\mathbf{z}}'_i = \hat{\mathbf{z}}_i + (\hat{g}r'_{\hat{\mathbf{z}}_i}, \hat{\mathbf{c}}\mathbf{k}r'_{\hat{\mathbf{z}}_i})$.
 - * Compute $\tilde{\mathbf{m}}'_i = \tilde{\mathbf{m}}_i + (r'_{\tilde{\mathbf{m}}_i} \tilde{h}, r'_{\tilde{\mathbf{m}}_i} \check{\mathbf{c}}\mathbf{k})$.
 - * Set $x' = (k, \hat{\mathbf{z}}'_1, \dots, \hat{\mathbf{z}}'_{n+1}, \tilde{\mathbf{m}}'_1, \dots, \tilde{\mathbf{m}}'_{n+1}, \hat{e}_1, \dots, \hat{e}_k)$.
 - $w' \leftarrow \text{ExtT}_w(\text{ck}, w, r_{\tilde{\mathbf{m}}_{n+1}}; \mathbf{r}')$
 - * Parse $w = (\phi, r_1, \dots, r_k, r_{\tilde{\mathbf{m}}_1}, \dots, r_{\tilde{\mathbf{m}}_n})$.
 - * Parse $\mathbf{r}'$ as $(r'_{\hat{\mathbf{z}}_1}, \dots, r'_{\hat{\mathbf{z}}_n}, r'_{\tilde{\mathbf{m}}_1}, \dots, r'_{\tilde{\mathbf{m}}_{n+1}})$.
 - * For all $i \in [n+1]$, compute $r_{\tilde{\mathbf{m}}_i}^* = r_{\tilde{\mathbf{m}}_i} + r'_{\tilde{\mathbf{m}}_i}$.
 - * For all $i \in [k]$, compute $r'_i = r_i + r'_{\hat{\mathbf{z}}_{\phi(i)}}$.
 - * Set $w' = (\phi, r'_1, \dots, r'_{k+1}, r_{\tilde{\mathbf{m}}_1}^*, \dots, r_{\tilde{\mathbf{m}}_{n+1}}^*)$.

Parse Trapdoor: Parse $\text{td} = (\text{td}_1, \text{td}_2)$ where $\text{td}_1 = (\rho, \sigma)$ and $\text{td}_2 = (p_1, \dots, p_n)$.

Use the trapdoor to create a satisfying assignment: Set variables $\hat{o} = \sum_{i=1}^k p_i \hat{e}_i, \hat{o}' = \sum_{i=1}^k p_i^2 \hat{e}_i$, $(\forall i) \check{a}'_i = \check{a}_i = \check{b}'_i = \check{b}_i = \check{c}_i = \hat{o}, \hat{E}_i = \hat{o}, \text{bit}_i = 0, f_i = 0, r_{\hat{\mathbf{z}}_i} = 0, r_{\tilde{\mathbf{m}}_i} = 0$. To commit to ψ we use the unit commitments with trivial randomness $\hat{\mathbf{u}}$ or $\check{\mathbf{u}}$ already available in crs_1 .

Proof and auxiliary values generation: Compute the proof and the auxiliary values with the above assignment as the prover algorithm would do with the following exceptions:

- Use the value $\psi = 1$ and the trivial commitment randomness while proving equation $\mathcal{P}^1$.
- Use the value $\psi = 0$ with randomness the GS simulation trapdoors (i.e., $(\rho, 0)$ or $(\sigma, 0)$) while proving all the other equations involving ψ .

Fig. 19. Simulator of our extendable shuffle argument.

Theorem 4. *The construction above is an extendable non-interactive proof system for R_{SH} with extended zero knowledge under the SXDH assumption¹³, the subset permutation pairing assumption (Ass. 6), and the subset simultaneous pairing assumption (Ass. 7).*

¹³ Under this assumption GS is non-interactive proof system (Sec. A.3) with a dual-mode CRS.

Set variables: Set the following variables $\forall i \in [n]$:

$r_{z_i}, r_{\tilde{m}_i}, \text{bit}_i, f_i, \tilde{a}_i, \tilde{a}'_i, \tilde{b}_i, \tilde{b}'_i, \hat{c}_i, \hat{E}_i$ and the following other variables $\psi, \hat{o}, \hat{o}'$.

Set equations: Set the following equations:

$$(\forall i) \mathcal{D}_{i,1}^1 : f_i \tilde{z}_i^1 = r_{z_i} \hat{g}, \quad \mathcal{D}_{i,2}^1 : f_i \tilde{z}_i^2 = f_i \hat{E}_i + r_{z_i} \hat{c} \mathbf{k}, \quad \mathcal{F}_i : f_i = \text{bit}_i \psi$$

$$(\forall i) \mathcal{D}_{i,1}^2 : \psi \tilde{\mathbf{m}}_i^1 = r_{\tilde{m}_i} \tilde{h}, \quad \mathcal{D}_{i,2}^2 : \psi \tilde{\mathbf{m}}_i^2 = \text{bit}_i \tilde{h} + r_{\tilde{m}_i} \hat{c} \mathbf{k}$$

$$(\forall i) \mathcal{B}_i : \text{bit}_i(1 - \text{bit}_i) = 0 \quad (\forall i) \mathcal{Q}_i^1 : \tilde{a}_i = \text{bit}_i \tilde{a}'_i \quad (\forall i) \mathcal{Q}_i^2 : \tilde{b}_i = \text{bit}_i \tilde{b}'_i$$

$$\mathcal{K} : \sum_{i=1}^n \text{bit}_i = k \psi \quad \mathcal{S}^1 : \sum_{i=1}^n \tilde{a}_i = \sum_{i=1}^k \psi \tilde{h}_i$$

$$\mathcal{S}^2 : \sum_{i=1}^n \tilde{b}_i = \sum_{i=1}^k \psi \tilde{\delta}_i \quad (\forall i) \mathcal{V}_i^1 : \hat{g} \cdot \tilde{a}_i = \hat{c}_i \cdot \tilde{h} \quad (\forall i) \mathcal{V}_i^2 : \hat{g} \cdot \tilde{b}_i = \hat{c}_i \cdot \tilde{a}_i$$

$$\mathcal{E}^1 : \hat{o} \cdot \tilde{h} + \sum_{i=1}^n \hat{E}_i \cdot \tilde{a}_i = \sum_{i=1}^k \hat{e}_i \cdot \tilde{h}_i \quad \mathcal{E}^2 : \hat{o}' \cdot \tilde{h} + \sum_{i=1}^n \hat{E}_i \cdot \tilde{b}_i = \sum_{i=1}^k \hat{e}_i \cdot \tilde{\delta}_i$$

$$\mathcal{P}^1 : \psi = 1 \quad \mathcal{P}^2 : \psi \hat{o} = \hat{o} \quad \mathcal{P}^3 : \psi \hat{o}' = \hat{o}$$

Map witness to satisfying assignment: For all $i \in [n]$ set the variables $r_{z_i}, r_{\tilde{m}_i}, \text{bit}_i, f_i, \tilde{a}_i, \tilde{a}'_i, \tilde{b}_i, \tilde{b}'_i, \hat{c}_i, \hat{E}_i$ as follows:

- For all $i \in [n]$ set $r_{\tilde{m}_i}$ as the value given in input.
- Set $\mathcal{I} \leftarrow \emptyset$.
- For all $j \in [k]$ let $\phi(j) = i$, set $\tilde{a}'_i = \tilde{a}_i = \tilde{h}_j, \tilde{b}'_i = \tilde{b}_i = \tilde{\delta}_j, \hat{c}_i = \hat{g}_j, \hat{E}_i = \hat{e}_j, \text{bit}_i = 1, f_i = 1, r_{z_i} = r_j, \mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$.
- For all $i \in [n] \setminus \mathcal{I}$, set $\tilde{a}'_i = \tilde{a}_i = \tilde{0}, \tilde{b}'_i = \tilde{b}_i = \tilde{0}, \hat{c}_i = \hat{0}, \hat{E}_i = \hat{0}, \text{bit}_i = 0, f_i = 0, r_{z_i} = 0$.

Set $\psi = 1, \hat{o} = \hat{0}$, and $\hat{o}' = \hat{0}$.

Proof and auxiliary values generation:

1. Generate GS proof elements and GS commitments for all the above equations. Let Π contain all the GS commitments and proofs elements related to each equation. Note that $\hat{\mathbf{u}}$ and $\tilde{\mathbf{u}}$ (available in the `crs`) are already a commitment to 1 with trivial randomness, therefore the prover does not have to compute GS commitments to ψ nor to explicitly prove equation $\mathcal{P}^1$.
2. Define $\text{AUX} = (\text{aux}_i)_{i \in [n]}$, with $\text{aux}_i = (\text{aux}_{\mathcal{K}}^i, \text{aux}_{\mathcal{S}^1}^i, \text{aux}_{\mathcal{S}^2}^i, \text{aux}_{\mathcal{E}^1}^i, \text{aux}_{\mathcal{E}^2}^i)$ where:
 - $\text{aux}_{\mathcal{K}}^i = (\text{aux}_{\tilde{\pi}_{\hat{0}}}^i)$ with $\text{aux}_{\tilde{\pi}_{\hat{0}}}^i = (r_{\text{bit}_i} \tilde{\mathbf{u}}^1, r_{\text{bit}_i} \tilde{\mathbf{u}}^2)$
 - $\text{aux}_{\mathcal{S}^1}^i = (\text{aux}_{\tilde{\pi}_{\hat{0}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{h}}}^i)$ where $\text{aux}_{\tilde{\pi}_{\hat{0}}}^i = (r_{\tilde{a}_i} \hat{\mathbf{u}}^1, r_{\tilde{a}_i} \hat{\mathbf{u}}^2)$, $\text{aux}_{\tilde{\pi}_{\tilde{h}}}^i = (s_{\tilde{a}_i} \hat{\mathbf{u}}^1, s_{\tilde{a}_i} \hat{\mathbf{u}}^2)$.
 - $\text{aux}_{\mathcal{S}^2}^i = (\text{aux}_{\tilde{\pi}_{\hat{0}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{h}}}^i)$ where $\text{aux}_{\tilde{\pi}_{\hat{0}}}^i = (r_{\tilde{b}_i} \hat{\mathbf{u}}^1, r_{\tilde{b}_i} \hat{\mathbf{u}}^2)$, $\text{aux}_{\tilde{\pi}_{\tilde{h}}}^i = (s_{\tilde{b}_i} \hat{\mathbf{u}}^1, s_{\tilde{b}_i} \hat{\mathbf{u}}^2)$.
 - $\text{aux}_{\mathcal{E}^1}^i = (\text{aux}_{\tilde{\pi}_{\hat{0}}}^i, \text{aux}_{\tilde{\pi}_{\hat{0}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{h}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{h}}}^i)$ where $\text{aux}_{\tilde{\pi}_{\hat{0}}}^i = (r_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{a}_i}^1, r_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{a}_i}^2)$, $\text{aux}_{\tilde{\pi}_{\tilde{h}}}^i = (\hat{0}, r_{\tilde{a}_i} \hat{E}_i)$, $\text{aux}_{\tilde{\pi}_{\tilde{h}}}^i = (s_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{a}_i}^1, s_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{a}_i}^2)$, $\text{aux}_{\tilde{\pi}_{\tilde{h}}}^i = (\hat{0}, s_{\tilde{a}_i} \hat{E}_i)$.
 - $\text{aux}_{\mathcal{E}^2}^i = (\text{aux}_{\tilde{\pi}_{\hat{0}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{h}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{h}}}^i, \text{aux}_{\tilde{\pi}_{\tilde{h}}}^i)$ where $\text{aux}_{\tilde{\pi}_{\hat{0}}}^i = (r_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{b}_i}^1, r_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{b}_i}^2)$, $\text{aux}_{\tilde{\pi}_{\tilde{h}}}^i = (0, r_{\tilde{b}_i} \hat{E}_i)$, $\text{aux}_{\tilde{\pi}_{\tilde{h}}}^i = (s_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{b}_i}^1, s_{\hat{E}_i} \tilde{\mathbf{d}}_{\tilde{b}_i}^2)$, $\text{aux}_{\tilde{\pi}_{\tilde{h}}}^i = (0, s_{\tilde{b}_i} \hat{E}_i)$.
3. Output (Π, AUX) .

Fig. 18. Prover algorithm of our extendable shuffle argument.

We prove the above theorem via the following lemmas. The specific assumptions used in each lemma can be deduced from its proofs.

Lemma 13. *Com_{ck} is a perfectly binding and computationally hiding commitment scheme.*

Proof. It trivially follows from the IND-CPA security of the ElGamal encryption.

Lemma 14. *SH is complete (Def. 10).*

Proof. It follows from the completeness of the GS proof system and from the fact that correctly computed auxiliary values satisfy the verification equation checked by AuxVerify.

Lemma 15. *SH has admissible transformations (Def. 11).*

Proof. AddT_x updates the statement by just replacing commitments to bits and elements in position α with freshly computed ones to 1 and freshly added $\hat{e}_{k+1}$ respectively. Then it re-randomizes all the commitments in the statement. Finally, it includes the freshly added $\hat{e}_{k+1}$ in the clear-text elements. AddT_w just updates the opening randomness of the commitments according to the re-randomization factors that were previously used. Finally, it just updates the mapping ϕ to include the newly added $\hat{e}_{k+1}$. Simple inspection and completeness of the re-randomization procedure shows that if $(ck, x, w) \in R_{SH}$ then $(ck, x', w') \in R_{SH}$. An analogous discussion applies to (ExtT_x, ExtT_w).

Lemma 16. *SH is transformation complete (Def. 12).*

Proof. We now show that the first requirement of transformation completeness is always satisfied. The discussion for the second requirement is basically the same. PAdd does the following operations:

- It replaces old GS commitments and proof elements related to equations $\mathcal{D}_{\cdot, \alpha}^1, \mathcal{D}_{\cdot, \alpha}^2, \mathcal{F}_\alpha, \mathcal{B}_\alpha, \mathcal{Q}_\alpha^1, \mathcal{Q}_\alpha^2, \mathcal{V}_\alpha^1, \mathcal{V}_\alpha^2$ with new ones producing accepting GS commitments/proof elements for such equations. This follows from the completeness of GS (steps 1-8).
- It removes the contribution of old committed variables from the proof elements related to equations $\mathcal{K}, \mathcal{S}^1, \mathcal{S}^2, \mathcal{E}^1, \mathcal{E}^2$ and adds the contribution of the freshly committed variables. Then, it re-randomizes the commitments $\hat{z}_i, \hat{m}_i$ with $i \in [n]$ (as AddT_x would do) and adjusts the GS commitments $\hat{d}_{r_{\hat{z}_i}}, \hat{d}_{r_{\hat{m}_i}}$ with $i \in [n]$ to contain the updates randomness. Since in equations $\mathcal{D}_{\cdot, i}^1, \mathcal{D}_{\cdot, i}^2$ the variables $r_{\hat{z}_i}$ and $r_{\hat{m}_i}$ only multiply public constants, there is no need to update any of the related proof elements. The overall process (step 9 to 15) produces accepting GS proofs w.r.t. the new statement x' . This follows from the linearity of the GS proof generation and verification equation.
- Finally, all the GS commitments and proof elements are re-randomized by calling GS.RandPr. This produces accepting proofs w.r.t. x' . This follows from the completeness of GS.

Notice that by construction AddT_x and PAdd produce the same updated statement x' when they are run with the same transformation randomness. Then, AddT_w, as argued before, produces a valid witness for x' . Additionally, AuxUpd just shifts the auxiliary values in the same way the proof elements are shifted by GS.RandPr. As a result, the checks done by AuxVerify will still pass with the updated information as the addition operation exactly matches the computation of a fresh proof over (x', w') .

Lemma 17. *SH is sound (Def. 13).*

Proof. Recall that crs₁ is generated as a perfectly binding string of the GS proof system. Consequently, the proofs related to all equations are perfectly sound proofs certifying that the committed variables satisfy such equations. Recall that Com_{ck} is perfectly binding, and thus there is only one possible satisfying assignment for variables $\hat{E}_i$ and bit_i. Additionally, we can perfectly extract all the variables that are group elements. Therefore, we can extract all the variables $\hat{a}_i, \hat{b}_i, \hat{c}_i, \hat{E}_i$ with $i \in [n]$. Then given such variables, we can compute the set of indexes $\mathcal{J} = \{\alpha_1, \dots, \alpha_k\}$ such that $\sum_{i \in \mathcal{J}} \hat{a}_i = \sum_{i=1}^k \hat{h}_i$ and $\sum_{i \in \mathcal{J}} \hat{b}_i = \sum_{i=1}^k \hat{\delta}_i$. Therefore, $\{\hat{a}_{\alpha_i}\}_{i \in [k]}$ and $\{\hat{b}_{\alpha_i}\}_{i \in [k]}$ satisfy a subset permutation pairing problem meaning that with overwhelming

probability $\{(\check{a}_{\alpha_i}, \check{b}_{\alpha_i})\}_{i \in [k]}$ is a permutation of $\{(\check{h}_i, \check{\delta}_i)\}$. We also have that $\psi = 1$ which implies $\hat{o} = \hat{o}' = \hat{o}$. It follows that $\sum_{i \in \mathcal{J}} \hat{E}_i \cdot \check{a}_i = \sum_{i=1}^k \hat{e}_i \cdot \check{h}_i$ and $\sum_{i \in \mathcal{J}} \hat{E}_i \cdot \check{b}_i = \sum_{i=1}^k \hat{e}_i \cdot \check{\delta}_i$. By rewriting such equations taking into consideration that $\{(\check{a}_{\alpha_i}, \check{b}_{\alpha_i})\}_{i \in [k]}$ is a permutation of $\{(\check{h}_i, \check{\delta}_i)\}$ we have that there exists a permutation π such that $\sum_{i=1}^k (\hat{E}_{\alpha_{\pi(i)}} - \hat{e}_i) \cdot \check{h}_i = 0_{\mathbb{T}}$ and $\sum_{i=1}^k (\hat{E}_{\alpha_{\pi(i)}} - \hat{e}_i) \cdot \check{\delta}_i = 0_{\mathbb{T}}$. This gives us a subset simultaneous pairing problem that implies that, except with negligible probability, $\hat{E}_{\alpha_{\pi(i)}} = \hat{e}_i$ for all $i \in [k]$.

Lemma 18. *SH is addition private (Def. 14).*

Proof. Let us point out the differences between the experiment executions with $b = 0$ and $b = 1$. In both cases, as argued before, the statement x' that is handled to $\mathcal{A}$ is exactly the same. Let us now look at the distribution of (Π, AUX) in both experiment. When $b = 0$ both Π and AUX are freshly computed using the witness provided by $\mathcal{A}$. When $b = 1$ the GS commitments to variables $f_\alpha, \text{bit}_\alpha, \check{a}_\alpha, \check{a}'_\alpha, \check{h}_{k+1}, \check{b}_\alpha, \check{b}'_\alpha, \hat{c}_\alpha, \hat{E}_\alpha$ are freshly computed, as well as the proof elements related to equations $\mathcal{D}_{\cdot, \alpha}^1, \mathcal{D}_{\cdot, \alpha}^2, \mathcal{F}_\alpha, \mathcal{B}_\alpha, \mathcal{V}_\alpha^1, \mathcal{V}_\alpha^2$ that are re-computed running GS.Priv . The distribution of these GS commitments and proof elements is identical in both cases.

The remaining proof elements and GS commitments are taken from Π^* before being updated/re-randomized. The auxiliary values are taken from AUX^* . The list AUX^* contains group elements satisfying the verification equations checked by AuxVerify^{14} . Let us first focus on equations $\mathcal{K}, \mathcal{S}^1, \mathcal{S}^2, \mathcal{E}^1, \mathcal{E}^1$ (i.e, the ones that come with associated auxiliary values). It is straightforward to see that after all the GS commitments related to the variables involved in such equations have been re-randomized by GS.RandPr , we get new commitments that are equally distributed by freshly generated ones. Auxiliary values are updated as random group elements, using the same randomness previously used to re-randomize the commitments. As already shown in previous works [38,11,23], re-randomized proof elements are distributed as randomly chosen proof elements from the space of all valid proof elements, given that the GS commitments to the involved variables are fixed. Since the GS commitments are fully re-randomized, the result is a randomly chosen proof given a fixed solution, along with auxiliary values related to the randomly chosen commitments so that the verification check of AuxVerify is successful. It follows that, the joint distribution of GS commitments, proofs elements, and AUX is identically distributed both when $b = 0$ and $b = 1^{15}$. Finally, for all the other equations not involving such variables it suffices to reduce to the re-randomization of GS proofs.

Lemma 19. *SH is extension private (Def. 15).*

Proof. The proof basically mirrors the one of addition privacy.

Lemma 20. *SH is extended zero knowledge (Def. 20).*

Proof. We first change the way crs_1 is generated by generating it as a perfectly hiding string of the GS proof system. By the SXDH assumption perfect binding and perfect hiding common reference strings for the GS proof system are computationally indistinguishable, so the adversary's success probability only changes negligibly. The sole distinction between an actual proof and a simulated proof lies in the witness provided to the GS prover. Due to the perfect witness indistinguishability of the GS proof over a perfectly hiding crs , genuine proofs and simulated proofs are indistinguishable. It remains to argue that the auxiliary values $\mathcal{A}$ receives do not give $\mathcal{A}$ any advantage in distinguishing honest and simulated proofs. It is straightforward to notice that also the auxiliary values are identically distributed in both cases. Indeed, the variables $\text{bit}_i, \check{a}_i, \check{b}_i, \hat{E}_i, \hat{c}_i$ for $i \in [n] \setminus \text{Im}(\phi)$ are set to neutral elements in both cases, and thus the auxiliary values related to such variables just allow $\mathcal{A}$ to verify that they are set as expected. Moreover, such auxiliary values are completely independent to the values (and GS commitment openings) of the above variables for indexes $j \in \text{Im}(\phi)$, which is the only difference between honest and simulated proofs.

¹⁴ We are guaranteed of that since otherwise $\mathcal{A}$ would not be admissible.

¹⁵ In particular, notice that after the invocation of PAdd , the proofs for all equations are accepting.

C The EP used in our $\mathfrak{Tetris}$

In this section, we describe our instantiation of the EP used in our $\mathfrak{Tetris}$. It uses similar techniques to our extendable shuffle argument and the ENIWI of [8]. We set $\mathcal{L}_{\text{trap}}$ to be the language of DH tuples, and let $x_{\text{trap}} = (\hat{a}, \hat{b}, \hat{c})$. The relation of our EP is defined below.

$$R_{\text{Dtr}} = \{(\text{ck}, x = (k, x_1, \dots, x_n, \tilde{\mathbf{m}}_1, \dots, \tilde{\mathbf{m}}_n), w = (\phi, w_1, \dots, w_k, r_{\tilde{\mathbf{m}}_1}, \dots, r_{\tilde{\mathbf{m}}_n})) \mid$$

$$\forall i \in [n] \exists \text{bit}_i : \tilde{\mathbf{m}}_i \leftarrow \text{Com}_{\text{ck}}(\text{bit}_i; r_{\tilde{\mathbf{m}}_i}) \wedge \text{bit}_i \in \{0, 1\} \wedge \sum_{i=1}^n \text{bit}_i = k \wedge$$

$$\forall_{i=1}^k (\text{bit}_{\phi(i)} = 1 \wedge (\text{ck}, x_{\phi(i)}, w_i) \in R_{\text{D}}) \wedge \phi \text{ is injective}\},$$

where

$$R_{\text{D}} = \{(\text{ck}, x = (\hat{z}, \text{pk}_{\text{T}}, \hat{\tau}, m, x_{\text{trap}}), w) \mid (w = (\hat{e}, r_{\hat{z}}, \text{sk}_{\text{T}}) \wedge e \leftarrow \text{Tag}(\text{sk}_{\text{T}}, \hat{\tau}, m) \wedge$$

$$\hat{z} \leftarrow \text{Com}_{\text{ck}}(\hat{e}; r_{\hat{z}}) \wedge (\text{pk}_{\text{T}}, \text{sk}_{\text{T}}) \in R_{\text{key}}) \vee (w = w_{\text{trap}} \wedge (x_{\text{trap}}, w_{\text{trap}}) \in R_{\mathcal{L}_{\text{trap}}})\}.$$

In Fig. 20 we report the prover algorithm of our EP.

As all of the other algorithms are almost identical to the ones presented in SH, we just briefly describe the other algorithms of our EP:

- **CRSSetup** just runs the setup algorithm of GS.
- The commitment scheme is the same used in SH.
- The transformations (**AddT**, **ExtT**) are identical to the ones of SH, except they do not consider any clear-text element.
- **PVfy** just runs the verifier algorithm of GS.
- **PExt** works similarly to **ShExtend**. It creates new variables, GS commitments, and proof elements for all equations but $\mathcal{K}$. It computes the new auxiliary value $\text{aux}_{\mathcal{K}}^{n+1}$ and updates the proof elements related to $\mathcal{K}$ in the same way of **ShExtend**. It re-randomizes the statement and adjusts the related GS commitments as in step 13 of **ShExtend** and runs **GS.RandPr**.
- **PAdd** works similarly to **ShAdd**. It replaces all the variables in position α with fresh ones re-computing the related GS commitments and the proof elements of the corresponding equations, except for $\mathcal{K}$. It updates the proof for $\mathcal{K}$ in the same way **ShAdd** does. It re-randomizes the statement and adjusts the related GS commitments in the same way **ShAdd** does.
- **AuxUpd** and **AuxVerify** work in the same way as the corresponding algorithms of SH. The only difference is that here only equation $\mathcal{K}$ is involved in the process.

Proof Intuition. Given the similarity with SH and with the ENIWI of [8] here we give a proof sketch of why EP is an extendable non-interactive proof system with WAI (Def. 17), FPWI (Def. 18), and EWI (Def. 19), under the SXDH assumption. We focus on WAI, FPWI, and EWI, as the other properties can be proven with a discussion that basically mirrors the one already done for SH.

1. **Witness addition indistinguishability:** It suffices to observe that x' is identical in both cases. The distribution of the re-randomization factors is identical. The only difference is that two different witness are used to perform an addition over the same position α , but with a different witness. The distribution of the auxiliary values is also identical in both executions, as after re-randomization it only depends on the active indexes, that are identical in both cases. It remains to argue that the distribution of Π is indistinguishable in both executions. This basically from the re-randomization property of GS, as two accepting GS proofs are indistinguishable after their re-randomization.
2. **Fixed position witness indistinguishability:** Since the active indexes are the same in both cases, it follows that the auxiliary values are identically distributed, as they are independent from the used witness. Therefore to argue that the two experiment executions are indistinguishable it suffices to reduce to the witness indistinguishability of GS.

Set variables: Define the following variables $\forall i \in [n] : r_{\hat{z}_i}, \hat{E}_i, r_{\hat{m}_i}, \text{bit}_i, f_i, s_i^0, s_i^1, \text{sk}_T^i, a_i, b_i$. Finally, define the variable ψ .

Set equations:

$$\begin{aligned}
(\forall i) \mathcal{D}_{i,1}^1 : s_i^0 \hat{z}_i^1 &= \hat{\text{ck}} r_{\hat{z}_i} \quad (\forall i) \mathcal{D}_{i,2}^1 : s_i^0 \hat{z}_i^2 = s_i^0 \hat{E}_i + \hat{g} r_{\hat{z}_i} \\
(\forall i) \mathcal{D}_{i,1}^2 : \psi \hat{\mathbf{m}}_i^1 &= r_{\hat{m}_i} \hat{\text{ck}} \quad (\forall i) \mathcal{D}_{i,2}^2 : \psi \hat{\mathbf{m}}_i^2 = \text{bit}_i \hat{h} + r_{\hat{m}_i} \hat{h} \\
(\forall i) \mathcal{F}_i : f_i(1 - f_i) &= 0 \quad (\forall i) \mathcal{B}_i : \text{bit}_i(1 - \text{bit}_i) = 0 \quad (\forall i) \mathcal{U}_i^1 : s_i^0 = \text{bit}_i(1 - f_i) \\
(\forall i) \mathcal{U}_i^2 : s_i^1 &= \text{bit}_i f_i \quad (\forall i) \mathcal{T}_i : s_i^0 \hat{E}_i = \hat{r} \text{sk}_T^i + \hat{\text{pk}}_2^i \text{msk}_T^i \quad (\forall i) \mathcal{Y}_i : s_i^0 \hat{\text{pk}}_1^i = \hat{g} \text{sk}_T^i \\
(\forall i) \mathcal{M}_i^1 : s_i^1 \hat{a} &= \hat{g} a_i \quad (\forall i) \mathcal{M}_i^2 : s_i^1 \hat{b} = \hat{g} b_i \quad (\forall i) \mathcal{M}_i^3 : b_i \hat{a} = \hat{c} s_i^1 \\
\mathcal{K} : \sum_{i=1}^n \text{bit}_i &= k\psi \quad \mathcal{P} : \psi = 1.
\end{aligned}$$

Map witness to satisfying assignment: For all $i \in [n]$ set the variables as follows:

- For all $i \in [n]$ set $r_{\hat{m}_i}$ as the value given in input.
- Set $\mathcal{I} \leftarrow \emptyset$.
- For all $j \in [k]$ let $\phi(j) = i$, set $r_{\hat{z}_i} = r_j, \hat{E}_i = \hat{e}_j, \text{bit}_i = 1, f_i = 0, s_i^0 = 1, s_i^1 = 0, \text{sk}_T^i = \text{sk}_T^j, a_i = 0, b_i = 0$
 $\mathcal{I} \leftarrow \mathcal{I} \cup \{i\}$.
- For all $i \in [n] \setminus \mathcal{I}$, set $r_{\hat{z}_i} = 0, \hat{E}_i = \hat{0}, \text{bit}_i = 0, f_i = 0, s_i^0 = 0, s_i^1 = 0, \text{sk}_T^i = 0, a_i = 0, b_i = 0$.

Set $\psi = 1$. Notice that variable f_i can be set to 1 whenever the prover wants to use a witness for x_{trap} .

Proof and auxiliary values generation:

1. Generate GS proof elements and GS commitments for all the above equations and put them in Π .
2. Define $\text{AUX} = (\text{aux}_i)_{i \in [n]}$, with $\text{aux}_i = \text{aux}_{\mathcal{K}}^i$, where $\text{aux}_{\mathcal{K}}^i = (\text{aux}_{\hat{\pi}_{\hat{\Theta}}})$ with $\text{aux}_{\hat{\pi}_{\hat{\Theta}}} = (r_{\text{bit}_i} \hat{\mathbf{u}}^1, r_{\text{bit}_i} \hat{\mathbf{u}}^2)$.

Fig. 20. Proving algorithm of our EP.

3. **Extended witness indistinguishability:** We sketch a sequence of indistinguishable hybrids that go from the experiment execution with $b = 0$ to the experiment execution with $b = 1$. We first switch the crs of the GS proof system to a hiding crs generated with the trapdoor (computational indistinguishability of the crs). We then use the trapdoor to prove all the equations except $\mathcal{P}$ by setting $\psi = 0$ as well as all the other variables (witness indistinguishability of GS). Then, we compute the commitments $\tilde{\mathbf{m}}_i$ with $i \in [n]$ with the bits induced from ϕ^1 (hiding of Com_{ck}). Finally, we compute the proof using w_1 (witness indistinguishability of GS).

Lemma 21. *Our EP achieves perfect Δ -extraction (Def. 16), where Δ is defined as follows on input $w_1 \dots, w_k$, where $w_i = (\hat{e}_i, r_{\hat{z}_i}, \text{sk}_T^i)$:*

$$\Delta(w_1 \dots, w_k) = \begin{cases} \perp & \text{if } \exists i \in [k] : (x_{\text{trap}}, w_i) \in R_{\mathcal{L}_{\text{trap}}} \\ (\hat{e}_1, \dots, \hat{e}_k) & \text{otherwise} \end{cases}$$

Proof. Let $\text{ck} \leftarrow \text{ComSetup}(\text{gk})$, and $(\text{crs}, \text{td}) \leftarrow \text{CRSSetup}(\text{gk})$ and let GS.Ext be the extractor of GS [28]. We define the following extractor Ext that takes in input a statement $(\text{crs}, \text{td}, \text{ck}, x = (k, x_1, \dots, x_n, \tilde{\mathbf{m}}_1, \dots, \tilde{\mathbf{m}}_n), \Pi_{\text{EP}})$, with $x_i = (\hat{z}_i, \text{pk}_T^i, \hat{\tau}, m, x_{\text{trap}})$.

Notice that the CRS of our EP is generated as in the binding mode of GS, so thanks to the perfect F -knowledge of GS, our Ext can run GS.Ext on input the proof, extracting the group elements and the group-element representation (i.e., exponentiation in either $\mathbb{G}$ or $\mathbb{H}$) of the scalars that are in the witness $w = (\phi, w_1, \dots, w_k, r_{\tilde{\mathbf{m}}_1}, \dots, r_{\tilde{\mathbf{m}}_n})$ for x . That is, for each $i \in [n]$, GS.Ext extracts and $\text{bit}_i = \tilde{h}\text{bit}_i$. Ext computes ϕ' as follows:

```

c ← 1
for j ∈ [k]:
    flag = 0
    while c < n ∧ flag = 0:
        if bit_c = h̃:
            φ'(j) ← c
            flag = 1
        c ← c + 1
return φ'

```

Additionally, GS.Ext can extract $f_{\phi'(j)}\hat{g}$ for all $j \in [k]$. If one of those extracted values is $\hat{g}$, Ext returns $(\phi', \perp)$. Indeed, this means that w_{trap} was used in at least one of the active indices. Otherwise, Ext uses GS.Ext to extract the committed variables $\hat{E}_{\phi'(j)}$ for $j \in [k]$. Ext returns $(\phi', \hat{E}_{\phi'(1)}, \dots, \hat{E}_{\phi'(k)})$.

Remark 2. The relation R_{key} of our DAPT (Sec. 7) contains a part that it is publicly verifiable that simply consists in checking that a pairing equation holds between the two group elements in the public key. Therefore, we do not include this check in the equation proved by the prover and assume it is verified by the verifier for every public key included in the statement.